

Mishkaat

المشكاة

A Multi-Tenant Management, Analytics and Forecasting Platform
for Qur'an Memorisation Centres

SYSTEM NAME	Mishkaat (المشكاة) — Qur'an Centre Management System
APPLICATION VERSION	2.1.0
DOCUMENT TYPE	Full technical, architectural and scientific documentation
DOCUMENT VERSION	1.0 — first consolidated edition
DATE OF ISSUE	11 September 2026
REPOSITORY	asse-byte/mishkaat
DEVELOPMENT PERIOD	7 May 2026 – 10 September 2026 (63 commits)
TARGET DEPLOYMENT	Qur'an memorisation centres, Republic of Mali (currency: FCFA)
PRIMARY INTERFACE LANGUAGE	Arabic (right-to-left)
AUTHOR & RIGHTS HOLDER	Abdoul Malick Cisse
CLASSIFICATION	Proprietary and confidential — all rights reserved

Copyright © 2026 Abdoul Malick Cisse. All rights reserved.

This document describes a proprietary and confidential system. It handles the personal data of minors; every recommendation in Chapter 10 (Security Engineering) and Chapter 14 (Deployment & Operations) should be read as a precondition to production use, not as an optional hardening exercise.

Abstract

Mishkaat (المشكاة, “the niche that holds the lamp”) is a production-oriented, multi-tenant web platform for the administration and academic supervision of Qur'an memorisation centres (دور تحفيظ القرآن). It replaces the paper registers, isolated spreadsheets and oral reporting on which such centres traditionally depend, with a single auditable system covering six user roles, eleven functional domains and a full analytical layer.

The system is built as a three-tier application: a React 19 / TypeScript single-page front end (installable as a Progressive Web App), a FastAPI / Python asynchronous REST back end exposing **136 endpoints across 22 routers**, and a MongoDB document store of **28 application collections** plus a GridFS binary store, governed by **71 index definitions** created at application start-up.

Beyond conventional record keeping, Mishkaat contributes an *analytical engine* that converts raw recitation and attendance records into interpretable academic signals: a four-class weighted error taxonomy, five per-student performance metrics computed at read time (mastery, momentum, precision, consistency and review depth), a multi-feature ordinary-least-squares model that forecasts Juz' and full-Qur'an completion dates from memorisation *quality* as well as quantity, and a multi-criteria motivation layer of four independent leaderboards, ten achievement badges, four periodic challenges and an append-only experience-point ledger.

Security is treated as a first-class functional requirement rather than a deployment concern, because the platform stores identifying data about children. The report documents a layered control set: short-lived access tokens with refresh-token rotation in an `httpOnly` cookie, a token revocation list with `user_version` invalidation, dual-key (per-IP and per-account) rate limiting, Fernet field-level encryption of personally identifying student attributes at rest, a SHA-256 hash-chained audit log with atomic sequencing, content-sniffing file-upload validation, and a single-gate authorisation model whose scope boundaries — system, centre, *حلقة* (study circle) and self — are defined in one module and asserted by **314 isolation assertions** across seven test suites.

This document specifies the problem domain, the delimited scope, the stakeholder model, the functional and non-functional requirements, the architecture, the complete data dictionary and relationship model, the API surface, the mathematics of the analytical engine, the security architecture, the business workflows, the user interface, the verification strategy, the operational topology, and the system's measured limitations and forward work.

Keywords

Qur'an memorisation management · educational management information system (EMIS) · multi-tenant SaaS · role-based access control · broken object level authorisation (BOLA) · learning analytics · completion forecasting · ordinary least squares regression · gamification · tamper-evident audit logging · field-level encryption · FastAPI · MongoDB · React · Progressive Web App · Arabic RTL interfaces.

At a glance

136 REST ENDPOINTS	22 API ROUTERS	28+2 DB COLLECTIONS (+ GRIDFS)
71 INDEX DEFINITIONS	6 USER ROLES	10,390 BACK-END LINES (PYTHON)

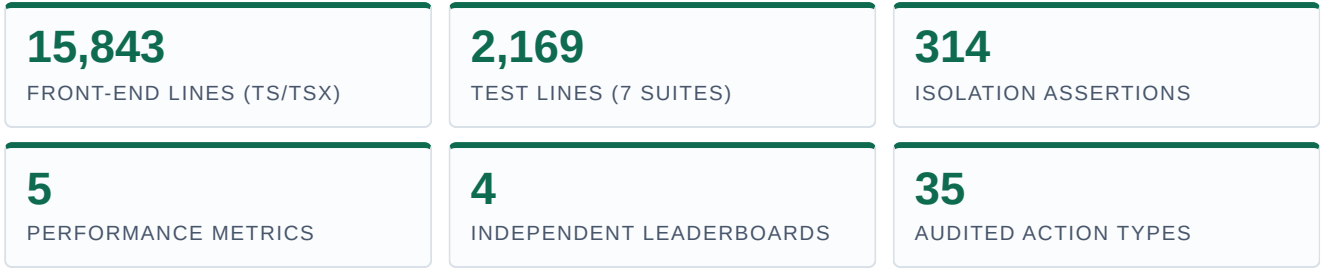


Table of Contents

1	Introduction	8
1.1	Project identification	
1.2	Background and motivation	
1.3	Problem statement	
1.4	Aim and objectives	
1.5	The proposed solution	
1.6	Contributions of the work	
1.7	Organisation of this report	
2	Scope, Assumptions and Delimitations	13
2.1	Functional scope (in scope)	
2.2	Out of scope	
2.3	Deliberate exclusions and their rationale	
2.4	Assumptions	
2.5	Constraints	
2.6	Operating environment	
3	Stakeholders and Users	18
3.1	User classes and characteristics	
3.2	The scope-resolution chain	
3.3	Role capability matrix	
3.4	The teacher: what is granted and what is withheld	
3.5	The communication chain	
3.6	Deliberate exceptions to the scope rules	
4	Requirements Specification	23
4.1	Requirement identification scheme	
4.2	Functional requirements	
4.3	Non-functional requirements	
4.4	Business rules	
4.5	Principal use cases	
5	System Architecture	32
5.1	Architectural style and rationale	
5.2	Logical view	
5.3	Deployment view	
5.4	Back-end module inventory	
5.5	Router inventory	
5.6	Front-end structure	

5.7	Request lifecycle	
5.8	Recurring design patterns	
6	Technology Stack	39
6.1	Back-end technologies	
6.2	Front-end technologies	
6.3	Infrastructure and operations	
6.4	Development tooling	
6.5	Dependency policy	
7	Database Design and Data Dictionary	42
7.1	Choice of a document store	
7.2	Collection inventory	
7.3	Entity-relationship model	
7.4	Relationship catalogue	
7.5	Data dictionary — core entities	
7.6	Data dictionary — academic records	
7.7	Data dictionary — financial records	
7.8	Data dictionary — competitions and certificates	
7.9	Data dictionary — analytics and motivation	
7.10	Data dictionary — communication and governance	
7.11	Data dictionary — security and ephemeral state	
7.12	Indexing strategy	
7.13	Integrity, retention and deletion semantics	
8	API Specification	62
8.1	Conventions	
8.2	Endpoint catalogue	
8.3	Error model	
9	The Analytical Engine	69
9.1	Provenance and scientific basis	
9.2	The weighted error taxonomy	
9.3	The five performance metrics	
9.4	Completion forecasting	
9.5	The motivation layer	
9.6	Derived instruments	
9.7	Invariant design decisions	
9.8	Calibration parameters	
10	Security Engineering	78
10.1	Data sensitivity and threat model	
10.2	Authentication and session lifecycle	

10.3	Authorisation architecture	
10.4	Credential policy and recovery	
10.5	Rate limiting and lockout	
10.6	Personal data protection at rest	
10.7	Tamper-evident audit logging	
10.8	File upload security	
10.9	Transport and HTTP hardening	
10.10	Fail-fast production guards	
10.11	Residual risks	
11	Business Workflows	85
11.1	Centre onboarding and approval	
11.2	Enrolment and account issuance	
11.3	The daily recitation cycle	
11.4	Attendance and absence notification	
11.5	The competition lifecycle	
11.6	Certificate issuance	
11.7	Financial operations and the void policy	
11.8	Messaging	
11.9	Idempotency guards	
12	User Interface and Interaction Design	90
12.1	Design language	
12.2	Route map and client-side guards	
12.3	Role dashboards	
12.4	Component inventory	
12.5	Progressive Web App delivery	
12.6	Printing and document output	
13	Verification and Quality Assurance	93
13.1	Testing philosophy	
13.2	Test suite inventory	
13.3	Representative invariants	
13.4	Static quality gates	
13.5	Coverage gaps	
14	Deployment and Operations	97
14.1	Deployment topologies	
14.2	Configuration reference	
14.3	Backup and disaster recovery	
14.4	Health and observability	
14.5	Centre onboarding checklist	

15	Project Metrics and Development History	100
16	Evaluation, Limitations and Future Work	102
17	Conclusion	105
—	References	106
A–D	Appendices	107
A	Environment variable reference	
B	Audit action catalogue	
C	Complete endpoint index	
D	Glossary of domain terms	

Introduction

1.1 Project identification

The system documented here is named **Mishkaat** — Arabic المشكاة, the niche in a wall that holds a lamp. The name is taken from the Qur'anic image of light held in a recess so that it illuminates the room around it; the intent of the naming is that the software should hold and make visible what a memorisation circle is already doing, rather than change the pedagogy itself.

Table 1.1 — Project identification

Attribute	Value
System name	Mishkaat (المشكاة)
Domain	Educational management information system for Qur'an memorisation centres
Application version	2.1.0 (<code>APP_VERSION</code> in <code>backend/app/config.py</code>)
Architecture class	Multi-tenant, three-tier web application delivered as a Progressive Web App
Deployment geography	Republic of Mali; monetary values default to the West African CFA franc (FCFA)
Interface language	Arabic, right-to-left, single-language by deliberate decision (see §2.3)
Licence	Proprietary and confidential; all rights reserved to Abdoul Malick Cisse, 2026
Source repository	<code>asse-byte/mishkaat</code> ; default branch <code>main</code>

1.2 Background and motivation

A Qur'an memorisation centre (مركز تحفيظ or دار تحفيظ) is an educational institution organised around *halaqāt* (حلقات, singular حلقة) — study circles in which a small group of students sits with one teacher, the *muhaqqif* (محقق, literally “the one who causes memorisation”). The core daily event is *tasmī* (تسميع): a student recites a passage from memory and the teacher listens, corrects, and forms a judgement of the quality of the recitation.

This model has three characteristics that make it difficult to administer at scale:

1. **The primary record is oral and ephemeral.** The teacher's judgement of a recitation — where the student hesitated, what kind of error occurred, whether the passage is secure — exists in the teacher's memory and, at best, in a shorthand mark in a paper notebook. It is not comparable across students, across teachers, or across time.
2. **Responsibility is distributed but accountability is centralised.** The teacher owns the circle; the centre manager answers for the centre's academic results and its finances; the parent pays fees and expects to know whether the child is progressing. These three parties rarely see the same information.
3. **The unit of progress is contested.** “How much has the student memorised?” is usually answered in *ajzā* (أجزاء, the thirty parts of the Qur'an), but two students who have both “finished five parts” may differ enormously in the security of that memorisation. Quantity is recorded; quality is not.

The consequence is a documented set of operational failures: enrolment registers that drift out of date, fees that are collected twice or not at all, teachers' salaries paid twice in the same month because nothing records that they were already paid, absence that parents learn about weeks later, and — most importantly for the academic mission — no defensible answer to the question “*when will this child finish, and is he progressing or only attending?*”

1.3 Problem statement

The problems Mishkaat is designed to solve are enumerated below. Each is stated as an observed failure of the status quo, paired with the design response that the system makes. The pairing is important: each mechanism in the system exists to answer a specific named failure, and Chapter 4 traces each back to a numbered requirement.

Table 1.2 — Problem–response matrix

#	Observed problem	Consequence in practice	Response in Mishkaat
P1	Recitation quality is unrecorded and incomparable	A teacher's note “made mistakes” cannot be compared between two students, nor for one student over a term. Free-text error notes are written differently by every teacher.	A closed, four-class weighted error taxonomy (§9.2). The teacher tags each error by type on a phone during the circle; the severity weight lives in one place in the code and is never stored alongside the error.
P2	Progress is measured only by quantity	Two students with identical page counts receive the same projected completion date, even when one misses half the sessions and errs on every page.	A three-feature regression model (momentum, weighted error density, attendance rate) trained on the centre's own history, so that recitation quality affects the projected date (§9.4).
P3	A single ranking demotivates exactly those who need encouragement	A student who appears last on one composite leaderboard every week learns to ignore it entirely.	Four independent leaderboards — momentum, precision, consistency, review depth — so that the slow student who never misses a session leads the consistency board (§9.5).
P4	Everyone in a centre could see everyone's data	In the pre-حلقة-scoped version of this system, a circle teacher could read every student in the centre — names, guardians' phone numbers, recitations, attendance, performance metrics, projected completion dates and fee records — and could log a recitation for a student who was not his. Seventeen distinct leaks from one root cause.	A four-level scope model (system / centre / circle / self) defined in a single module, <code>app/scope.py</code> , that every endpoint queries; plus one mandatory ownership gate, <code>check_student_access</code> , for any endpoint taking a <code>student_id</code> (§10.3).
P5	Administrative acts could be performed twice	Double-clicking “save” produced two recitations for the same day, two salary payments for the same month, two fee records for the same month. The duplicate salary is real money leaving twice; the duplicate recitation corrupts every derived metric and forecast.	Three uniqueness guards enforced with HTTP 409 and, where possible, with unique database indexes rather than code-level checks (§11.9).
P6	Every administrative act is fallible, but nothing could be corrected	A fee entered against the wrong student, a salary with a wrong amount, an attendance mark on the wrong child — with no correction path, staff work around the system and the records stop reflecting reality.	Edit and cancel paths for every administrative record, with a strict distinction: money is <i>voided</i> , never erased — marked <code>voided</code> with a written reason and an actor, and excluded from every total (§11.7).

#	Observed problem	Consequence in practice	Response in Mishkaat
P7	Absence notification is a broadcast into the void	An absence alert delivered only over a live connection reaches a parent who happens to be looking at the application at 06:30 on a weekday, i.e. almost never.	Every notification is persisted to the database first and streamed second, so the database is the single source of the stream and a notification survives both worker restarts and offline parents (§11.4).
P8	Financial and permission history was not defensible	Where a dispute arises over a payment or a permission change, an ordinary log can be edited by anyone with database access.	A SHA-256 hash-chained audit log with atomic sequence numbers, explicit gap marking, and a permanent-record class of actions that is never purged (§10.7).
P9	Supervisory bodies cannot tell an active centre from a large one	A centre that enrolled a hundred students and then abandoned the system outranks a centre with twenty students working daily, because the ranking counts head-count.	A centre activity index that measures transactions per student over a time window rather than absolute size (§9.6).
P10	Review (المراجعة) was unmanaged	A single hard-coded weekly review table was displayed to every student regardless of how much each had memorised, and its “done” marker compared a Juz' number to an āyah number — so reviewing verse 3 of any sūrah marked Saturday complete, permanently.	A daily review portion derived per student from what that student has actually memorised, at a rate the teacher sets, computed from the date rather than from stored state (§9.6).

1.4 Aim and objectives

Aim. To deliver a single, auditable, multi-tenant system through which a Qur'an memorisation centre can run its academic, administrative and financial operations, and through which the academic quality of memorisation — not merely its quantity — becomes measurable, comparable and explainable to the people who need it: the teacher, the manager, the student and the parent.

Table 1.3 — Objectives

ID	Objective	Realised in
01	Provide complete record management for centres, teachers, students, circles, schedules, attendance, recitation, fees, salaries and expenses.	Ch. 4, 7, 8
02	Enforce data isolation between tenants (centres) and between circles inside a tenant, with authorisation decided in one place and provable by test.	Ch. 3, 10, 13
03	Convert recitation records into interpretable academic metrics that separate quality from quantity.	§9.2, §9.3
04	Forecast Juz' and full-Qur'an completion dates from a model trained on the centre's own history, and disclose the method, sample size and fit with every prediction.	§9.4
05	Motivate students through multiple independent achievement axes rather than a single ranking, backed by an append-only ledger.	§9.5
06	Protect the personal data of minors at rest and in transit, and make financial and permission history tamper-evident.	Ch. 10
07	Make every administrative act correctable without allowing financial history to be erased.	§11.7

ID	Objective	Realised in
08	Deliver an Arabic, right-to-left, installable interface usable on a low-cost phone by a teacher who is not a computer user.	Ch. 12
09	Ship a reproducible, hardened deployment with backups, health checks and an onboarding procedure.	Ch. 14

1.5 The proposed solution

Mishkaat is a three-tier web application. A React single-page application, installable as a Progressive Web App, communicates over HTTPS with a FastAPI back end; the back end alone speaks to a MongoDB cluster that is never exposed to the network. The application layer is decomposed by *domain* — one router per business area — over a set of cross-cutting modules that hold the decisions no single domain should own: authentication, authorisation scope, personal-data encryption, audit chaining, performance metrics, forecasting and motivation.

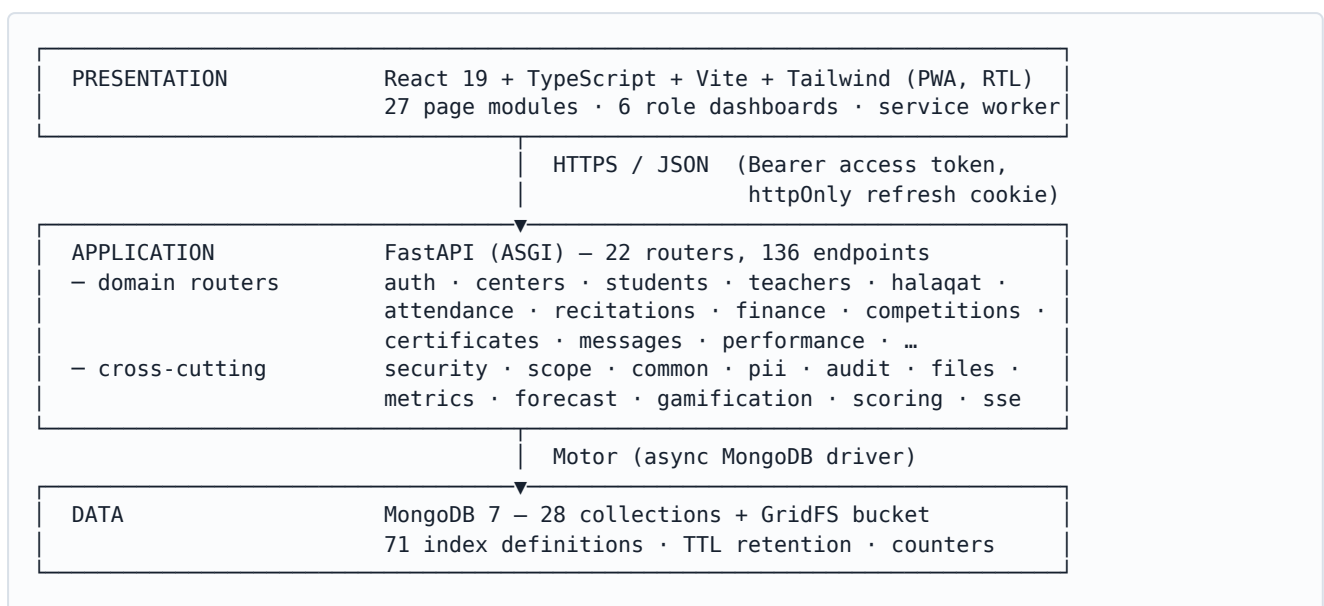


Figure 1.1 — Solution overview at the tier level. Chapter 5 expands each tier.

Two design commitments distinguish this system from a conventional school-administration product and recur throughout this report:

COMMITMENT 1 — DERIVED VALUES ARE NEVER STORED

Every performance metric, every leaderboard position and every forecast is computed at read time from the recitation and attendance records that produced it. A stored derived value drifts away from its source rows at the first correction or void, at which point a student's leaderboard position can no longer be explained by pointing at his sessions — and explicability is the property on which the whole motivation layer depends.

COMMITMENT 2 — AUTHORISATION IS DECIDED IN ONE PLACE

The boundaries of what each role may see are defined once, in `app/scope.py`, and every endpoint asks that module rather than re-deriving the rule. The alternative — patching each endpoint — was tried and failed: the leak returned with the next endpoint added. Correspondingly, an empty scope returns `None`, never `{}`, because an empty MongoDB filter means “everything”, which is the exact inversion of the intent and the kind of bug that looks correct while it leaks.

1.6 Contributions of the work

1. **A weighted, closed error taxonomy for recitation.** Four error classes with numeric severity weights, storing the class code and never the weight, so that recalibrating the weights retroactively corrects all historical data (§9.2).
2. **Quality-sensitive completion forecasting.** A model in which the projected date responds to how well a student recites, not only how much — with domain sign constraints applied to the fitted coefficients so that the system can never tell a parent that their child's errors are accelerating his completion (§9.4).
3. **Multi-criteria motivation.** Four separate leaderboards derived from the same session records as the metrics, with an append-only, idempotent experience-point ledger (§9.5).
4. **A single-source authorisation model with a proving test suite.** Four scope levels resolved through one documented chain, guarded by 314 assertions that test for *leakage of other people's data* rather than for request success (Ch. 3, 13).
5. **A tamper-evident administrative record.** Atomic sequencing, hash chaining, explicit gap marking and a never-purged class for money and permission events (§10.7).
6. **An engagement measure for supervisory bodies** that is normalised by centre size, distinguishing a working centre from a large one (§9.6).

1.7 Organisation of this report

Chapter 2 delimits what the system does and does not do, and records the assumptions and constraints under which it was built. Chapter 3 defines the six user classes and the scope model that binds them. Chapter 4 states the functional and non-functional requirements and the business rules. Chapter 5 describes the architecture and Chapter 6 the technology stack with the reasoning behind each choice. Chapter 7 is the complete database design: collection inventory, entity-relationship model, per-collection data dictionary, index strategy and retention policy. Chapter 8 catalogues the API. Chapter 9 gives the mathematics of the analytical engine. Chapter 10 documents the security architecture. Chapter 11 specifies the business workflows as state machines and procedures. Chapter 12 covers the user interface. Chapter 13 describes verification. Chapter 14 covers deployment and operations. Chapters 15–17 present project metrics, an honest evaluation with limitations and future work, and the conclusion. Four appendices provide the configuration reference, the audit action catalogue, the full endpoint index and a glossary of the Arabic domain terms used throughout.

Scope, Assumptions and Delimitations

A scope statement is only useful if it says what is excluded as clearly as what is included, and gives the reason for each exclusion. This chapter does both. Everything listed in §2.1 is implemented and present in the code base; everything in §2.2 and §2.3 is absent by decision, not by oversight.

2.1 Functional scope (in scope)

Table 2.1 — Functional domains delivered

#	Domain	What the system does
D1	Multi-tenancy & centre management	Registration of centres by a system administrator, self-registration by a prospective centre followed by an approval or rejection decision, centre profile and logo, activation and suspension by the supervisory super-administrator, per-centre currency, public “best centres” listing, and a system-wide centre activity index.
D2	Identity & access	Six roles; login, logout, refresh-token rotation, self-service password change, code-based password reset, administrator-initiated reset, profile self-update; issuance of student and guardian accounts by the centre manager after enrolment.
D3	People	Teacher records with employment attributes, salary, specialisation and type (circle teacher or external subject teacher); student records with guardian details, memorisation plan and student type; transfer of a teacher between circles with reassignment of the affected students.
D4	Study circles (حلقات)	Circle creation with teacher, schedule, time, location and capacity; circle membership through the student record; a manager-facing overview of every circle's aggregate performance, a per-circle detail view, and a circle ranking readable by teachers in aggregate form.
D5	Academic scheduling	A weekly timetable of subject, day, time slot, circle, teacher and room, readable by every role within its own scope.
D6	Recitation (التسميع)	Daily logging of a recitation with sūrah, āyah range, page count, type (new memorisation or review), qualitative evaluation, legacy error counters and the new four-class error tagging; correction and soft deletion by the recording teacher; a once-per-student-per-type-per-day uniqueness guard.
D7	Attendance	Circle roll-call for a date with four statuses, a unique constraint per student per day, correction and deletion by the teacher, automatic persisted notification to the guardian on absence, and per-centre and per-student attendance views.
D8	Finance	Student fees (monthly, annual, registration) with due dates, payment recording and overdue tracking; teacher salaries by month with a duplicate-payment guard; centre expenses; a finance summary; correction of each record type; and void-with-reason semantics for money.
D9	Performance analytics	Five per-student metrics, a weighted error taxonomy, a thirty-part journey map, four leaderboards, badges, challenges, an experience-point ledger, and completion forecasting with disclosed method and fit.
D10	Review planning	Per-student daily review portion derived from what the student has memorised at a teacher-set rate, plus a “who has reviewed today and who is behind” view, and free-text review plans attached to a student or circle.

#	Domain	What the system does
D11	Competitions	Centre competitions with a closed list of five branches, an appointed judging panel of circle teachers, per-judge scoring averaged into a final score, per-branch ranking, and a four-state lifecycle ending in an immutable approved result and an archive.
D12	Certificates	Milestone certificates at five, ten, fifteen and thirty parts measured against pages actually recited, and competition certificates issued only after results are approved; serial numbering per centre per year; single-issue enforcement by unique index; revocation with audit.
D13	Communication	A closed messaging chain with server-declared audiences, broadcast messages with document attachments, replies, correction and withdrawal of a sent message, persisted notifications, and a live event stream opened with a single-use, sixty-second ticket.
D14	Governance	A hash-chained audit log with an integrity verification endpoint, CSV export of students and attendance, and an administrator view of per-centre platform activity.

2.2 Out of scope

Table 2.2 — Explicitly excluded from this version

Excluded	Rationale
Online payment processing	Fees and salaries are <i>recorded</i> , not transacted. No payment gateway, mobile-money integration or reconciliation with a bank is present. Introducing money movement changes the regulatory and security posture of the whole system and is a separate project.
Native mobile applications	An earlier Expo/React Native prototype was abandoned in favour of a web application installable as a PWA (§12.5). One code base, no store review cycle, and installation over HTTPS on Android, iOS and desktop.
Audio capture or automatic recitation assessment	No recording, no speech recognition, no automated tajwīd scoring. The teacher's judgement is the input; the system structures it.
A Qur'anic text corpus	The system stores sūrah name and number and an āyah range as entered. It does not ship the Qur'anic text, nor a verse-level database of the 114 sūrahs, 30 ajzā', 60 aḥzāb and 240 arbā'. Page counts are derived from āyah counts by a declared approximation (6,236 āyāt ÷ 604 pages) when not entered explicitly.
Email and SMS delivery	Notifications are in-application only, persisted and streamed. Password-reset codes are retrieved by an administrator through an authenticated endpoint rather than mailed; there is no SMTP or SMS provider integration.
Sponsorship and donation processing	The public <code>/sponsorships</code> page is a presentation prototype backed by hard-coded sample opportunities. It is not connected to any back-end endpoint, collection or payment path, and is documented here as a prototype, not a feature.
Timetable optimisation	The academic schedule is recorded as entered. There is no conflict solver, no room-allocation optimiser and no automatic generation.
Server-rendered PDF generation	Certificates and student report sheets are produced by the browser's print pipeline from print-styled components, not by a server-side PDF service. CSV is the only server-generated export format.
Offline data entry	The service worker caches the application shell so the app starts without a network, but there is no offline write queue: recording a recitation requires connectivity.

2.3 Deliberate exclusions and their rationale

Three exclusions deserve separate treatment because they were considered, specified elsewhere, and consciously not carried into this system.

2.3.1 Teacher-generated student login codes

The research source for the analytical layer (§9.1) specifies student authentication by a short code that the teacher generates, as a solution for students without email addresses. Mishkaat instead issues each student a username and password through the centre manager, with a role and a permission scope attached. The code-based flow is not an improvement over an account that already exists, and altering the authentication path carries a security cost that is not justified by a need the system does not have.

2.3.2 A full English interface

An internationalisation layer existed and was used by five screens while the remaining text was Arabic in the source. A partial translation layer is worse than none: it produces an interface that is half-translated at runtime and gives a false impression of language support. The layer and its language picker were removed, and the interface is single-language Arabic. Completing a full translation is a broad mechanical task, independent of the analytical work, and deserves its own pass.

2.3.3 Timezone-aware timestamps

All stored timestamps are naive UTC. Python's `datetime.utcnow()` is deprecated, so the system routes every timestamp through `app/clock.py`, which returns the identical value with the deprecation removed. Migrating to timezone-aware datetimes is a *data* migration — every stored row and every comparison in the code base would have to change, or every comparison raises `TypeError: can't compare offset-naive and offset-aware datetimes` — and is recorded as future work (§16.3), not smuggled in as a function substitution.

2.4 Assumptions

Table 2.3 — Assumptions

ID	Assumption	Impact if false
A1	Each student belongs to exactly one centre and at most one circle at a time.	The scope chain, the circle leaderboards and the attendance uniqueness constraint would all need revision.
A2	A teacher account is bound to exactly one active teacher record within one centre.	A teacher with no linked record has an empty scope and sees nothing — the intended fail-closed behaviour, but it must be explained in the interface.
A3	A student recites at most once per day for new memorisation and once for review.	The 409 duplicate guard would block legitimate entries; the teacher must edit the existing record instead.
A4	The mushaf of reference has 604 pages and 6,236 āyāt (Madinah print).	Page derivation from āyah ranges would skew; the constants are environment-tunable for this reason.
A5	Teachers have a smartphone with a browser and intermittent connectivity during the circle.	Data entry would move to an end-of-day batch, weakening the accuracy of per-session error tagging.

ID	Assumption	Impact if false
A6	The deployment terminates TLS at a reverse proxy under the operator's control.	<code>TRUST_PROXY</code> must remain false, and the rate limiter then keys on the socket peer.
A7	A centre's staff are willing to record daily; the analytical layer has no value without regular input.	Metrics return <code>None</code> (“insufficient data”) rather than misleading zeros, by design.

2.5 Constraints

Table 2.4 — Constraints

ID	Constraint	Consequence for the design
C1	Interface must be Arabic and right-to-left.	Layout, number formatting, print styles and the PWA manifest (<code>"dir": "rtl"</code> , <code>"lang": "ar"</code>) are all RTL-first.
C2	Users include older teachers who are not computer users.	Large targets, minimal forms, a single primary action per screen, explicit empty-state messages that say <i>why</i> a list is empty.
C3	Low-bandwidth, low-cost devices.	GZip middleware, a cached application shell, no heavy chart libraries on first paint, and aggregate endpoints that avoid per-row round trips.
C4	Personal data of minors.	Field-level encryption at rest, no third-party analytics, an onboarding security checklist, and refusal to boot in production without encryption keys.
C5	Deployment on a single modest VPS is the expected case.	Four Uvicorn workers behind Nginx, MongoDB on the same host on an internal network, GridFS for files rather than object storage.
C6	Containers have ephemeral disks.	Uploaded files go to GridFS inside the database, so they survive redeployment and are captured by the same backup.
C7	Multi-worker deployment.	No in-process state for rate limiting, notification fan-out or counters; all shared state lives in MongoDB.

2.6 Operating environment

Table 2.5 — Minimum and recommended environment

Component	Minimum	Recommended	Notes
Python	3.9	3.11+	Async/await throughout; the ASGI stack requires a modern interpreter for performance.

Component	Minimum	Recommended	Notes
Node.js	18	20 LTS	Build-time only; not required at runtime if the built <code>dist/</code> is served by Nginx.
MongoDB	6.0	7.0	TTL indexes, partial filter expressions and atomic <code>findAndModify</code> counters are all used.
Reverse proxy	—	Nginx 1.27	TLS termination, security headers, body-size limit, edge rate limiting.
Client browser	Any modern evergreen browser	Chrome / Edge / Safari current	HTTPS is mandatory for PWA installation and for the secure refresh cookie.

Stakeholders and Users

Mishkaat defines six user roles. The role is not merely a label used to hide menu items: it is the input to a scope-resolution function that decides, for every query the system runs, which rows the caller may see. This chapter defines the roles, the scope model that binds them, and the deliberate exceptions where a role sees slightly more than its data scope would strictly allow.

3.1 User classes and characteristics

Table 3.1 — User classes

Role	Arabic	Who they are	Primary tasks	Data scope
super_admin	المدير العام	A supervisory authority — a ministry, an association or the platform operator — overseeing many centres.	Register centres, suspend or reactivate them, view platform-wide statistics and per-centre activity.	Unrestricted (system-wide).
admin	مدير النظام	The platform administrator who operates the installation.	Create and approve centres, reject applications, reset passwords, read the audit log, message centre managers.	Unrestricted (system-wide).
center_manager	مدير المركز	The director of one centre.	Enrol students, issue student and guardian accounts, hire and evaluate teachers, create circles, run finance, create competitions and appoint judges, approve results, issue certificates, correct any administrative record.	One centre, complete — read everything, but record nothing academic.
teacher	المحفظ / المعلم	The circle teacher who listens to recitation. May alternatively be an <i>external</i> teacher of a subject who visits circles.	Record recitation and attendance for his own students, correct his own records, read his students' performance, judge a competition when appointed, message his manager.	His own circles and their students — nothing else.
student	الطالب	An enrolled student, often a minor, holding an account issued by the manager.	View his own recitations, attendance, metrics, journey, badges, challenges, forecast, certificates and fee status; see his position among his circle peers.	Himself, plus the name of his circle and its teacher, plus aggregate peer positions in his circle.

Role	Arabic	Who they are	Primary tasks	Data scope
parent	ولي الأمر	The guardian of one or more enrolled students.	Follow each child's progress and attendance, receive absence notifications, view fees and certificates.	His children only, on the same terms as the student role.

A ROLE DECISION WORTH STATING PLAINLY

The centre manager **reads everything in his centre but records neither recitation nor attendance**. The reasoning is not technical: recording a recitation is testimony about a session the recorder attended. The manager corrects errors by editing and deleting, but does not create a record attesting to a circle he was not in. The same rule removes the manager from competition scoring: he appoints the panel and approves its result, but he does not enter marks. This is enforced server-side by `scope.teacher_only()`, not by hiding a button.

3.1.1 The external teacher

A centre may employ teachers who are not circle teachers: a language teacher, a visiting tajwīd specialist, an administrator who also teaches. These are recorded with `teacher_type: "external"` and **no system account is created for them**: a request that supplies a username or password for an external teacher is rejected with HTTP 400, and converting an existing circle teacher to external closes their account and invalidates their sessions. The reason is that an account in this system opens students' recitation, attendance and performance data — that is the trust of the circle, not the trust of a visitor giving a lesson in a subject. Someone who neither hears recitation nor marks attendance needs no door.

3.2 The scope-resolution chain

The teacher's scope is not stored on the user document. It is resolved at request time by walking a four-link chain, so that reassigning a circle immediately changes what a teacher can see, with no synchronisation step and no stale permission row:

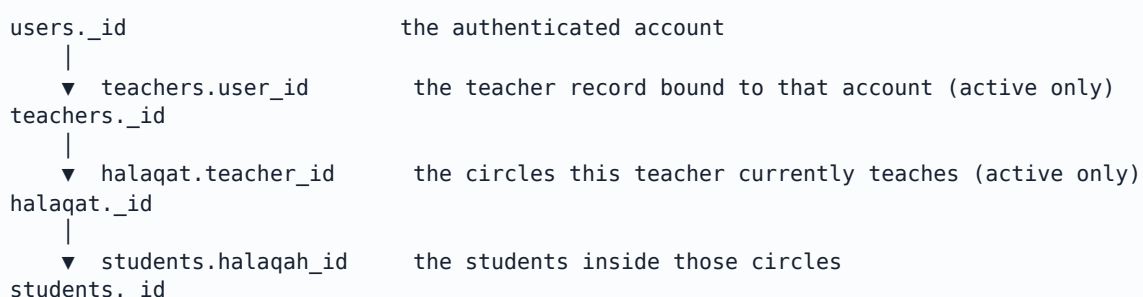


Figure 3.1 — Resolution of a teacher's data scope (`app/scope.py`).

Two properties of this chain are load-bearing:

- **A teacher with no linked record, or with no assigned circle, has an empty scope and sees nothing.** This is correct, not a defect: in the absence of an assignment the default is denial, not permission. The students screen says “no circle has been assigned to you yet” rather than showing an unexplained empty list.
- **An empty scope is represented as `None`, never as an empty filter.** In MongoDB, `{}` matches every document. A function that returned `{}` for “nothing is visible” would return the whole collection — a total inversion that reads as correct code. Every scope helper in the module returns `None` for the empty case and every caller treats `None` as “return an empty list”.

Student and guardian ownership resolves differently and equally deliberately. A student is matched to a record by `students.user_id` or by phone number; a guardian by `students.parent_user_id` or by phone. **Never by name.** A user can edit their own display name from their profile; matching on the name would mean that renaming yourself to a classmate's name opens that classmate's file, grades and fees. Name is not an ownership identifier.

3.3 Role capability matrix

Table 3.2 — Capability matrix (✓ permitted · ◦ read only · ✗ denied)

Capability	super_admin	admin	center_manager	teacher	student	parent
Create / approve / reject a centre	✓	✓	✗	✗	✗	✗
Suspend or reactivate a centre	✓	✗	✗	✗	✗	✗
Read the centre activity index	✓	✓	✗	✗	✗	✗
Enrol a student	✓	✓	✓	✗	✗	✗
Issue student / guardian accounts	✓	✓	✓	✗	✗	✗
Create / edit teachers and circles	✓	✓	✓	✗	✗	✗
Read teacher salaries	✓	✓	✓	✗	✗	✗
Record a recitation	✗	✗	✗	✓	✗	✗
Record attendance	✗	✗	✗	✓	✗	✗
Correct / delete a recitation or attendance row	✓	✓	✓	✓ own	✗	✗
Read a student's performance file	✓	✓	✓	✓ own circle	◦ self	◦ child
Fees, salaries, expenses	◦	✓	✓	✗	◦ own fees	◦ child's fees
Evaluate a teacher	✓	✓	✓	◦ own only	✗	✗
Create a competition, appoint judges, approve results	✓	✓	✓	✗	✗	✗
Enter competition marks	✗	✗	✗	✓ if on panel	✗	✗
Read competition results of own centre	✓	✓	✓	◦	◦	◦
Issue a certificate	✓	✓	✓	✗	✗	✗
View / print own certificates	✓	✓	✓	✗	◦	◦
Leaderboards	✓	✓	✓	✓ own circles	◦ own circle	◦ child's circle
CSV export	✓	✓	✓	✗	✗	✗
Audit log	✓	✓	✗	✗	✗	✗
Academic timetable	✓	✓	✓	◦ own circles	◦ own circle	◦ child's circle

3.4 The teacher: what is granted and what is withheld

Because the circle teacher is the role with the most contested boundary — closest to the data, furthest from administrative authority — its permissions are stated explicitly.

Table 3.3 — Circle teacher: granted and withheld

Granted	Withheld
Read his circles' students, their files, recitations, attendance and metrics	Read a student from another circle — HTTP 403
Record recitation and attendance for his students	Record either for a student who is not his
See his circles' timetable and their competition contestants	See another teacher's timetable or contestants
See himself in the teacher list	See colleagues or their salaries
Read his own evaluation	Read a colleague's evaluation
See the centre's circle ranking in aggregate figures	Open the detail of a circle that is not his
Message his centre manager, with attachments	Message a student, a guardian, or system administration
Correct his own recitation and his circle's attendance after recording	Edit a recitation recorded by a colleague
Read all of his centre's competition records	Act on them unless appointed to the judging panel
—	Create a competition, approve results, or issue a certificate
—	Enrol a new student in the system
—	Issue an account to a student or guardian
—	See finance, CSV export, the audit log, or centre details

3.5 The communication chain

Messaging is not an open directory. Each party addresses only those directly above and below them, which keeps system administration out of direct contact with children and their guardians:

```
system admin ⇌ centre manager ⇌ teacher
                      ↑
                  student / guardian
```

Figure 3.2 — The closed messaging chain.

Table 3.4 — Permitted message audiences

Sender	May address
admin / super_admin	Centre managers only — not teachers, students or guardians
center_manager	The teachers of their centre, their students, their guardians, and system administration
teacher	Their centre manager, and no one else
student / parent	Receive and reply; do not initiate broadcasts

The permitted audiences are read from `GET /api/messages/audiences` and are never hard-coded in the front end. A client-side list diverges from the server rule at the first change, and then offers the user an option that returns 403

when pressed. Attachments are downloaded with the permission check applied to the *message*, not to the file: were the file identifier alone sufficient, anyone who guessed one would read another centre's document.

3.6 Deliberate exceptions to the scope rules

Four places open slightly more than the strict data scope. Each is a deliberate decision with a narrow, documented boundary.

Table 3.5 — Sanctioned scope exceptions

#	Exception	Boundary and reason
E1	Leaderboard scope is the circle, not the self	A student's leaderboard was previously built on his own data scope — a single row — producing “rank 1 of 1” on all four boards. A leaderboard with no peers is not incomplete; it defeats its purpose. The leaderboard query therefore returns the student's circle: peer names, ranks and metric values. These are the aggregate figures written on a blackboard in any mosque. <i>It opens no files</i> : reading a peer's record still passes through <code>check_student_access</code> and still returns 403.
E2	Competition results are public within the centre	A student sees all branches and everyone's ranking. A competition is a declared public event whose results are read aloud.
E3	Circle ranking is visible to teachers	<code>GET /api/halaqat-overview/ranking</code> returns aggregate figures plus circle and teacher names — and not one student name from another circle.
E4	Competition records are readable by every teacher of the centre	A teacher reads his centre's competitions and their results; only <i>scoring</i> is restricted to the appointed panel. Previously a non-judge received 403 and therefore could not know his centre held competitions at all.

WHAT NONE OF THE FOUR EXCEPTIONS OPENS

The student file. Whoever sees a peer's name on a leaderboard or in a competition result cannot open that peer's record: the request returns 403, and that assertion is part of the isolation test suite (§13.3).

Requirements Specification

4.1 Requirement identification scheme

Requirements are numbered `FR-xx` (functional), `NFR-xx` (non-functional) and `BR-xx` (business rules). Functional requirements are grouped by domain. Each carries a priority — `M` mandatory, `S` should have, `C` could have — and a pointer to where it is realised. Requirements whose identifiers begin `FR6` – `FR18` in Chapter 9 refer to the external research specification described in §9.1 and are numbered in that source's scheme; they are distinguished there by context.

4.2 Functional requirements

4.2.1 Identity, authentication and accounts

Table 4.1 — Functional requirements: identity and access

ID	Pri.	Requirement	Realisation
FR-01	<code>M</code>	The system shall authenticate users by username and password and issue a short-lived access token and a long-lived refresh token.	<code>POST /api/auth/login</code>
FR-02	<code>M</code>	The refresh token shall be delivered in an <code>httpOnly</code> cookie scoped to <code>/api/auth</code> , unreadable by page scripts, and rotated on every use with the previous token revoked.	<code>POST /api/auth/refresh</code>
FR-03	<code>M</code>	Logout shall revoke the presented tokens server-side and clear the refresh cookie, so that a token presented afterwards is rejected.	<code>POST /api/auth/logout</code>
FR-04	<code>M</code>	A password change shall invalidate every previously issued token for that user, on every device.	<code>user_version</code> bump
FR-05	<code>M</code>	The system shall support password reset by a time-limited code and by direct administrator reset, both audited.	<code>/api/auth/forgot-password</code> , <code>/reset-password</code> , <code>/admin-reset-password</code>
FR-06	<code>M</code>	A disabled account shall be rejected both at login and when presenting a previously valid token.	<code>app/security.py</code>
FR-07	<code>M</code>	The centre manager shall be able to issue an account for a student and for that student's guardian after enrolment, with the password displayed exactly once and stored only as a hash.	<code>POST /api/students/{id}/accounts</code>
FR-08	<code>M</code>	Issued accounts shall be bound to the student record by identifier (<code>user_id</code> , <code>parent_user_id</code>), not by phone or name matching.	<code>app/routers/accounts.py</code>
FR-09	<code>S</code>	A user shall be able to update their own display name, email and phone.	<code>PUT /api/auth/profile</code>

4.2.2 Centres and multi-tenancy

Table 4.2 — Functional requirements: centres

ID	Pri.	Requirement	Realisation
FR-10	<code>M</code>	An administrator shall create a centre together with its manager account.	<code>POST /api/centers</code>

ID	Pri.	Requirement	Realisation
FR-11	M	A prospective centre shall be able to self-register publicly; the centre and its manager account shall be created inactive with <code>approval_status = "pending"</code> and the manager shall not be able to log in until approved.	POST <code>/api/public/register-center</code>
FR-12	M	An administrator shall approve or reject a pending centre; both decisions shall be written to the permanent audit record.	POST <code>/api/centers/{id}/approve reject</code>
FR-13	M	Public registration shall be rate limited per IP address (3 per 60 minutes), counted only on success so that input errors do not consume the quota.	<code>app/security.py</code>
FR-14	S	A centre shall be able to hold a logo image, uploadable only by that centre's manager, used on printed certificates and reports.	POST <code>/api/centers/{id}/logo</code>
FR-15	S	The super-administrator shall be able to suspend and reactivate any centre and read platform-wide statistics.	<code>/api/super/*</code>
FR-16	C	The system shall publish an unauthenticated listing of leading centres for the public landing page.	GET <code>/api/public/best-centers</code>
FR-17	S	System administration shall be able to read a per-centre activity index over a time window, normalised by centre size and containing no student identifying data.	GET <code>/api/admin/centers-activity</code>

4.2.3 People and circles

Table 4.3 — Functional requirements: teachers, students and circles

ID	Pri.	Requirement	Realisation
FR-18	M	The manager shall create, amend and deactivate teacher records, optionally creating a linked user account for a circle teacher.	<code>/api/teachers</code>
FR-19	M	An external teacher shall receive no account; a request supplying credentials for one shall be rejected, and converting a circle teacher to external shall close their account.	<code>app/routers/teachers.py</code>
FR-20	M	Salary and marital status shall be removed from teacher responses for every role outside centre administration.	<code>common.redact_teacher</code>
FR-21	S	A teacher shall be transferable from one circle to another, with the audit record naming both circles.	POST <code>/api/teachers/{id}/transfer</code>
FR-22	M	The manager shall evaluate a teacher on five named criteria scored out of ten each, with written strengths and weaknesses, producing a total out of one hundred.	POST <code>/api/teachers/{id}/evaluations</code>
FR-23	M	A teacher's evaluation shall be readable by that teacher and by centre administration only — never by colleagues.	<code>app/routers/teachers.py</code>
FR-24	M	The manager shall enrol students with guardian details, memorisation plan, student type and circle assignment.	<code>/api/students</code>
FR-25	M	Student date of birth, guardian address and guardian name shall be encrypted at rest and decrypted only on authorised read.	<code>app/pii.py</code>

ID	Pri.	Requirement	Realisation
FR-26	M	Circles shall be created with teacher, schedule, time, location and maximum capacity, and shall maintain a current student count.	/api/halaqat
FR-27	S	The manager shall see every circle's aggregate performance in one view and open a circle to see its teacher, its students and each student's level.	/api/halaqat-overview
FR-28	S	A weekly academic timetable shall be maintainable per centre and readable by every role within its own scope.	/api/academic-schedules

4.2.4 Academic operations

Table 4.4 — Functional requirements: recitation, attendance and review

ID	Pri.	Requirement	Realisation
FR-29	M	A circle teacher shall record a recitation with sūrah, āyah range, type, qualitative evaluation, optional page count and error tags; the recording teacher's identity shall be taken from the token, never from the request body.	POST /api/recitations
FR-30	M	At most one recitation of each type shall be recordable per student per calendar day; a second attempt shall return 409 naming the existing record.	date_str guard
FR-31	M	Errors shall be taggable by a closed four-class taxonomy; where present the taxonomy shall take precedence over the legacy counters so that no error is counted twice.	app/errors_taxonomy.py
FR-32	M	A recitation shall be correctable and soft-deletable; a deleted recitation shall leave every derived metric, score and forecast.	PUT / DELETE /api/recitations/{id}
FR-33	M	A circle teacher shall record attendance for a circle on a date, in one submission, with four statuses.	POST /api/attendance
FR-34	M	Attendance shall be unique per student per day, enforced by a unique database index and not merely by an application check.	uniq_student_day
FR-35	M	Marking a student absent shall create a persisted notification for the guardian, readable later, independent of whether the guardian is connected.	sse.push_notification
FR-36	M	An attendance record shall be correctable and deletable, with both actions audited.	PUT / DELETE /api/attendance/{id}
FR-37	S	The system shall compute each student's review portion for today from that student's own memorised volume at a teacher-set daily rate, derived from the date rather than from stored state.	GET /api/review-plans/today
FR-38	S	The teacher shall see who has reviewed today and who is behind.	app/routers/review_plans.py

4.2.5 Finance

Table 4.5 — Functional requirements: finance

ID	Pri.	Requirement	Realisation
FR-39	M	The system shall record student fees of three types with due dates and payment state, and shall mark a fee paid with the payment date.	/api/fees

ID	Pri.	Requirement	Realisation
FR-40	M	A monthly fee shall not be creatable twice for the same student, type and due month.	409 guard
FR-41	M	The system shall record teacher salaries by month, and shall refuse a second salary for the same teacher, month and centre.	POST /api/salaries
FR-42	M	The system shall record centre expenses with category and date.	/api/expenses
FR-43	M	Every financial record shall be amendable, and cancellable by <i>voiding</i> — never by erasure — with a written reason, the actor and the time; a voided record shall be excluded from every total.	voided / void_reason
FR-44	M	Every financial action shall write a permanent audit entry that is never purged.	AUDIT_PERMANENT_ACTIONS
FR-45	S	The system shall present a financial summary of collections, salaries, expenses and outstanding fees for a centre.	GET /api/finance/summary

4.2.6 Competitions and certificates

Table 4.6 — Functional requirements: competitions and certificates

ID	Pri.	Requirement	Realisation
FR-46	M	A competition shall have a closed list of branches; branches shall be ranked entirely separately so that a memoriser of Juz' 'Amma is never compared with a memoriser of the whole Qur'an.	COMPETITION_BRANCHES
FR-47	M	The manager shall appoint a judging panel of circle teachers; each judge shall record only their own marks and the final score shall be the panel average.	judge_scores[]
FR-48	M	A competition shall progress through draft → active → grading → approved → archived, and an approved competition shall be immutable: no scoring, no registration, no edit, no deletion.	CompetitionStatus
FR-49	M	Past years' competitions shall remain accessible by year.	GET /api/competitions/years
FR-50	M	The centre manager alone shall issue certificates: milestone certificates at 5, 10, 15 and 30 parts, and competition certificates after approval.	/api/certificates
FR-51	M	A milestone certificate shall be refused until the milestone is met, measured against pages actually recited, not against a claim.	409 guard
FR-52	M	Every certificate shall carry a serial number unique per centre per year, the issuer's identity and the issue time; the same certificate shall not be issuable twice, enforced by a unique index.	MK-YYYY-NNNNN
FR-53	S	A certificate issued in error shall be revocable, and the revocation shall be audited as the issue was.	DELETE /api/certificates/{id}
FR-54	S	The system shall list students who have reached a milestone but have not yet received its certificate.	GET /api/certificates/eligible

4.2.7 Analytics and motivation

Table 4.7 — Functional requirements: analytics

ID	Pri.	Requirement	Realisation
----	------	-------------	-------------

ID	Pri.	Requirement	Realisation
FR-55	M	The system shall compute five per-student metrics — mastery, momentum, precision, consistency and review depth — at read time from session records, never from stored derived values.	app/metrics.py
FR-56	M	Absence of data shall be represented as “no data”, not as zero, in every metric.	None semantics
FR-57	M	The system shall forecast Juz' and full-Qur'an completion dates using momentum, weighted error density and attendance rate, from a model trained on the centre's own history.	app/forecast.py
FR-58	M	Every forecast shall disclose the method used, the number of training rows and the coefficient of determination; a fallback estimate shall never be presented as a model output.	predict_weekly_pages
FR-59	M	Fitted coefficients that contradict domain direction shall be zeroed, never inverted.	_constrain_signs
FR-60	M	The system shall present four independent leaderboards rather than one composite ranking.	GET /api/performance/leaderboards
FR-61	M	Badges shall be awarded automatically on threshold attainment, granted once, and never withdrawn.	evaluate_badges
FR-62	M	Experience points shall be written to an append-only ledger with a unique key on (student, reason, source) so that reprocessing awards nothing twice.	xp_ledger
FR-63	S	Periodic challenges shall show live progress computed from the same session records.	challenge_progress
FR-64	S	A journey map shall show the student's position across the thirty parts of the Qur'an.	.../journey
FR-65	C	An administrator shall be able to re-evaluate badges across an entire centre after enabling the layer on historical data.	POST /api/performance/recompute

4.2.8 Communication and governance

Table 4.8 — Functional requirements: communication and governance

ID	Pri.	Requirement	Realisation
FR-66	M	Messaging shall follow a closed chain; permitted audiences shall be served by the API, not hard-coded in the client.	GET /api/messages/audiences
FR-67	S	Messages shall carry document attachments, uploaded separately and referenced by the message; download authorisation shall be evaluated on the message, not on the file identifier.	/api/messages/attachments
FR-68	S	A sent message shall be correctable and withdrawable by its sender.	PUT / DELETE /api/messages/{id}
FR-69	M	Notifications shall be persisted before being streamed, and the stream shall read from the database so that it works across worker processes.	app/sse.py

ID	Pri.	Requirement	Realisation
FR-70	M	The live stream shall be opened with a single-use ticket valid for sixty seconds, obtained with a normal Authorization header — never with an access token in the URL.	POST /api/notifications/stream-ticket
FR-71	M	Financial, authentication and permission events shall be written to a hash-chained audit log with atomic sequence numbers.	app/audit.py
FR-72	M	The audit chain shall be verifiable through an endpoint that recomputes the chain and reports any break.	GET /api/audit-logs/verify
FR-73	S	The system shall export students and attendance as UTF-8 CSV with a byte-order mark so that Arabic renders correctly in Excel; every export shall be audited.	/api/export/*.csv
FR-74	M	The system shall expose liveness and readiness endpoints without authentication.	/api/health, /api/health/ready

4.3 Non-functional requirements

Table 4.9 — Non-functional requirements

ID	Quality	Requirement and acceptance condition
NFR-01	Security — isolation	No request shall return data belonging to another centre, another circle or another student. Acceptance: 314 assertions across seven suites test for leakage rather than for request success; a 200 response is not a failure, a 200 response carrying another party's data is.
NFR-02	Security — sessions	Access tokens shall expire within 60 minutes; refresh tokens within 7 days; both shall carry a JTI and a user version; revocation shall be immediate and enforced on the server.
NFR-03	Security — credentials	Passwords shall be hashed with bcrypt at cost 12 or higher and shall require at least eight characters including a letter and a digit on every path that sets one.
NFR-04	Security — data at rest	Student date of birth, guardian address and guardian name shall be stored encrypted (Fernet, AES-128-CBC with HMAC). The system shall refuse to start in production without an encryption key.
NFR-05	Security — availability	Login shall be limited to 5 failures per IP per 15 minutes and 10 per account per 15 minutes; limiter state shall be correct across worker processes.
NFR-06	Auditability	Financial and permission events shall be tamper-evident; any break in the hash chain shall be detectable and any gap shall be explicitly marked rather than silently tolerated.
NFR-07	Performance	List endpoints shall not issue per-row database round trips. Acceptance: the honour roll computes scores for all students in two queries, not two per student; leaderboards and circle overviews batch their history loads.
NFR-08	Performance — indexing	Every hot query path, ownership lookup and TTL collection shall be indexed; indexes shall be created idempotently at start-up and conflicts resolved without failing the boot.
NFR-09	Scalability	The application shall run correctly with multiple Uvicorn workers; no correctness-bearing state shall live in process memory. The forecast model cache is explicitly per-worker and advisory only.
NFR-10	Reliability	An unhandled exception shall return a generic message with a request identifier and shall never return a stack trace to the client; every user-supplied identifier shall be validated, returning 400 rather than 500.

ID	Quality	Requirement and acceptance condition
NFR-11	Correctability	Every administrative record shall be amendable after creation; financial records shall be voidable but not erasable.
NFR-12	Usability	The interface shall be Arabic and right-to-left; primary actions shall be reachable in at most two taps from the role dashboard; empty states shall state the cause.
NFR-13	Portability	The application shall be installable from the browser on Android, iOS and desktop without an app store, and shall start from a cached shell.
NFR-14	Interpretability	Every analytical output shall be explainable by reference to the session records that produced it. No metric shall be presented whose provenance cannot be shown to a parent.
NFR-15	Honesty of claims	Forecast accuracy shall not be asserted. Each prediction shall carry its method, training size and fit; a fallback shall be labelled as such.
NFR-16	Maintainability	Authorisation rules shall exist in exactly one module and be documented in exactly one document, changed together. Severity weights and calibration constants shall live in configuration, not inside functions.
NFR-17	Operability	A single-command deployment shall exist; backups shall be encrypted before touching disk; restoration shall be rehearsable.
NFR-18	Privacy	Minor data shall not be exported to third-party analytics. Supervisory views shall carry counts and dates, never student identities.

4.4 Business rules

Table 4.10 — Business rules

ID	Rule
BR-01	Recording a recitation or attendance is testimony; only the circle teacher who was present may create the record. Administration corrects, it does not attest.
BR-02	A recitation belongs to the teacher who recorded it; a colleague may not edit it. Centre administration may.
BR-03	One new-memorisation recitation and one review recitation per student per calendar day.
BR-04	One attendance record per student per calendar day, guaranteed by unique index.
BR-05	One salary per teacher per month per centre, counting only non-voided records.
BR-06	One monthly fee per student per due month per type.
BR-07	Money is voided, never deleted. A void requires a written reason and records the actor and time, and the record is excluded from every total thereafter.
BR-08	An approved competition is frozen. No marks, no registrations, no edits, no deletion — it may only be archived.
BR-09	A competition's final score is the arithmetic mean of the panel's marks; every mark is attributed to the judge who gave it.
BR-10	Competition branches are ranked separately, each from position one.
BR-11	A milestone certificate requires the milestone to be met, measured in pages actually recited (a Juz' is $604/30 \approx 20.13$ pages), with a half-page tolerance.
BR-12	A competition certificate requires the competition result to be approved first.
BR-13	The same certificate is never issued twice; the constraint is a unique index, not a code check.

ID	Rule
BR-14	An external teacher has no account; conversion to external closes any existing account.
BR-15	Salary and marital status are visible only to centre administration.
BR-16	A teacher's evaluation is readable by that teacher and by administration only.
BR-17	Student and guardian accounts are issued after enrolment and bound by identifier; a forgotten password is replaced, never recovered.
BR-18	A publicly registered centre cannot operate until an administrator approves it.
BR-19	A badge, once earned, is never withdrawn; it certifies an achievement that occurred, not a current state.
BR-20	Experience points are granted once per (student, reason, source); reprocessing is a no-op.
BR-21	Precision is not computed from legacy counters, because those carry no error class; an untagged history yields “no data”, not zero.
BR-22	Judging is a task with a term, not a standing permission: the competitions menu item appears for a teacher only while a competition they judge is in progress.

4.5 Principal use cases

Table 4.11 — Use case summary

ID	Use case	Primary actor	Outcome
UC-01	Register a centre publicly	Prospective manager	Centre and manager account created inactive, pending approval; rate limited; audited.
UC-02	Approve a centre	Administrator	Centre and manager activated; permanent audit entry written.
UC-03	Enrol a student	Centre manager	Student record created with encrypted personal fields and circle assignment; circle count updated.
UC-04	Issue student and guardian accounts	Centre manager	Two accounts created, bound by identifier; passwords shown once; audited.
UC-05	Record a recitation	Circle teacher	Session stored with error tags; duplicate refused; XP granted idempotently; forecast cache invalidated; badges re-evaluated.
UC-06	Take the roll	Circle teacher	One record per student per day; absences generate persisted guardian notifications.
UC-07	Read a student's performance file	Teacher / manager / parent / student	Five metrics, forecast with disclosed method, badges, challenges and journey — all recomputed from records.
UC-08	Collect a fee	Centre manager	Fee marked paid with date; permanent audit entry; double payment refused.
UC-09	Pay a salary	Centre manager	Salary recorded; a second payment for the same month refused with 409.
UC-10	Void a financial record	Centre manager	Record marked voided with reason and actor; excluded from totals; audited permanently.
UC-11	Run a competition	Manager + judging panel	Panel appointed, contestants registered per branch, judges score independently, mean computed, ranks assigned per branch, manager approves and the result freezes.

ID	Use case	Primary actor	Outcome
UC - 12	Issue a certificate	Centre manager	Eligibility verified against recited pages or approved results; serial allocated atomically; single issuance enforced by index; audited.
UC - 13	Broadcast a message with a document	Manager / teacher	Audience validated server-side; attachment stored in GridFS after content sniffing; recipients notified.
UC - 14	Verify the audit chain	Administrator	Each entry's hash recomputed and linked; breaks and gaps reported.
UC - 15	Assess platform engagement	Administrator	Per-centre activity index over a window, normalised by size, with no student data.

4.5.1 Use case detail — UC-05, Record a recitation

Primary actor	Circle teacher (<code>teacher</code>)
Preconditions	Teacher is authenticated; the account is linked to an active teacher record; the student is in one of the teacher's active circles.
Main flow	1. Teacher opens the recitation screen and selects a student from his circle. 2. He enters sūrah, āyah range, type (new or review) and, optionally, the page count. 3. He tags the errors he heard using the four-class tagger, and gives an overall evaluation. 4. The server rejects the request unless the caller's role is <code>teacher</code>. 5. <code>check_student_access</code> confirms the student is in the caller's circle. 6. The recording teacher is resolved from the token, not from the request body. 7. A same-day record of the same type is searched; if found the request is refused with 409 naming the existing record's sūrah. 8. The record is written with <code>date</code>, <code>date_str</code> and <code>center_id</code>. 9. Experience points are granted idempotently, badges re-evaluated, and the centre's forecast model cache invalidated.
Alternative flows	7a. Duplicate exists → 409 with guidance to edit the existing record. 5a. Student outside the caller's circle → 403 “this student is not in your circle”. 6a. No linked teacher record → 403 “your account is not linked to a teacher record”. 9a. Motivation layer fails → the failure is logged and the recitation still succeeds: the session is the real record, the points are a layer above it.
Postconditions	One recitation stored; derived metrics, leaderboards and forecast reflect it at the next read; the XP ledger contains at most one entry per (student, reason, source).

System Architecture

5.1 Architectural style and rationale

Mishkaat is a **modular monolith**: one deployable back-end process, internally decomposed by domain into routers over a shared set of cross-cutting modules, serving a decoupled single-page front end over a JSON API. The choice is deliberate and is defended here because it is the architectural decision with the widest consequences.

Table 5.1 — Architectural options considered

Option	Why it was not chosen	What was chosen instead
Microservices	The expected deployment is a single modest VPS run by one operator. Splitting eleven domains into services would multiply the operational surface — service discovery, distributed tracing, cross-service authorisation — for a system whose entire authorisation model depends on being decided in <i>one</i> place.	One process, domain-separated routers, one authorisation module.
Server-rendered templates	The interface needs to be installable, work from a cached shell, and re-render live analytics without full page loads.	A JSON API plus a SPA delivered as a PWA.
A synchronous WSGI framework	Most endpoints are I/O-bound aggregations over MongoDB, with a live notification stream held open per connected user. Thread-per-request scales poorly for held-open connections.	ASGI (FastAPI + Uvicorn) with an async MongoDB driver end-to-end.
A relational database	The academic record is naturally document-shaped and evolves: a recitation gained page counts and an error-tag map without a migration; judge scores are an array inside a contestant.	MongoDB, with integrity enforced by unique and partial indexes where it matters (§7.13).

5.2 Logical view

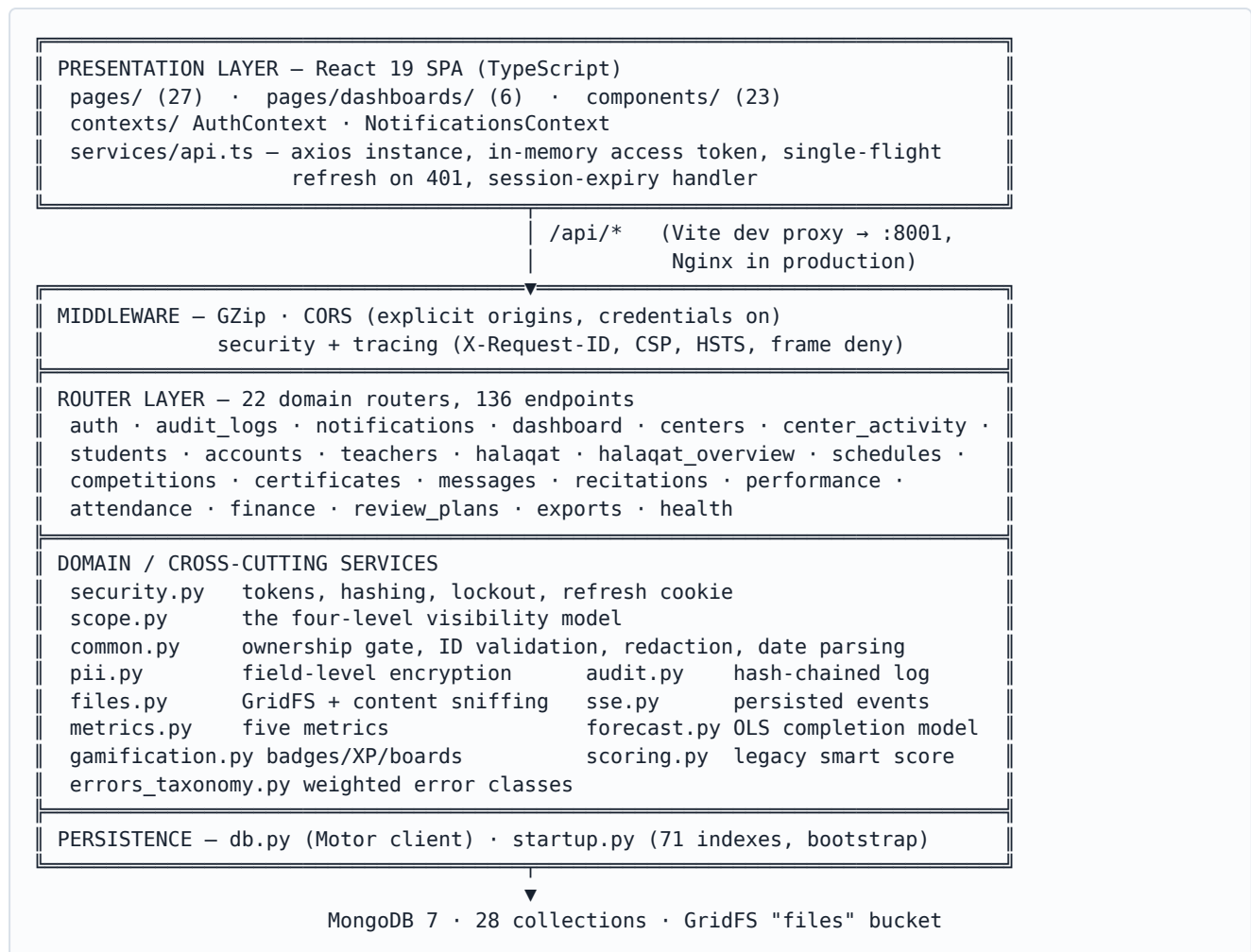


Figure 5.1 — Logical architecture.

5.3 Deployment view

The reference deployment is the Docker Compose stack shipped with the repository. Its defining property is that **only Nginx is reachable**: the application publishes no port and MongoDB publishes no port; both sit on a Docker network declared `internal: true`, which also denies them outbound internet access.

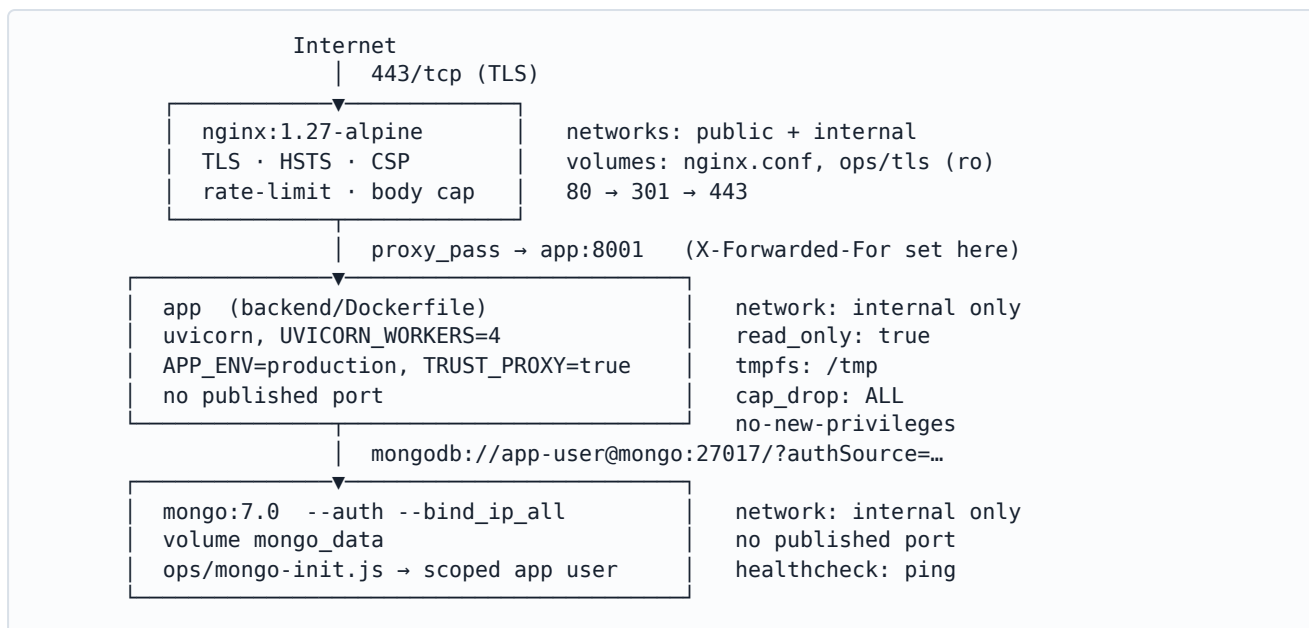


Figure 5.2 — Container deployment topology (`docker-compose.yml`).

WHY TRUST_PROXY IS TRUE HERE AND FALSE BY DEFAULT

The rate limiter keys on the client IP. Behind a proxy, the socket peer is the proxy for every user, so five bad passwords from one person would lock out every user of the deployment. The fix is to read `X-Forwarded-For` — but only when the application is genuinely behind a proxy under the operator's control, because trusting that header blindly lets any client forge it and evade the limiter altogether. In the Compose stack the app has no published port and Nginx sets the header itself, so the flag is safe; the shipped default in `.env.example` is `false`.

A second supported topology runs the same components directly on a VPS under systemd. The `mishkaat.service` unit runs the API under `DynamicUser` with `NoNewPrivileges`, no write access outside `/tmp`, no access to kernel tunables, and an outbound network allow-list restricted to localhost. A companion timer unit runs the encrypted backup daily (§14.3).

5.4 Back-end module inventory

Table 5.2 — Cross-cutting modules (`backend/app/`)

Module	Lines	Responsibility and notable design decisions
<code>main.py</code>	117	Application assembly: lifespan hook, GZip, CORS, the security/tracing middleware, and router inclusion. Router order is deliberate — fixed paths precede parameterised ones so that <code>/api/centers/{id}</code> cannot swallow <code>/api/centers/pending</code> . Interactive docs are served only outside production.
<code>config.py</code>	156	All settings, secrets and calibration constants. Contains the production fail-fast guards: default <code>SECRET_KEY</code> , missing <code>PII_ENCRYPTION_KEY</code> and a wildcard CORS origin each abort the boot.
<code>models.py</code>	638	All Pydantic request/response models, the role and status literals, competition branches, milestone definitions and teacher evaluation criteria, plus field validators for error tags and page counts.
<code>security.py</code>	311	Password hashing and complexity, JWT encode/decode with type, JTI, <code>user_version</code> and <code>iat</code> checks, revocation lookup, dual-key rate limiting, reset-code hashing, and the <code>httpOnly</code> refresh cookie.

Module	Lines	Responsibility and notable design decisions
scope.py	241	The single source of visibility boundaries: teacher record and circle resolution, own-student resolution, circle and student ID sets, the student query filter, the leaderboard query, the circle-in-scope assertion and the teacher-only guard.
common.py	194	<code>check_student_access</code> (the ownership gate), safe <code>ObjectId</code> conversion, client IP resolution, teacher redaction, date-boundary parsing, and the soft-delete and void filters.
startup.py	562	Idempotent creation of 71 indexes with conflict recovery, administrator and super-administrator bootstrap, and opt-in demo seeding.
metrics.py	208	The five performance metrics, page derivation, error density and attendance rate.
forecast.py	249	Training-row construction, Gaussian-elimination OLS with R^2 , domain sign constraints, prediction with plausibility guard, and date projection.
gamification.py	209	Ten badges, four challenges, four leaderboard definitions, idempotent XP award and bulk totals.
errors_taxonomy.py	117	The four error classes with weights, the legacy counter mapping, normalisation, weighted load, high-severity count and the “was this tagged?” predicate.
audit.py	98	Atomic sequence allocation, previous-hash resolution with explicit gap marking, chain entry construction and the hash-input reconstruction used by the verifier.
files.py	139	GridFS storage with chunked size-capped reads, magic-byte content sniffing, and an allow-list that excludes SVG.
sse.py	74	Single-use stream tickets and database-backed notification persistence.
scoring.py	74	The legacy composite “smart evaluation” score, extracted into one function after the honour roll and the ranking screen had drifted to different time windows.
pii.py	51	Fernet encrypt/decrypt with legacy-plaintext tolerance, and the student PII field list.
clock.py	29	The single time source — naive UTC, replacing the deprecated <code>datetime.utcnow()</code> without changing a single stored value.
db.py	11	The Motor client and database handle.

5.5 Router inventory

Table 5.3 — Domain routers

Router	Lines	Endpoints	Domain
centers.py	673	17	Centre CRUD, public registration, approval workflow, logo, super-admin centre control, public best-centres listing
finance.py	669	15	Fees, salaries, expenses, voids, financial summary
competitions.py	535	13	Competitions, judging panels, contestants, per-judge scoring, lifecycle transitions
auth.py	506	10	Login, refresh, logout, profile, password change and reset paths
recitations.py	483	6	Recitation CRUD, per-student history, completion prediction endpoint
teachers.py	426	8	Teacher CRUD, circle transfer, evaluations
performance.py	354	6	Metrics, journey, leaderboards, challenges, badge recomputation
attendance.py	335	7	Roll-call, per-circle, per-student and per-centre views, corrections

Router	Lines	Endpoints	Domain
certificates.py	334	5	Milestone and competition certificates, eligibility, revocation
messages.py	302	9	Broadcasts, inbox, audiences, replies, attachments
review_plans.py	285	5	Review plans and the daily derived review portion
halaqat_overview.py	264	3	Circle performance overview, ranking, circle detail
students.py	207	5	Student CRUD within scope
dashboard.py	201	4	Role-aware statistics, honour roll, rankings, per-student analytics
center_activity.py	199	1	Per-centre platform activity index
accounts.py	193	3	Student and guardian account issuance and reset
schedules.py	168	4	Academic timetable CRUD
notifications.py	140	5	Stored notifications, read state, stream ticket, event stream
exports.py	139	2	CSV export of students and attendance
halaqat.py	134	4	Circle CRUD
audit_logs.py	117	2	Audit log listing and chain verification
health.py	42	2	Liveness and readiness
Total	6,866	136	

5.6 Front-end structure

Table 5.4 — Front-end organisation (`frontend/src/`)

Directory	Modules	Contents
pages/	27	One module per screen: Landing, Login, Dashboard, Centers, Students, Teachers, Halaqat, Recitations, Attendance, Finance, Reports, Rankings, Leaderboards, StudentAnalytics, StudentPerformance, ReviewPlans, AcademicSchedules, Competitions, Certificates, BulkMessages, NotificationsPage, AuditLogPage, ProfilePage, SettingsPage, RegisterCenter, Sponsorships, ErrorPages.
pages/dashboards/	6	A distinct dashboard per role: SuperAdmin, Admin, CenterManager, Teacher, Student, Parent.
components/	23	Fifteen primitives in <code>ui/</code> (button, card, input, label, avatar, loading, StatCard, PageHeader, EmptyState, NumberInput, PermissionBadge, WelcomeHero, MishkaatMark, ErrorTagger, StudentReportSheet) plus domain components: CertificateSheet and Ornaments, HalaqatRanking and HalaqahDetailModal, CenterActivityPanel, StudentAccountsModal, TeacherEvaluationModal, DashboardLayout.
contexts/	2	<code>AuthContext</code> (session restoration, role predicate, expiry handling) and <code>NotificationsContext</code> .
services/	1	<code>api.ts</code> — the axios instance, the in-memory access token store, and the single-flight refresh interceptor.
lib/	5	theme, class-name utilities, number/date formatting, landscape printing, error mapping.
hooks/	1	<code>useNotifications</code> — polling and live stream subscription.

Directory	Modules	Contents
types/	1	Shared TypeScript interfaces mirroring the API contract.

5.6.1 Token handling in the client

The access token lives in a module-scoped variable — not in `localStorage` — so it disappears when the tab closes and is never readable by injected script. The refresh token is never seen by JavaScript at all: the server sets it as an `httpOnly` cookie scoped to `/api/auth` with `SameSite=lax`, and the browser attaches it automatically. Session persistence across a page reload is preserved by calling `/api/auth/refresh` on boot with no body: the cookie alone re-establishes the session.

A single-flight guard wraps refresh. Because the server rotates the refresh token and revokes the previous one, two concurrent refreshes would invalidate each other and log the user out; concurrent 401s therefore await one shared refresh promise. A 403 is never treated as session expiry — it is a permission shortfall, and logging the user out on one would be both wrong and confusing.

5.7 Request lifecycle

1	Nginx	TLS termination, security headers, body-size cap, edge rate limit
2	GZipMiddleware	compresses responses ≥ 1000 bytes
3	CORSMiddleware	explicit origin allow-list; credentials enabled; "*" refused in prod
4	security_and_tracing_middleware	└ assigns or echoes X-Request-ID └ catches unhandled exceptions $\rightarrow$ generic 500 + request id (the traceback is logged server-side, never returned) └ sets nosniff, DENY, no-referrer, COOP, Permissions-Policy, CSP "default-src 'none'", and HSTS in production
5	Depends(get_current_user)	└ decode JWT, verify signature and token type ("access") └ reject if the JTI is on the revocation list └ reject if the account is disabled $\rightarrow$ 403 └ reject if user_version $\neq$ token "uv" $\rightarrow$ 401 └ reject if password_changed_at > token "iat" $\rightarrow$ 401
6	Role guard	role allow-list, or scope.teacher_only() for attesting actions
7	Scope resolution	scope.student_query / visible_student_ids / visible_halaqah_ids (None $\Rightarrow$ nothing visible – never an empty filter)
8	Ownership gate	check_student_access(student_id) for any endpoint taking one
9	Handler	Motor query, Pydantic validation, serialisation
10	Side effects	audit entry $\cdot$ notification $\cdot$ XP award $\cdot$ model cache invalidation

Figure 5.3 — The path of an authenticated request.

5.8 Recurring design patterns

Table 5.5 — Patterns used consistently across the code base

Pattern	Application
Single gate	Every endpoint accepting a <code>student_id</code> passes it through <code>check_student_access</code> , never through a hand-written <code>center_id</code> comparison. One hole in the gate previously produced ten leaks; one fix closed all ten.
Fail closed	Absent assignment means denial. An empty scope returns <code>None</code> ; a manager with no <code>center_id</code> sees nothing rather than everything.

Pattern	Application
Identity from the token	The acting user is taken from the verified token, never from a request field. This removed a class of both bugs and forgeries: the client used to send the account ID where the teacher-record ID was expected, so every teacher's recitation was refused.
Compute on read	No derived academic value is stored. Metrics, ranks and forecasts are functions of the session records at the moment of the query.
Constraint in the database	Where duplication must be impossible, a unique (often partial) index enforces it. An application check followed by a write is not atomic — two simultaneous clicks both find nothing and both insert.
Idempotent side effects	XP awards carry a unique (student, reason, source) key; a duplicate-key error is treated as success, not failure.
Bulk over per-row	Aggregate screens load their history in one or two queries keyed by an <code>\$in</code> list, rather than a query per student.
Soft delete and void	Academic records are soft-deleted with <code>is_deleted</code> ; money is voided with a reason and an actor. Both filters use <code>\$ne: true</code> so that historical rows lacking the field are still visible.
Configuration over constants	Calibration values — severity weights, smoothing, windows, XP rates — live in <code>config.py</code> and are environment-tunable, not buried inside a function.
Decision records in code	Non-obvious decisions are documented at the site of the code, stating what the previous behaviour was and why it was wrong. This report draws directly on those records.

Technology Stack

6.1 Back-end technologies

Table 6.1 — Back-end dependencies (`backend/requirements.txt` , all version-pinned)

Component	Version	Role and rationale
Python	≥ 3.9	Implementation language. Chosen because the analytical layer — regression fitting, metric computation — is expressed naturally here, so no separate analytics service is needed: the linear algebra is implemented in-process with no numerical dependency.
FastAPI	0.110.1	ASGI web framework. Provides dependency injection (used for authentication), automatic request validation from type hints, and OpenAPI generation — enabled in development, disabled entirely in production.
Uvicorn	0.25.0	ASGI server, run with four workers behind Nginx.
Motor	3.3.1	Asynchronous MongoDB driver, including the GridFS bucket used for file storage. Keeps the request path non-blocking end to end.
Pydantic	2.12.5	Request and response models. Typed models eliminate mass-assignment: an endpoint that accepted a raw <code>dict</code> was replaced with a declared model for exactly this reason. Custom validators enforce error-tag codes and plausible page counts.
PyJWT	2.11.0	HS256 token encoding and decoding, with explicit verification of token type, JTI, user version and issue time.
bcrypt	via passlib 1.7.4	Password hashing at cost 12 (OWASP guidance), tunable by environment. Hashing and verification call bcrypt directly.
cryptography	44.0.1	Fernet (AES-128-CBC + HMAC-SHA256) for field-level encryption of student personal data at rest.
python-multipart	0.0.22	Multipart parsing for the login form and file uploads.
python-dotenv	1.2.1	Environment loading, invoked before any setting is read.

6.2 Front-end technologies

Table 6.2 — Front-end dependencies (`frontend/package.json`)

Component	Version	Role and rationale
React	19.1.0	UI library for the single-page application.
TypeScript	5.9	Static typing across the client; shared interfaces mirror the API contract, so a field renamed on the server surfaces as a compile error.
Vite	7.2	Build tool and development server, with a proxy to the API so that the front end and API share an origin in development as they do in production.
React Router	7.13	Client routing, with role-aware route guards mirroring the server's permissions.
Tailwind CSS	4.1	Utility-first styling with a custom token layer for the Islamic palette and dark mode.

Component	Version	Role and rationale
Radix UI	various	Accessible unstyled primitives: dialog, dropdown, select, tabs, toast, progress, avatar, label, slot.
axios	1.13	HTTP client, carrying the request interceptor that attaches the bearer token and the response interceptor that performs single-flight refresh.
react-hook-form + zod	7.71 / 4.3	Form state and schema validation on the client, mirroring the server's Pydantic constraints.
Recharts	2.12	Progress and analytics charts.
lucide-react	0.563	Icon set, also used for badge and challenge glyphs.
clsx / tailwind-merge / CVA	—	Deterministic class composition for the component primitives.

6.3 Infrastructure and operations

Table 6.3 — Infrastructure components

Component	Version	Role
MongoDB	7.0	Primary data store, started with <code>--auth</code> , bound to an internal network only, initialised with a scoped non-root application user.
GridFS	—	Binary store for centre logos and message attachments, inside the database so that container redeployment does not lose files and the database backup captures them.
Nginx	1.27-alpine	TLS termination, HTTP → HTTPS redirect, security headers, body-size cap, edge rate limiting on the login path, static asset delivery.
Docker / Compose	24+ / 2+	Reference deployment with isolated internal and public networks, a read-only application filesystem, dropped capabilities and <code>no-new-privileges</code> .
systemd	—	Alternative deployment: a sandboxed service unit plus a daily backup timer.
mongodump + GPG	—	Backups piped directly into encryption so that plaintext never touches disk, retained 30 days, optionally shipped off-host with rclone.

6.4 Development tooling

Table 6.4 — Development and quality tooling

Tool	Use
ESLint 9 + typescript-eslint	Front-end linting, including React hooks rules.
<code>tsc -b</code>	Type checking as a build gate; <code>noUnusedParameters</code> is enforced.
<code>compileall</code> / pyflakes	Back-end syntax and unused-symbol gates, listed as pre-onboarding checks.
Seven Python test suites	Behavioural isolation and business-rule tests run against a live MongoDB (Ch. 13).
Icon build scripts	All PWA icons are generated from one vector source; changing an icon requires bumping the service-worker cache name.

6.5 Dependency policy

- **Every back-end dependency is pinned to an exact version.** The runtime surface of this system includes cryptography and token handling; a floating version is an unreviewed change to security-relevant code.
- **The dependency list is deliberately short.** Nine runtime packages. The forecasting model is implemented with plain Python arithmetic rather than pulling in a numerical stack, which keeps the image small and the audit surface narrow.
- **Vulnerability scanning is specified as a CI gate** (`pip audit` , build fails on critical findings), with a quarterly review of Python and MongoDB minor versions.
- **No third-party analytics or telemetry** is included anywhere in the client, because the platform holds data about minors.

Database Design and Data Dictionary

7.1 Choice of a document store

The persistence layer is MongoDB 7. The choice is defensible on three grounds specific to this domain, and it carries one cost that the design pays explicitly.

- **The academic record is naturally document-shaped and evolving.** A recitation acquired an optional page count and a map of error tags after the system was in use; a competition contestant carries an array of per-judge marks whose length varies with the size of the panel; a message carries an array of attachment descriptors and an array of replies. In a relational schema each of these is a join table and a migration.
- **Read patterns are aggregate and scope-filtered.** Almost every screen asks for “all rows for this set of students” — a single `$in` query on an indexed field. The system deliberately avoids per-row queries (§5.8), and a document store serves this shape well.
- **Operational simplicity.** One database process, one backup artefact — and, because uploaded files live in GridFS inside the same database, one restore brings back logos and attachments with the records that reference them.

THE COST, AND HOW IT IS PAID

A document store enforces no referential integrity and no uniqueness unless asked. The design therefore treats **unique and partial indexes as business rules**, not as performance hints: attendance uniqueness per student per day, certificate uniqueness per student per milestone, XP uniqueness per (student, reason, source) and badge uniqueness per (student, badge) are all database constraints. An application-level check followed by a write is not atomic — two simultaneous clicks each find nothing and each insert — which is precisely the duplication the check was written to prevent.

7.2 Collection inventory

Table 7.1 — All collections, by purpose

Collection	Class	Lifetime	Purpose
users	Core identity	Permanent	Every account of every role; the only collection holding credentials.
centers	Core / tenant	Permanent	The tenant root. Every scoped record carries a <code>center_id</code> .
students	Core entity	Permanent	Enrolled students, with encrypted personal fields and links to their own and their guardian's accounts.
teachers	Core entity	Permanent	Employment records; the bridge between a user account and the circles that account may see.
halaqat	Core entity	Permanent	Study circles: teacher, schedule, capacity, occupancy.
recitations	Academic record	Permanent (soft delete)	The central academic event; the source of every performance metric and forecast.
attendance	Academic record	Permanent	One row per student per day; drives consistency, attendance rate and absence notification.
academic_schedules	Academic record	Permanent	The weekly timetable.

Collection	Class	Lifetime	Purpose
review_plans	Academic record	Permanent	Per-student or per-circle review plans and the daily review rate.
fees	Financial	Permanent (void only)	Student fees with type, due date and payment state.
salaries	Financial	Permanent (void only)	Teacher salary payments by month.
expenses	Financial	Permanent (void only)	Centre expenditure.
competitions	Events	Permanent	Competitions, branches, judging panels, lifecycle state.
competition_contestants	Events	Permanent	One row per registered student per competition, holding per-judge marks.
certificates	Events	Permanent	Issued certificates with serial, issuer and time.
teacher_evaluations	HR	Permanent	Five-criterion teacher evaluations with a computed total.
xp_ledger	Motivation	Permanent (append only)	One immutable row per experience-point award.
student_badges	Motivation	Permanent	One row per badge earned; never removed.
bulk_messages	Communication	Permanent	Broadcast messages with attachments and replies.
notifications	Communication	TTL 180 d	Stored per-user notifications; also the source of the live stream.
audit_logs	Governance	Mixed	Hash-chained action log; financial and permission actions never purged, the rest after 2 years.
counters	Infrastructure	Permanent	Atomic sequence generator: the audit sequence and one certificate series per centre per year.
revoked_tokens	Security	TTL to expiry	The JTI blacklist; entries self-delete when the token would have expired anyway.
login_attempts	Security	TTL ≥ 1 h	Dual-key (IP and account) failure counters and lockout deadlines.
register_attempts	Security	TTL 2 h	Per-IP public registration window.
reset_requests	Security	TTL 2 h	Password-reset request windows by IP and by account.
password_resets	Security	TTL to expiry	Hashed reset codes with an attempt counter.
stream_tickets	Security	TTL 5 min	Single-use tickets for opening the notification stream (valid 60 s in code).
files.files / files.chunks	Binary	Permanent	GridFS bucket holding centre logos and message attachments with their sniffed content type.

7.3 Entity-relationship model

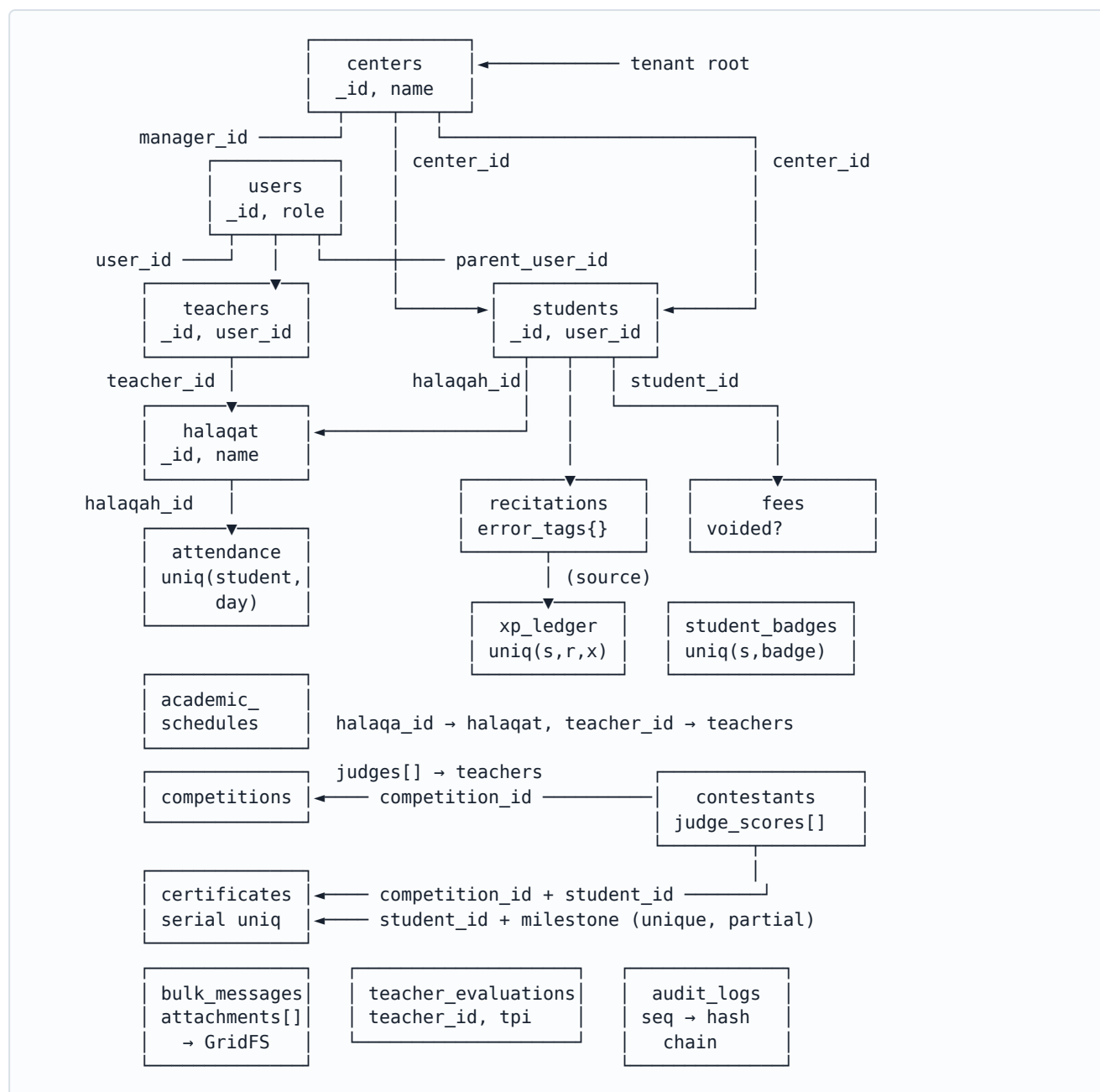


Figure 7.1 — Entity-relationship model. Arrows point from the referencing field to the referenced collection. Every arrow is a string-encoded `ObjectId` stored as a plain field, not a database-enforced foreign key (§7.13).

7.4 Relationship catalogue

Table 7.2 — Relationships

From	To	Cardinality	Field and meaning
centers	users	1 : 1	<code>centers.manager_id</code> — the centre's manager account.
users	centers	N : 1	<code>users.center_id</code> — the tenant an account belongs to. Null for system administrators.

From	To	Cardinality	Field and meaning
teachers	users	1 : 1	<code>teachers.user_id</code> — the account bound to this employment record. Absent for external teachers.
teachers	centers	N : 1	<code>teachers.center_id</code> .
halaqat	teachers	N : 1	<code>halaqat.teacher_id</code> — a teacher may hold several circles; a circle has one teacher. <code>teacher_name</code> is denormalised for listing.
halaqat	centers	N : 1	<code>halaqat.center_id</code> .
students	halaqat	N : 1	<code>students.halaqah_id</code> (+ denormalised <code>halaqah_name</code>). This field is circle membership — there is no join table.
students	users	1 : 1	<code>students.user_id</code> — the student's own account, issued after enrolment.
students	users	N : 1	<code>students.parent_user_id</code> — the guardian's account; one guardian may hold several children.
recitations	students	N : 1	<code>recitations.student_id</code> , with <code>center_id</code> denormalised so that centre-scoped activity queries need no join.
recitations	teachers	N : 1	<code>recitations.teacher_id</code> — the <i>teacher record</i> ID, resolved from the token, never the account ID.
attendance	students / halaqat	N : 1	<code>student_id</code> + <code>halaqah_id</code> , unique on (<code>student_id</code> , <code>date_str</code>).
fees	students	N : 1	<code>fees.student_id</code> ; <code>center_id</code> copied from the student at creation.
salaries	teachers	N : 1	<code>salaries.teacher_id</code> + <code>month</code> , unique in effect by the 409 guard.
academic_schedules	halaqat / teachers	N : 1	<code>halaqa_id</code> , <code>teacher_id</code> .
teacher_evaluations	teachers	N : 1	<code>teacher_id</code> , scoped by <code>center_id</code> .
competitions	teachers	1 : N	<code>competitions.judges[]</code> — an embedded array of { <code>teacher_id</code> , <code>teacher_name</code> }; names are read from the teacher record, never from the request.
competition_contestants	competitions / students	N : 1	<code>competition_id</code> + <code>student_id</code> ; one registration per student per competition.
competition_contestants	(embedded)	1 : N	<code>judge_scores[]</code> — one entry per judge, each carrying the judge's identity, marks and timestamp.
certificates	students	N : 1	Unique per (student, milestone) and per (competition, student), enforced by partial unique indexes.
xp_ledger	students	N : 1	Unique per (student_id, reason, source_id) — the mechanism that makes awarding idempotent.

From	To	Cardinality	Field and meaning
student_badges	students	N : 1	Unique per (student_id, badge_code).
bulk_messages	users / files	N : 1 / 1 : N	sender_id ; attachments[].file_id into GridFS; replies[] embedded.
notifications	users	N : 1	user_id ; data carries the context (e.g. the student the alert concerns).
audit_logs	users / centers	N : 1	actor_id , center_id ; chained to the previous entry by previous_hash .
centers	files	1 : 1	centers.logo_file_id into GridFS.

7.4.1 The access path expressed as relationships

The authorisation chain of §3.2 is a traversal of four of these relationships in reverse: `users._id` → `teachers.user_id` → `halaqat.teacher_id` → `students.halaqah_id` . Because each link is a plain indexed field rather than a materialised permission table, changing a circle's teacher changes that teacher's visibility on the next request, with nothing to synchronise and no stale grant to revoke.

7.5 Data dictionary — core entities

Field types are given as stored in MongoDB. **PK** marks the primary key, **FK** a reference to another collection, **ENC** a field encrypted at rest, and **IDX** a field participating in at least one index.

Table 7.3 — users

Field	Type	Keys	Description
_id	ObjectId	PK	Account identifier; referenced as a string by <code>teachers.user_id</code> , <code>students.user_id</code> , <code>students.parent_user_id</code> and <code>centers.manager_id</code> .
username	string	IDX unique	Login name. The unique index is the only defence against a race between two simultaneous registrations of the same name; the code catches the resulting duplicate-key error rather than relying on a prior read.
name	string		Display name. Self-editable and therefore never an ownership identifier.
email	string?		Optional contact address.
phone	string?	IDX	Used as a legacy ownership match for student and guardian accounts issued before identifier binding.
role	enum	IDX	One of <code>super_admin</code> , <code>admin</code> , <code>center_manager</code> , <code>teacher</code> , <code>student</code> , <code>parent</code> .
center_id	string?	FK IDX	Tenant binding. Null for system-wide roles; an absent value for a centre-scoped role yields an empty scope, not a global one.
hashed_password	string		bcrypt hash at cost 12. Plaintext is never stored, logged or returned.
is_active	bool		Disabled accounts are rejected at login <i>and</i> when presenting a previously valid token.
user_version	int		Incremented on every password change. A token carrying a stale UV claim is rejected, so one password change logs out every device.
password_changed_at	datetime?		Second invalidation check: a token issued before this instant is rejected.
approval_status	string?		On a manager account created by public registration: <code>pending</code> until an administrator approves.
created_at	datetime		Creation timestamp (naive UTC).

Table 7.4 — centers

Field	Type	Keys	Description
_id	ObjectId	PK	The tenant identifier carried by almost every other collection.
name	string		Centre name, printed on certificates and reports.
address	string		Physical address.
phone	string?		Contact number.

Field	Type	Keys	Description
manager_id	string?	FK IDX	The manager's account. Indexed because the manager → centre lookup runs on every scoped request.
manager_name	string?		Denormalised for listings.
currency	string		Default <code>FCFA</code> ; used by the finance screens.
status	string		Commercial state, default <code>trial</code> .
approval_status	string?		<code>pending</code> / <code>approved</code> / <code>rejected</code> for publicly registered centres.
is_active	bool		False until approved, and settable to false by the super-administrator to suspend a centre.
logo_file_id	string?	FK	GridFS identifier of the centre's logo. Uploadable only by that centre's own manager — an earlier defect let a system administrator attach a logo to an arbitrary centre.
created_at	datetime		Creation timestamp.

Table 7.5 — students

Field	Type	Keys	Description
_id	ObjectId	PK	Student identifier, referenced by every academic and financial record.
name	string		Full name.
date_of_birth	string	ENC	Encrypted at rest with Fernet. Decrypted only on an authorised read.
parent_name	string?	ENC	Encrypted: it identifies the guardian of a minor.
guardian_address	string?	ENC	Encrypted.
phone	string?	IDX	Deliberately not encrypted: it is one of the ownership-matching keys used by <code>check_student_access</code> and must be queryable.
parent_phone	string?	IDX	Same rationale; also the fallback link for guardian accounts predating identifier binding, and the key used to find the guardian on an absence alert.
center_id	string	FK IDX	Tenant.
halaqah_id	string?	FK IDX	Circle membership. A null value places the student outside every teacher's scope.
halaqah_name	string?		Denormalised circle name, used on certificates and listings.
memorization_plan	enum?		<code>plan_2_years</code> ... <code>plan_5_years</code> , or <code>plan_review</code> .
student_type	enum		<code>memorizing</code> or <code>reviewing</code> .
user_id	string?	FK IDX	The student's own account, issued by the manager after enrolment. The primary ownership key.
parent_user_id	string?	FK	The guardian's account — an identifier link that survives a phone number change.

Field	Type	Keys	Description
enrollment_date	datetime		Enrolment timestamp.
progress	float		Legacy percentage figure retained for older screens. <i>Not</i> the source of any metric in Chapter 9.
current_surah / current_ayah	string? / int?		Last recorded position.
is_active	bool	IDX	Compound-indexed with <code>center_id</code> for the common “active students of this centre” query.
is_deleted	bool?		Soft-delete marker. Filters use <code>\$ne: true</code> so that historical rows without the field remain visible.

Table 7.6 — teachers

Field	Type	Keys	Description
_id	ObjectId	PK	The teacher-record identifier — the value written to <code>recitations.teacher_id</code> and <code>halaqat.teacher_id</code> . Distinct from the account ID, and confusing the two previously made recitation recording fail for every teacher.
user_id	string?	FK IDX	The bound account. Absent for external teachers, who have none.
name	string		Teacher name.
phone	string?		Contact number.
center_id	string	FK IDX	Tenant.
teacher_type	enum		<code>halaqah</code> (circle teacher) or <code>external</code> (no account, narrower scope, not wider).
job_title	string?		Free-text post for external and administrative staff.
specialization	string?		Recitation transmission or subject (e.g. <code>حفص عن عاصم</code>).
marital_status	enum?		Redacted from responses for every role outside centre administration.
work_schedule	enum?		<code>full_time</code> / <code>part_time</code> .
salary	float?		Redacted outside centre administration. Exposing it previously let every student, guardian and colleague read every teacher's pay.
is_active	bool		Only active records resolve a scope; deactivation removes visibility immediately.
hire_date	datetime		Employment start.

Table 7.7 — halaqat (study circles)

Field	Type	Keys	Description
_id	ObjectId	PK	Circle identifier — the unit of the teacher's data scope.
name	string		Circle name (e.g. <code>حلقة الفجر</code>).

Field	Type	Keys	Description
teacher_id	string?	FK IDX	The teaching teacher's record ID. This field, with <code>is_active</code> , is the teacher's permission grant.
teacher_name	string?		Denormalised for listings and for the student's view of “my circle and its teacher”.
center_id	string	FK IDX	Tenant.
schedule	string		Human-readable recurrence (e.g. “daily after Fajr”).
time / location	string?		Time span and room.
max_students	int		Capacity, default 20.
current_students	int		Occupancy counter, recomputed on membership change.
is_active	bool		Inactive circles drop out of every teacher scope resolution.

7.6 Data dictionary — academic records

Table 7.8 — `recitations`

Field	Type	Keys	Description
_id	ObjectId	PK	Session identifier; also the <code>source_id</code> of the XP award for that session.
student_id	string	FK IDX	Subject of the session. Compound-indexed with <code>date</code> descending for the timeline, and with <code>date_str</code> for the daily uniqueness check.
student_name	string?		Denormalised.
teacher_id	string	FK IDX	The teacher record that recorded it, taken from the token , never from the request body.
teacher_name	string?		Denormalised.
center_id	string	FK IDX	Copied from the student. Added because recitations were the only table without it, forcing every centre-activity query through the students collection.
surah_number	int?		Sūrah number 1–114.
surah_name	string		Sūrah name as entered.
start_ayah / end_ayah	int		Recited range. Used to derive a page count when one is not entered.
pages_count	float?		Explicit page count, validated to (0, 604]. When present it takes precedence over derivation; all metrics are per page.
recitation_type	enum		<code>new</code> (fresh memorisation) or <code>review</code> . Momentum counts only <code>new</code> ; review depth is the ratio between the two.
evaluation	enum		<code>excellent</code> / <code>good</code> / <code>acceptable</code> / <code>needs_improvement</code> — the teacher's overall judgement, input to the legacy composite score.

Field	Type	Keys	Description
error_tags	object?		Map of error class code → count, validated against the closed taxonomy. An empty map means “tagged, no errors found” (perfect precision); an absent field means “predates tagging” and yields no precision value at all.
mistakes_count hesitations_count tajweed_errors_count	int		Legacy free counters, retained for historical data and older clients. Mapped into the taxonomy on read, and superseded entirely when <code>error_tags</code> is present so that no error is counted twice.
notes	string?		Teacher's free note.
date	datetime	IDX	Exact timestamp.
date_str	string	IDX	YYYY-MM-DD day key. Written explicitly because a millisecond timestamp supports neither a daily uniqueness check nor a day index.
is_deleted	bool?		Soft delete: the row survives for audit, and leaves every metric, score and forecast.

Table 7.9 — attendance

Field	Type	Keys	Description
_id	ObjectId	PK	
student_id	string	FK IDX	Part of the unique key <code>uniq_student_day</code> .
student_name	string?		Denormalised.
halaqah_id	string	FK IDX	The circle whose roll was taken; compound-indexed with <code>date_str</code> for the daily roll view.
center_id	string	FK IDX	Copied from the student; without it the dashboard attendance rate returned empty.
status	enum		<code>present / absent / late / excused</code> . Present, late and excused all count as attendance for the consistency metric.
notes	string?		Reason or comment.
date	datetime	IDX	Session timestamp.
date_str	string	IDX unique with student	Day key. The unique index makes re-submission of the roll an <i>update</i> , which is what a teacher correcting a mark expects — previously it inserted a second, contradictory row and the absence stayed in the attendance rate forever.
recorded_by	string	FK	The account that took the roll.

Table 7.10 — academic_schedules and review_plans

Field	Type	Description
academic_schedules — the weekly timetable		
subject	string	Subject taught in the slot.

Field	Type	Description
day / time_slot	string	Weekday and period; compound-indexed with <code>center_id</code> .
halaqa_id / teacher_id	string	References to the circle and the teacher, with denormalised names on read.
center_id	string?	Tenant. Optional on the request model because a system administrator is bound to no centre — reading it as a required field previously produced a 500.
room_number	string?	Room.
review_plans — review programme		
center_id	string	Tenant; compound-indexed with <code>created_at</code> .
juz_per_day	float?	The rate that personalises the daily portion. Set by the teacher when the default of one Juz' per day does not suit a student — a young beginner or an accomplished ḥāfiẓ.
student_id / halaqah_id / teacher_id	string?	Target of the plan.
title / description / notes	string?	Free text.
surahs	string[]?	Named sūrahs in the plan.
start_date / end_date	string?	Plan period.

7.7 Data dictionary — financial records

Table 7.11 — `fees` , `salaries` , `expenses`

Field	Type	Description
fees		
student_id / student_name	string	Subject of the charge; indexed alone and with <code>status</code> .
center_id	string	Copied from the student at creation; indexed with <code>status</code> for the outstanding-fees dashboard.
amount	float	Charge in the centre's currency.
fee_type	enum	<code>monthly / annual / registration</code> . Monthly fees are guarded against duplication per student per due month.
due_date	string	Due date; the month component participates in the duplicate guard.
status	enum	<code>pending / paid / overdue</code> .
paid_date	datetime?	Set when collected; a second collection attempt returns 409.
created_at / created_by	datetime / string	Provenance.
salaries		
teacher_id / teacher_name	string	Payee.
month	string	<code>YYYY-MM</code> . The triple (teacher, month, centre) must be unique among non-voided rows — a duplicate is money leaving twice.

Field	Type	Description
amount / notes	float / string?	Payment.
center_id / created_at / status	string / datetime / string	Tenant, timestamp and state (<code>paid</code>).
expenses		
title / category / amount / date	string / string? / float / string	Expenditure line.
center_id / created_at	string / datetime	Tenant and timestamp; compound-indexed.
Void fields — present on all three collections		
voided	bool?	Set instead of deletion. Every aggregate filters with <code>{voided: {\$ne: true}}</code> — marking a row without excluding it from the sums would be a cancellation on screen and money still present in the figures.
void_reason	string?	Mandatory written reason.
voided_at / voided_by	datetime / string	When and by whom.

7.8 Data dictionary — competitions and certificates

Table 7.12 — `competitions` and `competition_contestants`

Field	Type	Description
competitions		
title / date / year	string / string / int	Identity. The year is derived from the date when not supplied, and drives the archive-by-year view.
center_id	string	Tenant; indexed.
branches	string[]	Selected from the closed list: whole Qur'an, 15 parts, 10 parts, 5 parts, Juz' 'Ammā. A branch outside the list is rejected with 400 — free text would make results impossible to aggregate or archive across centres.
judges	object[]	<code>{teacher_id, teacher_name}</code> . Each judge is verified to belong to the centre, and the <i>name</i> is read from the teacher record, not accepted from the request.
status	enum	<code>draft</code> → <code>active</code> → <code>grading</code> → <code>approved</code> → <code>archived</code> . Transition to <code>grading</code> happens automatically on the first mark.
approved_at / approved_by	datetime? / string?	Approval freezes the competition against all further change.
notes / created_at	string? / datetime	
competition_contestants		
competition_id / student_id	string	Compound-indexed; one registration per student per competition.

Field	Type	Description
center_id / student_name / halaqah_name	string	Tenant and denormalised display fields.
category	string	The branch entered. Ranking is computed <i>within</i> the branch, each from position one.
judge_scores	object[]	One entry per judge: <code>judge_teacher_id</code> , <code>judge_name</code> , <code>hifdh_score</code> (0–70), <code>tajweed_score</code> (0–20), <code>voice_score</code> (0–10), <code>total</code> , <code>notes</code> , <code>graded_at</code> . Re-marking replaces only that judge's entry.
total_score / judges_count	float / int	The mean of the panel's totals and the number of judges who have marked. Previously a single score on the contestant meant the last judge to open the screen silently erased his colleague's marking.
rank_in_branch	int?	Position within the branch, computed on read.
grades	object?	Legacy single-grade structure retained for older records.

Table 7.13 — certificates

Field	Type	Description
serial	string	<code>MK-YYYY-NNNNN</code> , allocated from an atomic counter keyed by centre and year. A <code>count + 1</code> scheme would give two simultaneous issues the same number — exactly what happens on an awards day. Globally unique index.
kind	enum	<code>milestone</code> or <code>competition</code> .
center_id / center_name	string	Issuing centre; compound-indexed with <code>issued_at</code> .
student_id / student_name / halaqah_name	string	Recipient.
title / subtitle	string	Printed headings, e.g. “Certificate of memorisation and honour” with the milestone label.
milestone	enum?	<code>juz5</code> / <code>juz10</code> / <code>half</code> / <code>khatm</code> — 5, 10, 15 and 30 parts. Unique per student, enforced by a partial unique index.
pages_memorized	float?	The measured basis of the award at the moment of issue.
competition_id / branch / rank / score / judges_names	string? / string? / int? / float? / string[]	Competition certificates. Unique per (competition, student) by partial unique index.
issued_by / issued_by_name / issued_at	string / string / datetime	Issuance is a recorded act, not an ephemeral print: a certificate that is later questioned can be traced in the record, and it cannot be printed twice unknowingly.

Table 7.14 — teacher_evaluations

Field	Type	Description
teacher_id / center_id	string	Subject and tenant; compound-indexed.
evaluation_date	string	Date of the assessment.

Field	Type	Description
performance, commitment, sincerity, seriousness, diligence	int?	The five criteria, each validated to 0–10. Together they total one hundred.
strengths / weaknesses	string[]	Written observations. The point of the design is that the score is never a single opaque number whose composition is unknown.
tpi	float	The computed total.
attendance_rate, tajweed_proficiency, student_retention, average_memorization_speed, discipline	optional	Superseded criteria, kept optional so that earlier evaluations remain readable.
evaluated_by / teacher_name / notes / created_at	string / string / string? / datetime	Provenance. Readable by the teacher concerned and by administration only.

7.9 Data dictionary — analytics and motivation

Table 7.15 — `xp_ledger` and `student_badges`

Field	Type	Description
<code>xp_ledger</code> — append-only		
student_id	string	Recipient; indexed with <code>created_at</code> .
center_id	string?	Tenant, for centre-wide queries.
amount	int	Points granted: 10 per page recited, 5 per attended session, plus badge awards.
reason	string	<code>recitation</code> , <code>attendance</code> , <code>badge</code> .
source_id	string?	The originating record: a recitation ID, a <code>student:date</code> pair for attendance, or a badge code. The unique index on (student_id, reason, source_id) is what makes awarding idempotent — re-submitting a roll or reprocessing a session grants nothing twice, and the resulting duplicate-key error is treated as success.
created_at	datetime	Award time. Rows are never updated or deleted.
<code>student_badges</code>		
student_id / badge_code	string	Unique together: a badge is earned once.
center_id / earned_at	string? / datetime	Tenant and time. A badge is never withdrawn — it certifies an achievement that occurred; the current state is what the metrics and boards show.

7.10 Data dictionary — communication and governance

Table 7.16 — `bulk_messages`, `notifications`, `audit_logs`, `counters`

Field	Type	Description
<code>bulk_messages</code>		

Field	Type	Description
center_id / sender_id / sender_role / sender_name	string	Origin; compound-indexed on (centre, sender). The role is stored so that the chain rule can be evaluated on read.
recipient_role	string	The audience, validated server-side against the chain of §3.5.
subject / content	string	Message body.
attachments	object[]	<code>{file_id, filename, content_type, size}</code> , uploaded first and referenced here. Download authorisation is evaluated on the message, never on the file identifier alone.
replies	object[]	Embedded <code>{teacher_id, teacher_name, content, timestamp}</code> .
sent_at / edited_at	datetime / datetime?	A correction to a sent message is visible as such.
notifications — TTL 180 days		
user_id	string	Recipient; indexed with <code>created_at</code> both ascending (for the stream) and descending (for the list), and with <code>read</code> .
type / title / body / data	string / string / string / object	Event type (e.g. <code>absentee_alert</code>), display text and structured context.
read / created_at	bool / datetime	Read state; <code>created_at</code> also carries the TTL that expires the row after 180 days.
audit_logs		
seq	int	Atomic sequence from <code>counters</code> ; unique via a partial index that tolerates pre-sequencing rows. The chain is built on this order, not on timestamps — two simultaneous writes previously read the same predecessor and split the chain silently.
actor_id / center_id / action	string	Who, where, what. Thirty-five action types are catalogued in Appendix B.
payload	object	Action-specific detail, serialised deterministically (sorted keys) when hashed.
timestamp	datetime	Truncated to milliseconds before hashing, because MongoDB stores only millisecond precision — hashing microseconds would make every recomputed hash differ from the stored one and report tampering in a sound record.
client_ip	string	Origin address, or <code>system</code> .
previous_hash / log_hash	string	SHA-256 chain (§10.7).
chain_gap	bool?	Set when the predecessor could not be resolved. The record says so explicitly rather than pretending the chain is intact.
purge_at	datetime?	Set to +730 days for ordinary actions; absent for financial and permission actions, which are never purged.
counters		
_id / seq	string / int	Atomic sequence generator. Keys in use: <code>audit_logs</code> and <code>cert:<center_id>:<year></code> .

7.11 Data dictionary — security and ephemeral state

Table 7.17 — Security collections (all TTL-managed)

Collection	Key structure	Fields and lifetime
revoked_tokens	jti	jti (unique), expires_at (TTL 0 — deleted the moment the token would have expired anyway, so the blacklist never grows without bound).
login_attempts	_id = "ip:<addr>" or "user:<name>"	count, last_seen (TTL ≥ 1 h), locked_until. Two independent keys: per-IP (5 attempts / 15 min) and per-account (10 / 15 min). An IP-only limiter is evaded by address rotation; behind a proxy it locks out every user at once.
register_attempts	_id = IP address	timestamps[], last_seen (TTL 2 h). Three successful registrations per hour. Recorded on success only — previously three validation errors consumed the whole hour's quota and locked out someone who had not registered once.
reset_requests	_id = "ip:..." or "user:..."	timestamps[], last_seen (TTL 2 h). Five reset requests per hour per key.
password_resets	username (unique)	code_hash (HMAC-SHA256 of “username:code” under the application secret, so a code for one account is useless for another), expires_at (TTL 0, default 10 minutes), attempts (five wrong guesses invalidate), requested_from_ip, created_at. Any earlier plaintext code field is actively unset on write.
stream_tickets	ticket (unique)	user_id, created_at (TTL 300 s sweep), expires_at (60 s validity enforced in code). Consumed by an atomic find-and-delete, so a ticket cannot be used twice even by two simultaneous requests.
files.files / files.chunks	GridFS	Metadata carries kind, center_id, owner_id, the sniffed content_type, size and uploaded_at.

7.12 Indexing strategy

Seventy-one index definitions are declared in `app/startup.py` and created on every boot. Creation is idempotent and defensive: a definition that conflicts with an existing index of the same key (MongoDB error 85, `IndexOptionsConflict`) causes the old index to be dropped and recreated rather than aborting the boot, and a unique index that cannot be built because legacy duplicates exist logs an explicit warning and leaves the service running — **activating the constraint would stop the whole deployment; the warning tells the operator to clean the duplicates and restart so that the guarantee becomes real.**

Table 7.18 — Index catalogue

Collection	Index key	Properties	Query it serves
users	username	unique	Login; also the race guard on registration.
users	role · center_id · phone	—	Role filters, tenant listing, guardian matching.
students	center_id · halaqah_id	—	Tenant and circle scope resolution.
students	(center_id, is_active)	compound	“Active students of this centre”, the most frequent scoped query.
students	user_id · phone · parent_phone	—	Ownership lookups inside <code>check_student_access</code> .
teachers	center_id · user_id	—	Tenant listing; the account → teacher-record link in the scope chain.
halaqat	center_id · teacher_id	—	Tenant listing; the teacher → circles link in the scope chain.
centers	manager_id	—	Manager → centre resolution.
recitations	student_id	—	Per-student history.
recitations	(student_id, date ↓)	compound	Timeline and metric computation.
recitations	(student_id, date_str)	compound	The once-per-day guard. Deliberately not unique: a student may have one new-memorisation and one review session on the same day, and the distinction belongs in code, not in the key — and legacy rows carry no <code>date_str</code> at all.
recitations	teacher_id · (center_id, date ↓)	—	Per-teacher views; centre activity measurement.

Collection	Index key	Properties	Query it serves
attendance	(student_id, date_str)	unique	<code>uniq_student_day</code> — the business rule, not a hint. Re-submitting the roll updates rather than inserting.
attendance	student_id · (student_id, date ↓) · halaqah_id · (halaqah_id, date_str) · (center_id, date ↓)	—	Per-student history, per-circle roll for a day, per-centre attendance rate.
fees	student_id · status · (center_id, status) · (student_id, status)	—	Outstanding-fee dashboards, per-student fee views.
salaries	teacher_id · (center_id, created_at ↓) · (center_id, month)	—	Listings and the duplicate-payment guard.
expenses	(center_id, created_at ↓)	compound	Financial reports.
review_plans	(center_id, created_at ↓) · teacher_id	—	Plan listings.
academic_schedules	(center_id, day, time_slot)	compound	Weekly timetable retrieval.
teacher_evaluations	(center_id, teacher_id)	compound	Per-teacher evaluation history.
competitions	center_id	—	Tenant listing.
competition_contestants	(competition_id, student_id)	compound	Contestant lookup and duplicate-registration check.
certificates	(student_id, milestone)	unique, partial <code>kind = milestone</code>	One milestone certificate per student — enforced in the database, because a code check followed by a write succeeds twice when the button is pressed twice at once.
certificates	(competition_id, student_id)	unique, partial <code>kind = competition</code>	One competition certificate per contestant.
certificates	serial · (center_id, issued_at ↓)	unique / compound	Serial uniqueness; centre certificate register.
xp_ledger	(student_id, reason, source_id)	unique	Idempotent awarding. Without it any repeat can inflate a student's balance without limit.
xp_ledger	(student_id, created_at ↓) · (center_id, created_at ↓)	—	Balances and centre-wide totals.
student_badges	(student_id, badge_code)	unique	One award per badge per student.
student_badges	(center_id, earned_at ↓)	compound	Recent awards in a centre.
bulk_messages	(center_id, sender_id)	compound	Inbox and outbox.

Collection	Index key	Properties	Query it serves
notifications	(user_id, created_at ↓) · (user_id, created_at ↑) · (user_id, read)	—	The list view, the ascending stream poll, and the unread badge.
notifications	created_at	TTL 180 d	Automatic retention limit.
audit_logs	seq	unique, partial (numeric only)	Chain ordering; partial so that pre-sequencing rows do not block the index.
audit_logs	timestamp · action · (center_id, timestamp ↓)	—	Chronological paging and filtering.
revoked_tokens	jti / expires_at	unique / TTL 0	Blacklist lookup; self-cleaning at natural token expiry.
login_attempts	last_seen	TTL ≥ 1 h	Lockout state garbage collection.
register_attempts	last_seen	TTL 2 h	Registration window.
reset_requests	last_seen	TTL 2 h	Reset-request window.
password_resets	username / expires_at	unique / TTL 0	One live code per account; expired codes delete themselves instead of accumulating.
stream_tickets	ticket / created_at	unique / TTL 300 s	Single-use stream tickets.

7.13 Integrity, retention and deletion semantics

7.13.1 Referential integrity

References are string-encoded `ObjectId` values in ordinary fields. There is no cascading delete. Integrity is maintained by three mechanisms instead: **(a)** every write validates that the referenced document exists and lies inside the caller's scope before inserting — a competition judge must belong to the centre, an attendance record's student must belong to the named circle, a salary's teacher must belong to the paying centre; **(b)** deletion is predominantly soft, so dangling references do not arise; **(c)** reads tolerate a missing referent by rendering a placeholder rather than failing.

7.13.2 Three deletion semantics

Table 7.19 — Deletion semantics by record class

Semantics	Applies to	Behaviour
Soft delete	Recitations, students	<code>is_deleted: true</code> . The row survives for audit but leaves every metric, score, leaderboard and forecast. Filters are written <code>{is_deleted: {\$ne: true}}</code> and never <code>{is_deleted: false}</code> , because historical rows do not carry the field at all and an equality test would hide every one of them.

Semantics	Applies to	Behaviour
Void	Fees, salaries, expenses	<code>voided: true</code> with a mandatory reason, actor and time. The row is excluded from every total. Money is cancelled, never erased — and a void that did not also remove the amount from the sums would be a cancellation on screen and money still present in the figures.
Hard delete	Attendance rows, competition registrations, messages, certificates (revocation)	Physical removal, always accompanied by an audit entry naming the actor and the removed record.

7.13.3 Retention

- **Never purged:** audit entries for financial actions (fee collection, salary payment, expense creation and deletion) and for permission and credential actions (password changes and resets, centre registration, approval, rejection and status change). These carry no `purge_at` field.
- **Two years:** all other audit entries carry `purge_at = timestamp + 730 days`.
- **180 days:** notifications, expired automatically by a TTL index on `created_at`.
- **Minutes to hours:** every security collection, expired by TTL so that transient state cannot grow without bound in a long-running deployment.
- **Indefinite:** all academic, financial and identity records, subject to the centre's own policy.

7.13.4 Normalisation posture

The schema is normalised where correctness depends on it and denormalised where display does. Error severity weights are the clearest case: the weight is *not* stored with the error. Only the class code is stored, and the weight lives in one module. This is a deliberate removal of a transitive dependency — the weights are declared provisional and will be recalibrated on real data, and had they been written alongside each error, recalibration could never repair historical records and the same error would carry different weights in different rows.

Conversely, names are denormalised freely: `student_name` on a recitation, `teacher_name` on a circle, `halaqah_name` on a student, `center_name` on a certificate. These are display copies on records that are already historical — a certificate should print the centre name as it was at issue — and they remove a join from every list view. `center_id` is copied onto recitations, attendance and fees for the same reason: it converts a two-collection scope query into a single indexed lookup.

API Specification

8.1 Conventions

- All endpoints are prefixed `/api` and exchange JSON, except the login form (multipart), file upload (multipart), CSV export (`text/csv`), logo retrieval (image) and the notification stream (`text/event-stream`).
- Authentication is a bearer access token in the `Authorization` header. Only the refresh endpoint relies on a cookie. Unauthenticated endpoints are exactly five: health, readiness, public centre registration, the public best-centres listing, and login.
- **Scope is applied server-side to every list endpoint.** A query parameter may *narrow* within the caller's scope; it can never widen it. Passing `?haLaqah_id=` or a path identifier outside the caller's scope returns 403, not a filtered 200.
- List endpoints accept `limit` and `skip` where volume warrants it, with server-enforced ceilings.
- Every identifier supplied by a client is converted through `safe_object_id`, which returns HTTP 400 for a malformed value rather than allowing a 500.
- Interactive OpenAPI documentation is served at `/api/docs` in development and is disabled entirely — schema included — in production.

8.2 Endpoint catalogue

Table 8.1 — Authentication (`auth.py`)

Method	Path	Access	Purpose
POST	<code>/api/auth/login</code>	public	Authenticate; returns access token, user, expiry, and sets the refresh cookie. Rate limited per IP and per account; logs <code>LOGIN_SUCCESS</code> , <code>LOGIN_FAILED</code> or <code>LOGIN_DENIED_PENDING</code> .
POST	<code>/api/auth/refresh</code>	cookie	Rotate the refresh token and issue a new access token; the old refresh token is revoked.
GET	<code>/api/auth/me</code>	any	The authenticated user's profile.
POST	<code>/api/auth/logout</code>	any	Revoke tokens server-side and clear the cookie.
POST	<code>/api/auth/change-password</code>	any	Change own password; bumps <code>user_version</code> , invalidating all other sessions.
POST	<code>/api/auth/forgot-password</code>	public	Issue a hashed, time-limited reset code; rate limited by IP and account.
GET	<code>/api/auth/reset-codes</code>	admin	Administrative retrieval of outstanding reset requests (no mail transport exists).
POST	<code>/api/auth/reset-password</code>	public + code	Complete a reset; five wrong attempts invalidate the code.
POST	<code>/api/auth/admin-reset-password</code>	admin	Direct reset by an administrator; permanently audited.
PUT	<code>/api/auth/profile</code>	any	Update own name, email and phone.

Table 8.2 — Centres and supervision (`centers.py` , `center_activity.py`)

Method	Path	Access	Purpose
GET	/api/centers	scoped	List centres; a non-administrator sees only their own.
POST	/api/centers	admin	Create a centre and, optionally, its manager account with complexity-checked credentials.
PUT	/api/centers/{id}	admin, own manager	Amend a centre.
DELETE	/api/centers/{id}	admin	Remove a centre.
GET	/api/centers/{id}/details	admin, own manager	Centre detail with counts.
GET	/api/centers/pending	admin	Centres awaiting a decision.
POST	/api/centers/{id}/approve	admin	Activate the centre and its manager; permanent audit entry.
POST	/api/centers/{id}/reject	admin	Reject the application; permanent audit entry.
POST/GET/DELETE	/api/centers/{id}/logo	own manager	Upload, serve and remove the centre logo (GridFS, content-sniffed, ≤ 512 KB).
POST	/api/public/register-center	public	Self-registration; creates an inactive centre and manager pending approval. 3 per hour per IP.
GET	/api/public/best-centers	public	Aggregate listing for the landing page.
GET	/api/admin/system/status	admin	Platform status summary.
GET	/api/admin/centers-activity	admin	Per-centre activity index over a window; counts and dates only, no student data.
POST	/api/super/centers	super_admin	Register a centre with its manager.
POST	/api/super/centers/{id}/status	super_admin	Suspend or reactivate a centre; permanently audited.
GET	/api/super/dashboard/stats	super_admin	Platform-wide statistics.

Table 8.3 — People, circles and timetable

Method	Path	Access	Purpose
GET	/api/students	all roles, scoped	Students within the caller's scope; a teacher sees only his circles', a student only himself.
GET	/api/students/{id}	scoped	Single student, through the ownership gate.
POST / PUT / DELETE	/api/students[/]{id}]	admin, manager	Enrol, amend, deactivate. Personal fields are encrypted on write.
GET / POST	/api/students/{id}/accounts	manager	List issued accounts (never passwords); issue student and/or guardian accounts.
POST	/api/students/{id}/accounts/reset	manager	Replace one holder's password; shown once, audited.

Method	Path	Access	Purpose
GET	/api/teachers	staff, scoped	Teacher list; salary and marital status redacted outside administration; a teacher sees only himself.
POST / PUT / DELETE	/api/teachers[/]{id}]	admin, manager	Create (optionally with an account for a circle teacher), amend, deactivate.
POST	/api/teachers/{id}/transfer	admin, manager	Move a teacher between circles; audited with both circles.
POST / GET / DELETE	/api/teachers/{id}/evaluations[/]{eid}]	manager; own teacher reads	Five-criterion evaluation with strengths and weaknesses.
GET	/api/halaqat	all roles, scoped	Circles within scope — a teacher sees his own, a student his own, not the whole centre.
POST / PUT / DELETE	/api/halaqat[/]{id}]	admin, manager	Circle lifecycle.
GET	/api/halaqat-overview	staff	Every circle of the centre with its aggregate performance in one row.
GET	/api/halaqat-overview/ranking	staff	Circle ranking in aggregate figures — readable by teachers, naming no student of another circle.
GET	/api/halaqat-overview/{id}	staff, in scope	Circle detail: teacher, evaluation, students and each student's level. A circle outside scope returns 403.
GET / POST / PUT / DELETE	/api/academic-schedules[/]{id}]	read: all scoped write: staff	Weekly timetable.

Table 8.4 — Academic records

Method	Path	Access	Purpose
GET	/api/recitations	all roles, scoped	Recitations within scope; a cross-scope <code>? student_id=</code> returns 403.
POST	/api/recitations	teacher only	Record a session. Teacher identity from the token; duplicate same-day session refused with 409; XP and badges granted idempotently.
GET	/api/recitations/student/{id}	scoped	One student's history.
PUT / DELETE	/api/recitations/{id}	recording teacher; administration	Correct or soft-delete; audited.
GET	/api/analytics/predict/{student_id}	scoped	Completion forecast for one student.
GET	/api/attendance	staff, scoped	Attendance within scope; a circle outside scope returns 403.
GET	/api/attendance/halaqah/{id}	staff, in scope	The roll for one circle on a date.

Method	Path	Access	Purpose
POST	/api/attendance	teacher only	Submit the roll; upserts on (student, day); absences notify guardians; attendance XP granted once per day.
GET	/api/attendance/student/{id}	scoped	One student's attendance history.
GET	/api/attendance/center	staff	Centre-wide attendance view.
PUT / DELETE	/api/attendance/{id}	teacher; administration	Correct a status or remove a row; audited.
GET / POST / PUT / DELETE	/api/review-plans[/ {id}]	staff	Review plans and the per-student daily rate.
GET	/api/review-plans/today	scoped	Today's derived review portion for every student in scope, plus who has reviewed and who is behind.

Table 8.5 — Finance

Method	Path	Access	Purpose
GET	/api/fees	scoped	Fees; a guardian sees their children's, a student their own.
POST	/api/fees	admin, manager	Create a charge; duplicate monthly fee refused with 409.
POST	/api/fees/{id}/pay	admin, manager	Record collection; a second attempt returns 409. Permanently audited.
PUT / DELETE	/api/fees/{id}	admin, manager	Correct; or void with a reason — never erase.
GET	/api/fees/student/{id}	scoped	One student's fees.
GET / POST	/api/salaries	admin, manager	List and pay salaries; a second payment for the same teacher and month is refused with 409 naming the existing amount.
PUT / DELETE	/api/salaries/{id}	admin, manager	Correct or void.
GET / POST	/api/expenses	admin, manager	List and create expenses.
PUT / DELETE	/api/expenses/{id}	admin, manager	Correct or void with a reason.
GET	/api/finance/summary	admin, manager	Collections, salaries, expenses and outstanding balances, all excluding voided rows.

Table 8.6 — Competitions and certificates

Method	Path	Access	Purpose
GET	/api/competitions/branches	any	The closed branch list.
GET	/api/competitions/my-role	any	Whether the caller is currently on a panel — drives menu visibility, since judging is a task with a term, not a standing permission.
GET / POST	/api/competitions	read: centre create: manager	List and create; judges are validated against the centre and their names read from their records.

Method	Path	Access	Purpose
GET	/api/competitions/years	centre	Years with competitions, for the archive.
PUT / DELETE	/api/competitions/{id}	manager	Amend or remove — refused once approved.
POST	/api/competitions/{id}/approve	manager	Approve results; freezes the competition. Audited.
POST	/api/competitions/{id}/archive	manager	Archive an approved competition.
POST	/api/competitions/{id}/register	manager, panel judge	Register a contestant in a branch; one registration per student.
GET	/api/competitions/{id}/contestants	centre, scoped	Contestants grouped by branch, each branch ranked from one.
POST	/api/competitions/contestants/{id}/grade	panel judge only	Record this judge's marks (memorisation ≤ 70 , tajwīd ≤ 20 , voice ≤ 10); recomputes the panel mean. Administration is refused: it approves the result, it does not set it.
DELETE	/api/competitions/contestants/{id}	manager	Withdraw a contestant.
GET	/api/certificates	scoped	Issued certificates; students and guardians see their own and print them.
GET	/api/certificates/eligible	manager	Students who have reached a milestone but hold no certificate for it.
POST	/api/certificates/milestone	manager	Issue a memorisation certificate; refused until the milestone is met in recited pages.
POST	/api/certificates/competition/{cid}	manager	Issue a competition certificate; refused before results are approved.
DELETE	/api/certificates/{id}	manager	Revoke one issued in error; audited as the issue was.

Table 8.7 — Analytics, communication and governance

Method	Path	Access	Purpose
GET	/api/dashboard/stats	any	Role-aware statistics; the role is inferred from the token, not taken from the path.
GET	/api/dashboard/honor-roll	scoped	Top students by the composite score, computed in two queries for the whole set.
GET	/api/analytics/rankings	scoped	Ranked students within scope.
GET	/api/analytics/student/{id}	scoped	Analytical profile of one student.
GET	/api/performance/error-types	any	The taxonomy with labels, descriptions and weights — so the client never hard-codes a weight.
GET	/api/performance/students/{id}	scoped	Five metrics, forecast with method and fit, badges, challenges and XP.

Method	Path	Access	Purpose
GET	/api/performance/students/{id}/journey	scoped	Position across the thirty parts.
GET	/api/performance/leaderboards	scoped	Four independent boards plus the caller's own position.
GET	/api/performance/challenges/{id}	scoped	Challenge progress computed from session records.
POST	/api/performance/recompute	manager	Re-evaluate badges across a centre after enabling the layer on historical data.
GET	/api/messages/audiences	any	Whom the caller may address — the authoritative list.
POST	/api/messages/attachments	sender roles	Upload a document (≤ 5 MB; PNG, JPEG, WEBP or PDF by content sniffing).
POST	/api/messages/broadcast	sender roles	Send to a permitted audience.
GET	/api/messages/inbox · /broadcasts	any	Received and sent messages.
POST	/api/messages/{id}/reply	recipient	Reply on the thread.
PUT / DELETE	/api/messages/{id}	sender	Correct or withdraw a sent message.
GET	/api/messages/{id}/attachments/{fid}	message recipients	Download an attachment — authorised on the message, not on the file identifier.
GET	/api/notifications	any	Stored notifications.
POST	/api/notifications/{id}/read · /read-all	any	Mark read.
POST	/api/notifications/stream-ticket	any	Obtain a single-use, 60-second ticket with a normal Authorization header.
GET	/api/notifications/stream	ticket	Server-sent event stream, reading from the database so it works from any worker.
GET	/api/audit-logs	admin	Filterable audit listing.
GET	/api/audit-logs/verify	admin	Recompute the hash chain and report breaks and marked gaps.
GET	/api/export/students.csv	admin, manager	UTF-8 CSV with BOM; audited as <code>EXPORT_STUDENTS</code> .
GET	/api/export/attendance.csv	admin, manager	Attendance export for a date range.
GET	/api/health · /api/health/ready	public	Liveness and database readiness.

8.3 Error model

Table 8.8 — Status codes and their meaning in this API

Code	Meaning here	Representative causes
200	Success	Note that a 200 carrying an empty list is the correct answer for an empty scope.

Code	Meaning here	Representative causes
400	Malformed input	Invalid <code>ObjectId</code> ; unknown error-tag code; page count outside (0, 604]; unrecognised competition branch; credentials supplied for an external teacher; username already taken.
401	Not authenticated	Missing, expired, malformed or revoked token; token type mismatch; stale <code>user_version</code> ; token issued before the last password change.
403	Authenticated but not permitted	Out-of-scope circle or student; an administrator attempting to record a recitation; a non-judge attempting to mark; a disabled account presenting a valid token. Never treated by the client as session expiry.
404	Not found	Missing record, or a file identifier that does not resolve.
409	Conflicting state	The duplicate guards: a same-day recitation, a same-month salary, a same-month fee, an already-paid fee, an already-voided record, a certificate already issued, a milestone not yet reached, an edit to an approved competition.
413	Payload too large	Logo over 512 KB or document over 5 MB, detected during chunked reading rather than after buffering the whole upload.
429	Rate limited	Login lockout (per IP or per account), registration window exceeded, reset-request window exceeded. The message states the remaining time.
500	Internal error	Returns <code>{detail, request_id}</code> only. The traceback is logged with the same request identifier and never sent to the client.

Error messages are written in Arabic for the user and are deliberately specific where specificity helps (“a memorisation session was already recorded for this student today — edit the existing record”) and deliberately generic where specificity would help an attacker (password reset returns the same response whether or not the account exists).

The Analytical Engine

This chapter specifies the part of Mishkaat that distinguishes it from a records system: the layer that converts recitation and attendance rows into measures of academic quality, a completion forecast, and a motivation structure. Every formula, constant and design constraint is stated, together with the reason it takes the form it does.

9.1 Provenance and scientific basis

The analytical layer derives from the requirements specification of an academic project, *Halaqtna* (CPIT-498, King Abdulaziz University), and was carried into Mishkaat with its requirement identifiers preserved so that the mapping can be audited rather than reconstructed. The error taxonomy rests on the observation — recorded in the literature on Qur'an memorisation assessment, including the classification of Hassan and Zailaini — that an error recorded as free text **cannot be compared** either between students or across time.

Table 9.1 — Requirement-to-implementation mapping

Req.	Statement	Implementation
FR6	Tag each error with one of four types, each carrying a severity weight	<code>app/errors_taxonomy.py</code>
FR7	A mastery index per student	<code>metrics.mastery</code>
FR8	Momentum as smoothed weekly pages	<code>metrics.momentum</code>
FR9	Precision, consistency and review depth	<code>metrics.precision / consistency / review_depth</code>
FR10	Forecast Juz' completion from momentum, error density and attendance	<code>app/forecast.py</code>
FR11	A leaderboard ordered by four separate criteria	<code>GET /api/performance/leaderboards</code>
FR12	Badges awarded automatically on threshold attainment	<code>gamification.evaluate_badges</code>
FR13	Experience points in a ledger that is not edited	<code>xp_ledger</code> + <code>award_xp</code>
FR15	Export one student's history as a sheet for the guardian	<code>StudentReportSheet.tsx</code>
FR16	The student sees only their own data and aggregate positions	<code>check_student_access</code>
FR18	An audit log that is not edited	Pre-existing in Mishkaat, with hash chaining (§10.7)

A STATEMENT ABOUT THE STATUS OF THESE NUMBERS

The source classifies every constant in this chapter as **provisional**: the ordering of severity comes from teacher interviews and from the published classification, but the numeric values are a design choice to be calibrated against real data. They are therefore held in configuration and are environment-tunable (§9.8), not buried inside a function — which is where an earlier generation of error weights (4.0 / 1.5 / 2.0) in fact lived, inside a single prediction endpoint. No accuracy claim is made anywhere in this system, and §9.4 explains what is returned instead of one.

9.2 The weighted error taxonomy

Four error classes, ordered from most to least severe. The recitation stores **only the class code** and a count; the weight lives in one module and is never written to a record.

Table 9.2 — Error classes

Code	Class	Weight	Definition
MEM_GAP	Memorisation gap انقطاع الحفظ	0.85	The student stopped and could not continue without prompting.
LNK_ERR	Linking error خطأ في الربط	0.50	The student moved to the wrong verse or the wrong position.
TAJ_ERR	Tajwīd error خطأ تجويد	0.25	A fault in articulation point, elongation or stopping.
SLF_CRT	Self-correction تصحيح ذاتي	0.05	The student erred and corrected themselves without prompting.

Why exactly four. The teacher tags errors on a phone during the circle. A long list is not used; four classes group the linguistic distinctions into practical severity bands.

Why the weight is not stored. Storing the weight with the error would make later recalibration incapable of repairing historical data, and the same error would carry different weights in different rows depending on when it was recorded. This is the removal of a transitive dependency, and it is the reason the taxonomy is served to the client by an endpoint (`GET /api/performance/error-types`) rather than duplicated in the front end.

9.2.1 Reading historical records

Sessions recorded before the taxonomy carry three free counters. Ignoring them would reset every metric to zero and discard the centre's whole history — worse than a declared approximation. They are therefore mapped:

Table 9.3 — Legacy counter mapping

Legacy counter	Mapped class	Reasoning
mistakes_count	MEM_GAP	An error that required the teacher to intervene.
tajweed_errors_count	TAJ_ERR	Direct correspondence.
hesitations_count	SLF_CRT	Hesitation without a settled error.

The ratio among the old weights was 4.0 : 2.0 : 1.5 and among the new is 0.85 : 0.25 : 0.05 — the ordering is preserved and the separation widened. *No claim is made that the two scales are equivalent.* Where a session carries `error_tags`, the taxonomy takes precedence absolutely and the legacy counters are ignored, so a session written by both paths is never counted twice.

One distinction carries real weight: **an empty tag map means “the teacher considered all four classes and found none”** — perfect precision — **while an absent field means the record predates tagging** and yields no precision value at all. Conflating the two would place every student with a pre-taxonomy history at the bottom of the precision board on a judgement that was never made.

9.3 The five performance metrics

All metrics are **per page**. Mishkaat records an āyah range; where a teacher enters an explicit page count it is used, otherwise pages are derived as $6,236 \text{ āyāt} \div 604 \text{ pages} \approx 10.32 \text{ āyāt per page}$. The derivation is a declared approximation, not a measurement.

9.3.1 Mastery

$$\text{Mastery} = 100 \times (1 - \min(1, D / d_{\max})), \quad D = \frac{\sum_{\text{sessions}} \sum_{\text{classes}} (\text{count} \times \text{weight})}{\sum_{\text{sessions}} \text{pages}}$$

$d_{\max} = 3.0$ weighted errors per page — the density at which mastery reaches zero. D is computed over the **totals**, not as a mean of per-session means: a half-page session with one error must not weigh as much as a ten-page session with one error.

9.3.2 Momentum

$$M_w = \alpha \cdot P_w + (1 - \alpha) \cdot M_{w+1}, \quad \alpha = 0.4$$

P_w is the number of *new-memorisation* pages in the week w weeks ago; the recursion runs from the oldest week forward to the present, so the most recent week carries the greatest weight. Review sessions are excluded. **Empty weeks enter the computation as genuine zeros, not as gaps to be skipped** — skipping them would reward interruption.

Smoothing is deliberate: a raw thirty-day total — which is what the system previously computed — jumps after a holiday week and then collapses, and any forecast built on it oscillates with it.

9.3.3 Precision

$$\text{Precision} = 100 \times (1 - \text{severe} / \text{total})$$

computed over **tagged sessions only**. *severe* counts errors whose weight ≥ 0.5 (that is, `MEM_GAP` and `LNK_ERR`); *total* counts all tagged errors. Returns 100 when there are tagged sessions with no errors, and `None` when nothing has been tagged.

Legacy counters are excluded here on principle: their mapping assumes every generic “mistake” is a memorisation gap. That assumption is acceptable for density, where magnitude matters and ordering is preserved, and unacceptable here, where the ratio between severity bands is the measurement.

9.3.4 Consistency

$$\text{Consistency} = 100 \times \text{attended} / \min(8, \text{sessions recorded})$$

over the most recent eight scheduled sessions. Present, late and excused all count as attendance; only an unexcused absence does not.

9.3.5 Review depth

$$\text{Review depth} = \min(200, 100 \times \text{review pages} / \text{new pages}) \text{ over the last 28 days}$$

Capped at 200%: a student who reviews twice what they memorise has reached the maximum this indicator measures. Without the cap, a student who stopped memorising entirely and kept reviewing would top the board on an unbounded ratio.

Table 9.4 — Metric summary

Metric	Range	Source data	What it answers
Mastery	0–100	Recitations	How secure is what this student recites?

Metric	Range	Source data	What it answers
Momentum	pages/week	New-memorisation recitations	How fast are they moving <i>now</i> ?
Precision	0–100	Tagged recitations	When they err, how serious is it?
Consistency	0–100	Attendance	Do they turn up?
Review depth	0–200	Recitations by type	Do they consolidate what they memorise?

NONE IS NOT ZERO

Every metric returns `None` when there is insufficient data. A new student with no sessions is not a student whose mastery is zero, and entering them on a leaderboard at zero both wrongs them and distorts the ranking. The interface renders a dash.

9.3.6 The legacy composite score

An older composite score survives, used by the honour roll and the rankings screen:

$$\text{Score} = \text{clamp}_{[0,100]}(0.4 \cdot H + 0.3 \cdot R + 0.2 \cdot A - 0.1 \cdot E)$$

H and R are the mean qualitative evaluation of new and review sessions on the scale excellent = 100, good = 80, acceptable = 60, needs improvement = 40 (defaulting to 80 where absent); A is mean attendance on present = 100, late = 80, excused = 60, absent = 0 (defaulting to 100); E is the count of recorded mistakes.

It is documented here because it is live, and because its history is instructive: the same formula was written out twice, in the honour roll and in the ranking, and the two copies had *already drifted* — one used a thirty-day window while the other used ninety under a variable named `thirty_days_ago`. It is now one function, so any future change applies to both screens. Its performance history is equally instructive: it previously issued two queries per student — two thousand round trips on every home-page load in a centre of a thousand students — and now issues two queries in total.

9.4 Completion forecasting

9.4.1 What is different

The previous forecast regressed accumulated āyāt against elapsed days: *quantity alone*. Two students with identical counts received identical dates even when one missed half the sessions and erred on every page. The contribution of the present model is that **recitation quality, not only quantity, affects the projected date**.

9.4.2 Features and model

Table 9.5 — Model features

Feature	Unit	Required sign	Rationale
Momentum	pages/week	+1	Memorising more does not slow completion.
Weighted error density	errors/page	−1	Erring more does not accelerate completion.
Attendance rate	0–1	+1	Attending more does not slow completion.

The target is the number of new-memorisation pages produced in a week. The model is ordinary least squares with an intercept, solved from the normal equations $(X^T X)\beta = X^T y$ by Gaussian elimination with partial pivoting — implemented in plain Python, because the server is already Python and no numerical dependency is warranted for a 4×4 system.

9.4.3 Training-row construction

The model is trained on the **whole centre's** history, not one student's: three features cannot be fitted from one student's record without overfitting. Each (student, week) pair is one row. Crucially:

```
for each student with ≥ 3 sessions:
    for each week w after the student's first session:
        features ← computed from sessions STRICTLY BEFORE the start of week w
                    (momentum, error density, attendance rate)
        target ← new-memorisation pages recorded DURING week w
        emit row only if ≥ 2 prior sessions exist
```

Computing the features over the whole period instead would leak the target into its own inputs and make R^2 an illusion. A minimum of **12 training rows** is required before a model is fitted at all.

9.4.4 Sign constraint

A regression on a small centre's data can return a *positive* coefficient for error density: students who read more pages accumulate more errors, so the model captures correlation rather than causation. Left alone, the system would tell a guardian that their child's errors are bringing their completion date forward — which destroys the credibility of the prediction however good R^2 is. Any coefficient contradicting its known direction is therefore **set to zero, not inverted**: the system refuses a claim that contradicts the domain, and it does not assert the opposite without evidence.

9.4.5 Prediction, guards and honesty

$$\text{weekly pages} = \beta_0 + \beta_1 \cdot \text{momentum} + \beta_2 \cdot \text{error density} + \beta_3 \cdot \text{attendance rate}$$

accepted only if $0.05 \leq \text{prediction} \leq \max(4 \times \max(\text{momentum}, 1), 20)$. Outside that band — or when no model could be fitted — the estimate falls back to the student's own momentum, and the response says so.

Every forecast returns the method used (`trained_model` or `momentum_fallback`), the number of training rows, the coefficient of determination and the model version (`mishkaat-lr-3feat-1.0`). **No confidence figure is returned, because none has been measured.** The source specification classifies its “80% of forecasts within ± 2 weeks” target as a design goal to be evaluated later, and this system does not present a design goal as a result.

Projection to dates is then arithmetic: with 604 pages over 30 parts (≈ 20.13 pages per Juz'), the remaining pages of the current Juz' and of the whole Qur'an are divided by the daily rate. A projection further away than sixty years returns no date at all rather than a figure that would merely mislead. The model is cached per centre for one hour and invalidated when a new recitation is recorded; the cache is per worker process and explicitly advisory.

9.5 The motivation layer

9.5.1 Four leaderboards, not one

The design premise, which came from the teachers themselves, is that **a single ranking demotivates precisely those who most need encouragement**: a student who appears at the bottom every week learns to ignore it. Mishkaat

therefore publishes four independent boards:

Table 9.6 — The four leaderboards

Board	Unit	Who leads it
Momentum الزخم	pages/week	Whoever memorises most.
Precision الدقة	%	Whoever makes the lightest errors.
Consistency الانتظام	%	Whoever never misses a session.
Review depth عمق المراجعة	%	Whoever consolidates what they have memorised.

The slow student who never misses leads consistency; the student who memorises little and reviews much leads review depth. All four derive from the *same session records* as the metrics — not from a parallel points system — so a student's position on any board can always be explained by pointing at their recitations and attendance. For a student or guardian the board is scoped to their own circle (§3.6, E1), which supplies peers to compare against without opening anyone's file.

9.5.2 Badges

Table 9.7 — Badge catalogue

Code	Name	Condition	XP
FIRST_STEP	First step أوّل الطريق	First recorded session	20
JUZ_1	First part الجزء الأوّل	20 pages memorised	100
JUZ_5	Five parts خمسة أجزاء	100 pages memorised	300
HALF_QURAN	Half the Qur'an نصف القرآن	302 pages memorised	800
KHATMA	Completion الختمة	604 pages memorised	2000
PRECISE	Precise reciter متقن التلاوة	Precision $\geq 90\%$	150
MASTERY_90	High mastery إتقان عالٍ	Mastery $\geq 90\%$	150
NEVER_ABSENT	Never absent لا يتغيّب	Consistency = 100% over the last eight sessions	200
REVIEWER	Consolidator مُنَبِّت الحفظ	Review depth $\geq 100\%$	200
STEADY	Steady ثابت الخطى	Momentum ≥ 2 pages/week	120

A badge is awarded once and is **never withdrawn**. A student who reached 90% precision and later fell back keeps the badge: it certifies an achievement that occurred, not a current state. Current state is what the metrics and the boards show. Un-earned badges are still shown to the student, as goals.

9.5.3 Challenges

Table 9.8 — Periodic challenges

Code	Period	Target	XP
WEEK_PAGES_3	Week	3 pages memorised	60
WEEK_NO_ABSENT	Week	2 attended sessions	50

Code	Period	Target	XP
MONTH_PAGES_12	Month	12 pages memorised	250
MONTH_REVIEW	Month	10 pages reviewed	200

Progress is computed on read from the same session records. There is no stored `progress` column, and therefore nothing that can drift out of step with the sessions.

9.5.4 The experience-point ledger

Points accrue at 10 per page recited and 5 per attended session, plus badge awards. Every award is a row in `xp_ledger`; rows are never updated or deleted. The unique index on `(student_id, reason, source_id)` is what makes awarding idempotent: re-submitting an attendance roll or reprocessing a session grants nothing twice, and the resulting duplicate-key error is *success*, not failure. Attendance uses `student:date` as its source key precisely so that correcting the roll does not pay twice. XP rates were chosen so that attendance alone cannot top a ranking.

9.6 Derived instruments

9.6.1 The journey map

For a student, “you have memorised 143 pages” does not say where they stand in the mushaf. The journey endpoint returns all thirty parts, each with a completion percentage and a state — `done`, `active` or `todo` — together with pages reviewed in the last thirty days, so that settled memorisation is visibly distinguished from memorisation being actively maintained.

9.6.2 The daily review portion

Each student's review portion is derived from their own memorised volume:

$cycle\ days = \lceil memorised\ parts / rate \rceil$, $day\ index = (today - epoch) \bmod cycle\ days$, $portion = parts[day\ index \cdot rate + 1 \dots]$

rate defaults to one part per day — the customary rule that a ḥāfiẓ completes a full revision cycle within a month — and is adjustable per student by the teacher. A student with fewer than half a part memorised receives no portion: none is invented.

Two properties follow from deriving the portion from the *date* rather than from stored state: nothing can be “forgotten to update”, and every person who opens the screen on the same day sees the same portion. The fixed epoch (Saturday 3 January 2026, the start of a study week) keeps the cycle stable across reloads. Paired with it is the question the teacher actually asks — *who has reviewed today and who is behind*.

9.6.3 The centre activity index

Supervisory ranking by head-count rewards a centre that enrolled a hundred students and then abandoned the system over a centre where twenty students work daily. Count measures size; movement measures use. The index therefore measures transactions in a window, normalised by centre size:

$score = \sum_k w_k \cdot \min(1, n_k / (e_k \cdot students \cdot days)) \cdot 100$

with weights $w = \{ \text{recitations } 0.45, \text{ attendance } 0.30, \text{ finance } 0.15, \text{ enrolment } 0.10 \}$ and per-student-per-day expectations $e = \{ 0.7, 0.9, 0.05, 0.02 \}$. Recitation is weighted highest because it is the centre's actual work; finance signals a live administration rather than teaching.

Scores map to four bands — intensive (≥ 75), regular (≥ 45), intermittent (≥ 15), weak (> 0) — plus “no activity”. The response carries counts, the per-signal breakdown, the last activity timestamp and days since; **it carries no student names and no student data**. Supervision learns that a centre is working without reading anyone's file.

9.7 Invariant design decisions

Table 9.9 — Decisions not to be reversed without a stated reason

#	Decision	Why
1	No derived value is stored.	A stored metric diverges from its source rows at the first correction or void, and the leaderboard then ceases to be explainable by reference to sessions — the property the entire motivation layer rests on.
2	Weights live in one place and are never stored with an error.	The weights are provisional and will be recalibrated; stored weights would make recalibration unable to repair history.
3	None means “no data”, not zero.	A new student's mastery is not zero, and listing them at zero both wrongs them and distorts the board.
4	Precision is never computed from legacy counters.	They carry no class, and their mapping assumes every error is a memorisation gap — sound for density, unsound for a severity ratio. The empty map is tagged (perfect); an absent field is not.
5	Sign constraints on model coefficients.	A system that tells a guardian that their child's errors accelerate completion loses all credibility, whatever its R^2 .
6	A badge is granted once and never withdrawn.	It records an event, not a state.
7	The unique key on the XP ledger is load-bearing.	Without it any repeat inflates a balance without limit.
8	Printed output uses fixed colour values, never design-system tokens.	Tokens invert in dark mode, so printing at night would produce pale text on white paper.

9.8 Calibration parameters

Table 9.10 — Environment-tunable analytical constants

Variable	Default	Controls
MASTERY_DMAX	3.0	The weighted error density at which mastery reaches zero.
MOMENTUM_ALPHA	0.4	Momentum smoothing; higher gives more weight to the most recent week.
HIGH_SEVERITY_THRESHOLD	0.5	The weight at or above which an error counts as severe in precision.
CONSISTENCY_WINDOW_SESSIONS	8	Sessions in the consistency window.
REVIEW_DEPTH_WINDOW_DAYS	28	Review-depth window.
XP_PER_PAGE / XP_PER_SESSION	10 / 5	Point rates.
VERSES_PER_PAGE	6236 / 604	Āyah-to-page derivation.
TOTAL_QURAN_PAGES / _VERSES	604 / 6236	Mushaf of reference.

OPERATIONAL NOTE

After enabling this layer on a centre with existing history, run `POST /api/performance/recompute` once. Badges are evaluated when a session is written and when a student's page is opened; a student whose page is never opened would otherwise never receive what they had already earned.

Security Engineering

10.1 Data sensitivity and threat model

Mishkaat stores the names, dates of birth, guardians' names, addresses and telephone numbers of **children under eighteen**, together with a continuous behavioural record of their attendance and academic performance, and the financial affairs of the centres that teach them. The security posture assumes the strictest applicable privacy regime, and security requirements are treated as functional requirements rather than as deployment hardening.

Table 10.1 — Threat model

#	Threat	Realistic scenario	Control
T1	Cross-tenant data access	A manager of centre A reads the students, fees or recitations of centre B by changing an identifier in a URL.	Scope resolution on every query; the ownership gate on every student identifier; 314 isolation assertions (§13).
T2	Intra-tenant over-reach	A circle teacher reads the file, guardian's phone number and metrics of a student from a colleague's circle.	The circle is a scope in its own right, not a detail inside the centre; <code>check_student_access</code> verifies circle membership, not merely centre membership.
T3	Identity spoofing through request fields	A client submits another teacher's identifier as the recorder of a recitation.	The acting identity is always taken from the verified token, never from the body.
T4	Credential stuffing / brute force	Automated password guessing, possibly rotating source addresses.	Dual-key logout: per IP and per account, stored in the database so it is correct across workers.
T5	Token theft via XSS	A compromised dependency or browser extension reads a long-lived token from browser storage.	The refresh token is in an <code>httpOnly</code> cookie unreadable by script; the access token lives only in memory and lasts one hour.
T6	Session persistence after revocation	A dismissed employee continues using an issued token; or a user changes their password after a theft and the thief remains logged in.	JTI revocation list, <code>user_version</code> invalidation, a <code>password_changed_at</code> check against the token's issue time, and rejection of disabled accounts at token verification.
T7	Database compromise	A stolen backup or direct database access exposes children's personal data.	Field-level Fernet encryption of date of birth, guardian name and address; backups encrypted with GPG before touching disk; the private key kept off the production host.
T8	Repudiation of financial acts	A disputed payment; a log row edited by someone with database access.	Hash-chained audit log with atomic sequencing and a never-purged class for money and permissions.
T9	Malicious file upload	An executable or an SVG carrying script uploaded as a “logo” and then served from the application's own origin.	Magic-byte sniffing rather than the declared type; an allow-list that excludes SVG; hard size caps enforced during chunked reading.

#	Threat	Realistic scenario	Control
T10	Registration and reset abuse	Flooding public registration; enumerating accounts through password reset.	Per-IP windows counted on success only; identical responses whether or not an account exists; hashed, short-lived, attempt-capped codes.
T11	Token leakage through URLs	An access token passed as a query parameter to open an event stream is written into proxy logs, browser history and <code>Referer</code> headers.	Single-use stream tickets valid sixty seconds, obtained with a normal Authorization header.
T12	Information disclosure through errors	A stack trace returned to the client reveals paths, versions and query structure.	A generic 500 with a request identifier; the traceback is logged server-side under the same identifier.

10.2 Authentication and session lifecycle

Table 10.2 — Token design

Property	Access token	Refresh token	Purpose
Lifetime	60 minutes	7 days	A stolen access token expires quickly; the refresh token is not reachable by script.
Transport	<code>Authorization: Bearer</code>	<code>httpOnly</code> cookie, path <code>/api/auth</code> , <code>SameSite=lax</code> , <code>Secure</code> in production	The path restriction means the cookie is not sent with ordinary API calls, narrowing exposure; <code>SameSite=lax</code> protects the refresh endpoint from cross-site forgery.
Client storage	A module-scoped variable	Never visible to JavaScript	Nothing in <code>localStorage</code> : one XSS flaw previously sufficed to steal a seven-day token that could be rotated quietly, keeping an attacker inside the account even after the owner changed their password elsewhere.
Claims	<code>sub</code> , <code>uv</code> (user version), <code>exp</code> , <code>iat</code> , <code>jti</code> , <code>type</code>		Type is verified so that a refresh token cannot be presented as an access token.
Rotation	Issued on refresh	Rotated; the previous one is revoked	Replay of a used refresh token fails.

10.2.1 The five verification checks

Every authenticated request performs all five, in order:

1. Signature and expiry verification (HS256).
2. Token type must be `access`.
3. The JTI must not appear in `revoked_tokens`.
4. The account must exist and be active — otherwise 403, so that deactivation takes effect on already-issued tokens.
5. The token's `uv` must equal the account's current `user_version`, and its `iat` must not precede `password_changed_at`.

Checks 4 and 5 are what make a password change a genuine global logout rather than a local one.

10.2.2 Client-side session handling

Session persistence across a reload relies on the cookie alone: at boot the client posts an empty body to `/api/auth/refresh`. A single-flight guard ensures concurrent 401s trigger one shared refresh — because the server rotates and revokes, two parallel refreshes would invalidate each other and eject the user. A 403 is never treated as expiry; it is a permission shortfall, and logging the user out on one was a defect that has been removed.

10.3 Authorisation architecture

Authorisation is applied in four layers, and no layer is sufficient alone.

Table 10.3 — Authorisation layers

L	Layer	Mechanism	What it stops
1	Role gate	An explicit role allow-list at the top of a handler; <code>teacher_only()</code> for attesting actions.	A guardian calling a finance endpoint; a manager recording a recitation.
2	Scope filter	<code>scope.student_query</code> , <code>visible_student_ids</code> , <code>visible_halaqah_ids</code> applied to the query itself.	A list endpoint returning rows outside the caller's centre or circle.
3	Ownership gate	<code>check_student_access(student_id)</code> on every endpoint that takes one.	Direct object reference: guessing or enumerating another student's identifier (BOLA / IDOR).
4	Field redaction	<code>redact_teacher</code> and equivalent response shaping.	Salary and marital status reaching a student, guardian or colleague inside an otherwise legitimate response.

THREE RULES THAT ARE NOT TO BE BROKEN

1. `check_student_access` is the *only* gate for any endpoint taking a `student_id`. It is never bypassed with a hand-written `center_id` comparison — one hole in it previously produced ten separate leaks.
2. An empty scope returns `None`, never `{}`. An empty MongoDB filter means *everything*, the exact inverse of the intent, and it reads as correct code.
3. A path or query parameter narrows within the scope or returns 403. It never widens it.

10.3.1 Ownership resolution by role

Table 10.4 — How `check_student_access` decides

Caller role	Condition for access
admin / super_admin	Granted.
center_manager	The student's <code>center_id</code> equals the caller's.
teacher	Same centre and the student's <code>halaqah_id</code> is among the caller's active circles. Centre alone is insufficient — that was the defect that exposed every student in the centre to every teacher.
student	<code>students.user_id</code> equals the caller's account ID, or the phone numbers match. Never by name: names are self-editable, so name matching would let a student rename themselves to a classmate and read that classmate's file, grades and fees.

Caller role	Condition for access
parent	<code>students.parent_user_id</code> equals the caller's account ID, or the phone numbers match (legacy accounts).

10.4 Credential policy and recovery

- **Hashing:** bcrypt at cost 12, tunable upward by environment.
- **Complexity:** at least eight characters, including a letter and a digit, enforced uniformly on every path that sets a password — self-service change, teacher account creation, manager account creation, public registration, student and guardian issuance, and administrative reset.
- **Issued passwords** are generated from an alphabet that excludes visually confusable characters (`0/0` , `l/1/I`), because they are written on paper and handed over. They are shown exactly once and stored only as a hash. A forgotten password is *replaced*, never recovered, and the replacement bumps `user_version` , closing open sessions.

10.4.1 The reset flow

1. A reset request is rate limited on **both** the requesting IP and the target account: five per hour each.
2. A code is generated and stored as `HMAC-SHA256(secret, "username:code")` — never in plaintext, and bound to the username so that a code for one account cannot be used on another. Any plaintext `code` field left by earlier versions is actively removed on write.
3. The record expires after ten minutes by TTL index, and five wrong attempts invalidate it.
4. Responses are identical whether or not the account exists, so the endpoint cannot enumerate accounts.
5. A successful reset bumps `user_version` , clears the account lockout, and writes a permanent audit entry.

10.5 Rate limiting and lockout

Table 10.5 — Limits

Action	Key	Limit	Reasoning
Login	IP address	5 / 15 min	A single key is insufficient. IP alone is evaded by address rotation, and behind a proxy it locks out <i>every</i> user after five wrong passwords from one person. Account alone allows unlimited spraying across many accounts. Both keys are checked and both are incremented.
Login	Account	10 / 15 min	
Public centre registration	IP address	3 / 60 min	Counted on success only . Previously three validation failures — a missing field, a short password, a taken username — consumed the whole hour and locked out someone who had not registered once. A limit exists to deter flooding, not to punish typing.
Password reset request	IP and account	5 / hour each	Prevents reset flooding and account harassment.
Reset code verification	Reset record	5 attempts	Caps brute forcing of a short code.

All limiter state lives in MongoDB with TTL indexes, so it is correct across worker processes and cannot grow without bound. The earlier in-process dictionary was both incorrect with multiple workers and an unbounded memory leak. Client IP resolution honours `X-Forwarded-For` **only** when `TRUST_PROXY` is set, because trusting the header unconditionally lets any client forge it.

10.6 Personal data protection at rest

Three student fields are encrypted with Fernet (AES-128-CBC with HMAC-SHA256) before being written: `date_of_birth`, `guardian_address` and `parent_name`. Decryption tolerates legacy plaintext, so enabling encryption on an existing database does not break reads.

WHY PHONE NUMBERS ARE DELIBERATELY NOT ENCRYPTED

Phone numbers back the ownership matching in `check_student_access` and the guardian lookup used to send an absence alert. An encrypted field cannot be queried by equality, so encrypting them would break authorisation itself. This is a documented, bounded trade-off rather than an omission.

Two operational consequences follow, and both are stated in the deployment documentation: the key must be in a secret manager rather than only in an `.env` file, and **losing it destroys those fields permanently** — `decrypt_pii` tolerates plaintext, but not a wrong key. Key rotation requires a one-off re-encryption pass, which must be planned before it is needed.

10.7 Tamper-evident audit logging

Thirty-five action types are recorded (Appendix B). Each entry is linked to its predecessor:

$$h_n = \text{SHA-256}(\text{seq} \parallel \text{actor_id} \parallel \text{center_id} \parallel \text{action} \parallel \text{canonical}(\text{payload}) \parallel \text{timestamp}_{\text{ISO}} \parallel h_{n-1})$$

with `canonical(payload)` a JSON serialisation with sorted keys, and $h_0 = 64$ zeros. Altering any field of any entry breaks every subsequent hash.

Three details make the chain sound in practice rather than only in principle:

- **Atomic ordering.** The sequence number comes from an atomic counter, and the chain is built on that order. Reading the predecessor by sorting on timestamp meant two simultaneous writes obtained the *same* predecessor hash, splitting the chain silently and causing later verification to report tampering with no discoverable cause.
- **Millisecond truncation.** Timestamps are truncated to milliseconds before hashing, because MongoDB stores only millisecond precision. Hashing microseconds would make every recomputed hash differ from the stored one and report tampering in a perfectly sound record.
- **Explicit gaps.** When a predecessor cannot be resolved — after a short retry window that allows for concurrent writes from another worker — the entry is written with `chain_gap: true`. The record states the break rather than pretending the chain is intact.

`GET /api/audit-logs/verify` recomputes every hash from the stored fields and reports mismatches and marked gaps. Financial and credential actions carry no `purge_at` and are never removed; all other entries expire after 730 days.

10.8 File upload security

Table 10.6 — Upload controls

Control	Implementation and reason
Content sniffing	The type is determined from the file's magic bytes (PNG signature, JPEG <code>FF D8 FF</code> , RIFF/WEBP, <code>%PDF-</code>), never from the declared <code>content_type</code> , which the sender writes and can forge. The sniffed type is what is stored and what is served.
No SVG	SVG is a document that can carry script; serving one from the application's origin opens an injection path in a page the user trusts. Logos accept PNG, JPEG and WEBP only.
Size caps during read	512 KB for a logo, 5 MB for a document, enforced while reading in 64 KB chunks. Reading the whole file and then measuring it lets one sender load the server with whatever they please before being refused. Nginx caps the body independently at a lower level.
Authorisation on the message	An attachment download is authorised by the caller's right to the <i>message</i> , not by possession of the file identifier — otherwise anyone who guessed an identifier would read another centre's document.
Storage in GridFS	Files live in the database with metadata recording kind, centre, owner, sniffed type, size and upload time. A directory on a container's disk would lose the logo at the first redeployment; the database backup captures the files with the records that reference them.

10.9 Transport and HTTP hardening

Table 10.7 — Response headers set by the application

Header	Value and effect
Content-Security-Policy	<code>default-src 'none'; frame-ancestors 'none'; base-uri 'none'</code> — the API serves JSON only; no script, no frame, no base rewriting.
X-Content-Type-Options	<code>nosniff</code> .
X-Frame-Options	<code>DENY</code> — no clickjacking surface.
Referrer-Policy	<code>no-referrer</code> .
Cross-Origin-Opener-Policy	<code>same-origin</code> .
Permissions-Policy	<code>geolocation=(), microphone=(), camera=()</code> .
Strict-Transport-Security	<code>max-age=31536000; includeSubDomains</code> , in production only.
X-Request-ID	Echoed or generated; the correlation key between a client-visible 500 and the server-side traceback.

CORS allows an explicit list of origins with credentials enabled; a wildcard origin **aborts the boot** in production, because a wildcard with credentials is a contradiction. The origin list is whitespace-trimmed — a correctly written `"a, b"` configuration previously kept the space attached to the origin and silently blocked the real front end. Nginx adds an independent header set, an HTTP → HTTPS redirect, a `client_max_body_size` cap and an optional edge rate-limit zone on the login path as defence in depth.

10.10 Fail-fast production guards

The application **refuses to start** in production if any of the following holds. Each condition was chosen because the alternative is a silently insecure deployment that appears to work:

Table 10.8 — Boot-time refusals

Condition	Why the boot is aborted
<code>SECRET_KEY</code> is still the built-in default	Every token in the deployment would be forgeable by anyone who has read the source.
<code>PII_ENCRYPTION_KEY</code> is unset	Children's personal data would be written in plaintext. In development an ephemeral key is generated with a warning that the data will not survive a restart.
<code>ALLOWED_ORIGINS</code> contains <code>*</code>	A wildcard origin with credentials enabled exposes every authenticated endpoint to every site.
<code>INITIAL_ADMIN_PASSWORD</code> is unset while bootstrapping the administrator	Otherwise the most powerful account would be created with a predictable password.
<code>INITIAL_SUPER_ADMIN_PASSWORD</code> is unset while bootstrapping the super-administrator	This account sees and suspends every centre. It previously fell back to a fixed string whenever the administrator password was unset — including in production, because the earlier guard checked only the administrator account and was skipped entirely if that account already existed. The two accounts now also take <i>separate</i> passwords, so that a leak of one is not a leak of both.
Demo seeding	<code>SEED_DEMO_DATA</code> defaults to false; demo accounts with published passwords are opt-in and are verified absent during onboarding.

10.11 Residual risks

Table 10.9 — Known residual risks and their status

Risk	Current mitigation	Residual exposure
Phone numbers stored in plaintext	Required for ownership matching and guardian lookup	A database compromise exposes contact numbers. Accepted and documented; encrypting them would break authorisation.
Reset codes have no delivery channel	Retrieved by an administrator through an authenticated endpoint	Recovery depends on an administrator being reachable. Email or SMS delivery is future work.
PII key rotation	Documented as requiring a re-encryption pass	No tooling ships for the pass itself; it must be written before a rotation is attempted.
No server-side output rate limiting on read endpoints	Scope limits what can be read; CSV export is audited	An authorised insider can extract their own scope in bulk. Mitigated by audit, not prevented.
Naive UTC timestamps	A single time source; all comparisons consistent	Deployments spanning timezones must interpret timestamps as UTC. Migration is a data migration, recorded as future work.
The unique attendance index may fail to build on a legacy database	An explicit warning is logged and the service continues	Until duplicates are cleaned and the service restarted, daily uniqueness rests on the application-level upsert alone.

Business Workflows

11.1 Centre onboarding and approval

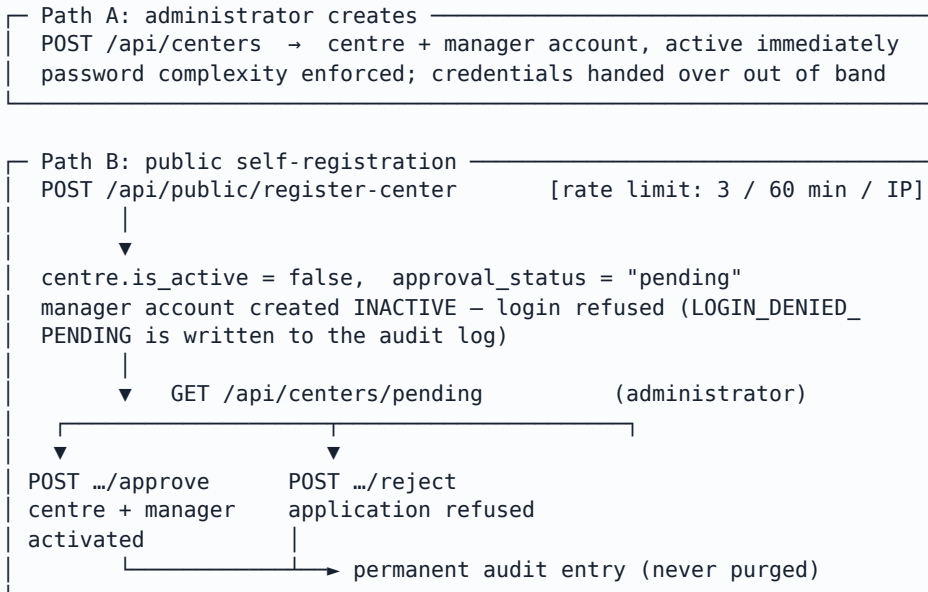


Figure 11.1 — Centre onboarding. Path B previously self-activated, which allowed anyone on the internet to create an operating tenant.

11.2 Enrolment and account issuance

Enrolment and account issuance are deliberately separate steps. The student record is created first; accounts are issued afterwards, by the manager, and bound to the record by identifier.

1. **Enrol.** `POST /api/students` with guardian details, memorisation plan and circle. Date of birth, guardian name and address are encrypted before writing. The circle's occupancy count is updated.
2. **Issue accounts.** `POST /api/students/{id}/accounts` creates an account for the student, the guardian, or both. Each password is generated from an alphabet without confusable characters, returned **exactly once** so the manager can hand it over, and stored only as a hash. The account identifiers are written to `students.user_id` and `students.parent_user_id`, so the binding survives a change of phone number. The issuance is audited.
3. **Replace, never recover.** `POST .../accounts/reset` issues a new password for one holder. There is no retrieval path — and the reset bumps `user_version`, closing that holder's open sessions on every device.

11.3 The daily recitation cycle

```
teacher opens the recitation screen
|
├─ students listed = scope(teacher) = his circles only
▼
enters sūrah · āyah range · type (new | review) · pages? · error tags · evaluation
|
▼ POST /api/recitations

1. teacher_only()                manager/admin → 403
2. check_student_access()         not his circle → 403
3. teacher record resolved FROM THE TOKEN (not from the body)
4. duplicate check on (student, type, date_str) → 409 if present
5. insert with date, date_str and center_id
6. side effects, none of which may fail the write:
   • XP: 10 × pages, keyed by the recitation id (idempotent)
   • badge re-evaluation against the fresh snapshot
   • invalidate the centre's cached forecast model
```

Figure 11.2 — Recording a recitation.

Step 6 is wrapped so that a failure in the motivation layer is logged and ignored: **the session is the real record and the points are a layer above it**. Binding their fate together would mean a teacher loses their work because of a defect in gamification.

Corrections are first-class. The recording teacher may edit or soft-delete their own session; centre administration may correct any session in the centre. A soft-deleted session leaves every metric, score and forecast, but remains in the database for audit.

11.4 Attendance and absence notification

```
teacher submits the roll for a circle and a date
|
▼ POST /api/attendance (records[], date)
per record: student exists · student belongs to this circle · circle belongs
             to this centre → otherwise 403
|
▼ upsert on key (student_id, date_str)
|   ┌ inserted → "created"
|   └ matched → "updated"      ← what a teacher correcting a mark expects
|   (a DuplicateKeyError from a simultaneous double-click is caught and
|    retried as an update, so the correction is not lost to a 500)
▼
status = absent ?
├─ no → XP: 5 points, source key "student:date" (idempotent across re-submission)
└─ yes → guardian resolved by parent_phone → notification PERSISTED
        ("absentee_alert" with the student, the date and the circle)
        and only then streamed
```

Figure 11.3 — Roll-call and absence alerting.

Two defects shaped this design. First, the roll was inserted rather than upserted, so re-submitting it — a double-click, or a teacher correcting an absence to a presence — created two contradictory rows for the same student on the same day, and the absence stayed in the attendance rate forever. Second, the alert was streamed only, so it reached a guardian who happened to have the application open at the moment the roll was taken. Persisting first and streaming from the database fixed both the reachability problem and a multi-worker problem: with four workers, an in-process fan-out reached only the quarter of users connected to the worker that wrote the event.

11.5 The competition lifecycle

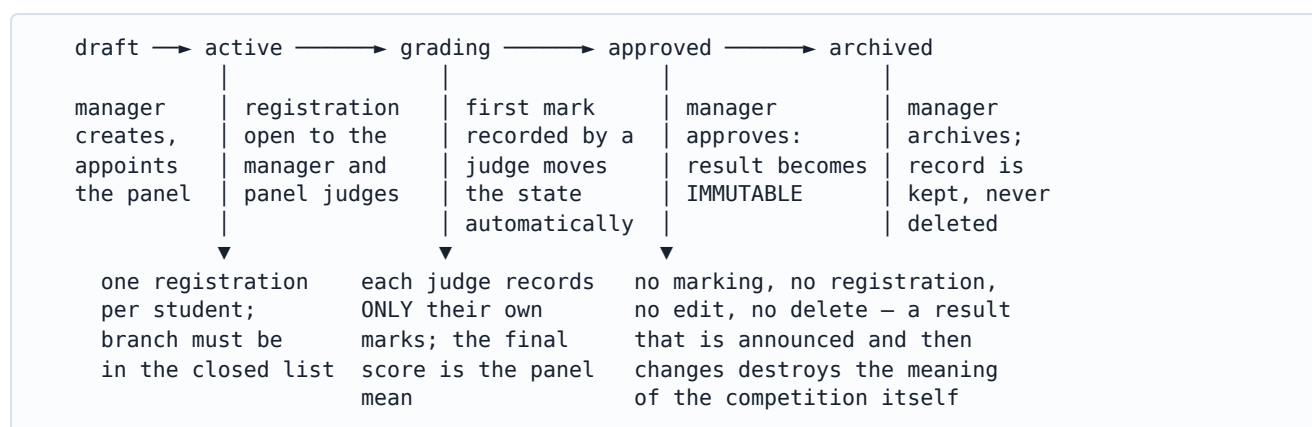


Figure 11.4 — Competition state machine.

Table 11.1 — Competition marking scheme

Criterion	Maximum	Notes
Memorisation (الحفظ)	70	Each judge's total is the sum of the three. The contestant's score is the arithmetic mean of the judges' totals, and the number of judges who have marked is reported alongside it. Re-marking replaces only that judge's entry.
Tajwīd (التجويد)	20	
Voice (الصوت)	10	

Three rules are absolute here. **Branches are ranked separately**, each from position one, so a memoriser of Juz' 'Amma is never placed at the bottom of a list headed by memorisers of the whole Qur'an; and the branch list is closed, because free text would make results impossible to aggregate or archive across centres. **Every mark is attributed**: previously a single score sat on the contestant, so the last judge to open the screen silently erased his colleague's marking and no one could tell who had given what. **Administration does not mark** — the manager appoints the panel and approves the outcome; recording a mark is testimony from someone who listened.

Finally, judging is a task with a term, not a standing permission: the competitions menu item appears for a teacher only while a competition they judge is in progress, and disappears after approval. The hiding is in the interface; the prohibition is on the server.

11.6 Certificate issuance

Table 11.2 — Milestones

Code	Milestone	Parts	Pages required
juz5	Five parts memorised	5	≈ 100.7
juz10	Ten parts memorised	10	≈ 201.3
half	Half the Qur'an	15	≈ 302.0
khatm	The whole Qur'an	30	604

Issuance is restricted to the centre manager, and three conditions apply:

1. **It is a recorded act, not a print button.** Each certificate carries a serial (MK-YYYY-NNNNN), allocated from an atomic counter per centre per year), the issuer's identity and the issue time. A certificate that is later questioned can be traced; and it cannot be printed twice without anyone knowing.

2. **It is never issued twice.** Partial unique indexes enforce one milestone certificate per student per milestone and one competition certificate per contestant. A code check before a write succeeds twice when the button is pressed twice simultaneously.
3. **It is never issued early.** A milestone certificate is refused with 409 until the pages actually recited reach the threshold (with a half-page tolerance), and the refusal states how far the student has reached. A competition certificate is refused until the results are approved — a certificate bearing a score that can still change is a document that cannot be trusted.

Revocation exists, for a wrong name or a wrong milestone, and is audited exactly as issuance is. Students and guardians see and print their own certificates; only the manager issues them.

11.7 Financial operations and the void policy

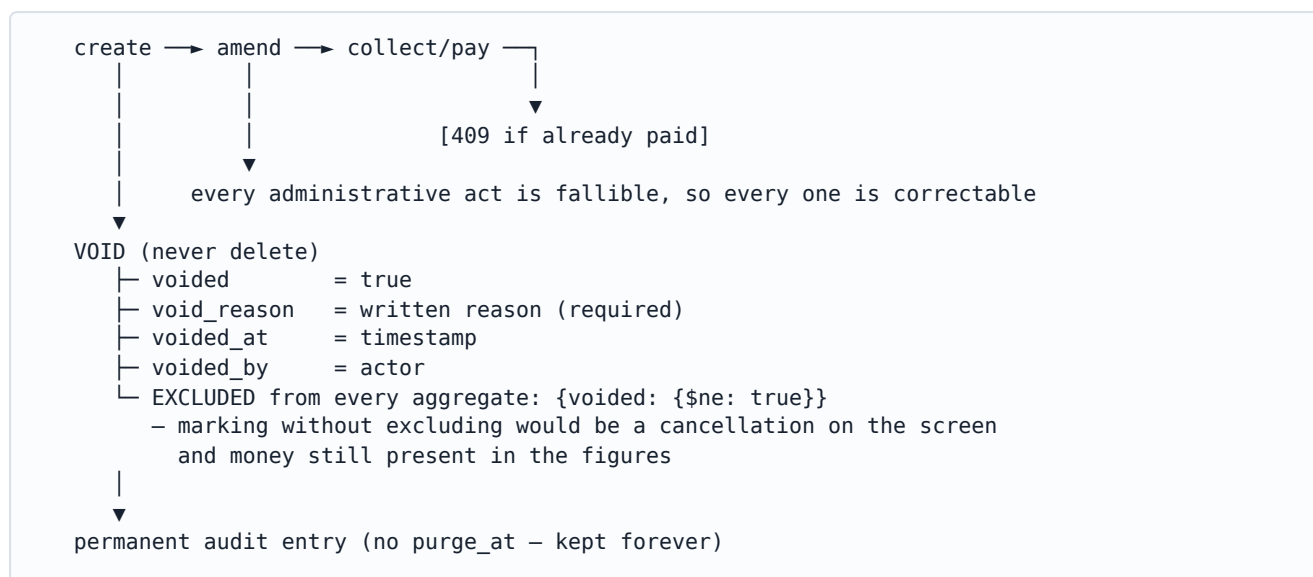


Figure 11.5 — The lifecycle of a financial record.

The void filter is written `{voided: {$ne: true}}` rather than `{voided: false}` because historical rows do not carry the field at all, and an equality test would silently drop every one of them from every total.

11.8 Messaging

The chain of §3.5 is enforced on the server and published by `GET /api/messages/audiences`; the client never contains its own copy of the rule. Attachments are uploaded first — returning a descriptor — and then referenced when the message is sent, which is simpler than mixing a payload with a file in one request and allows the sender to change their mind before sending. A sent message can be corrected (the edit is visible as such) or withdrawn.

The design was driven by two concrete failures: a system administrator could not send a single message, because the handler required a centre and the administrator account has none, while the interface displayed the full set of broadcast controls; and a centre manager had no route to system administration at all.

11.9 Idempotency guards

Table 11.3 — Operations that happen once and are refused twice

Operation	Uniqueness key	Enforced by	What a duplicate would cost
-----------	----------------	-------------	-----------------------------

Operation	Uniqueness key	Enforced by	What a duplicate would cost
Recitation	student + type + <code>date_str</code>	409 guard	Every metric, the XP balance and the forecast would be inflated — the student's picture would be wrong, not merely the list.
Attendance	student + <code>date_str</code>	Unique index + upsert	Two contradictory rows for one day, with the absence permanently embedded in the attendance rate.
Salary	teacher + month + centre (non-voided)	409 guard	Money leaving twice, with a second expense line and a second audit entry.
Monthly fee	student + type + due month	409 guard	A family billed twice.
Fee collection	fee already paid	409 guard	A double receipt.
Milestone certificate	student + milestone	Unique partial index	Two certificates with different serials for the same achievement.
Competition certificate	competition + student	Unique partial index	As above.
XP award	student + reason + source	Unique index	An unbounded, inflatable balance.
Badge	student + badge code	Unique index	Repeated XP for the same achievement.
Competition registration	competition + student	400 guard	One student ranked twice in a branch.

Every 409 names the existing record and directs the user to edit it rather than add another — a refusal that does not say what to do instead simply produces a workaround.

User Interface and Interaction Design

12.1 Design language

Table 12.1 — Interface principles

Principle	Application
Arabic, right-to-left	The sole interface language. Layout, iconography, number formatting and print styles are RTL-first; the PWA manifest declares <code>"dir": "rtl"</code> and <code>"lang": "ar"</code> . A partial translation layer was deliberately removed (§2.3.2).
A calm palette	Green and gold — the conventional register for this domain — carried through a token layer rather than hard-coded values, with a matching dark theme.
Large, few controls	The primary users include older teachers who are not computer users. Screens have one dominant action; targets are large; forms are short.
Explained emptiness	An empty list states its cause: “no circle has been assigned to you yet” rather than a blank page. Empty states are a component (<code>EmptyState</code>), not an afterthought.
Single source of truth for theme	Theme state previously lived in two places — the top-bar toggle wrote to the document root and local storage, while the settings page kept its own state and displayed “dark mode (coming soon)”, disabled, for a feature that had worked for months. The settings screen showed “light” selected while the application was dark. It is now one read, one write and one event.

12.2 Route map and client-side guards

Client guards mirror the server's permissions exactly. This is presentation, not security — the server remains the authority — but a route that renders and then fails every call shows the user a broken screen instead of a clear message. Before guards existed, any user could type `/finance` or `/audit-logs` and receive a page whose every request returned 403; the “forbidden” page existed in the code base and nothing ever invoked it.

Table 12.2 — Routes and their guards

Route	Permitted roles	Screen
<code>/ · /sponsorships · /register · /login</code>	public	Landing page with a live “best centres” listing, a sponsorship prototype, centre self-registration, login.
<code>/dashboard</code>	all	Dispatches to one of six role dashboards.
<code>/centers</code>	admin, super_admin, center_manager	Centre management and the approval queue.
<code>/students · /halaqat</code>	staff	Enrolment and circle management (scoped).
<code>/teachers</code>	admin, super_admin, center_manager	Teachers, transfers and evaluations.
<code>/recitations</code>	all (scoped)	Session logging with the error tagger; read-only for students and guardians.
<code>/attendance</code>	admin, center_manager, teacher	Roll-call and corrections.
<code>/finance · /reports</code>	admin, center_manager	Fees, salaries, expenses, summaries and printable reports.
<code>/rankings · /leaderboards</code>	all (scoped)	Composite ranking; the four independent boards.

Route	Permitted roles	Screen
/analytics/:id · /performance/:id	all (scoped)	Student analytics; the metrics, forecast, journey, badges and challenges.
/review-plans · /academic-schedules	all (scoped)	Daily review portion and timetable. The timetable guard was widened to every role after it emerged that the sidebar offered students a page the guard then refused — while the server permitted it all along.
/competitions	all (scoped)	Competitions, panels, marking and results.
/certificates	admin, super_admin, center_manager, student, parent	Issue, view and print. Not the teacher's concern — removed from both the route guard and the sidebar.
/bulk-messages · /notifications	all	Messaging within the chain; stored notifications.
/audit-logs	admin, super_admin	Audit listing and chain verification.
/profile · /settings	all	Profile, password change, theme.
/403 · /404 · *	all	Explicit error pages rather than a blank screen.

The sidebar groups navigation into three sections — *daily work* (العمل اليومي), *administration* (الإدارة) and *system* (النظام) — and each item declares the roles that see it. Two items are conditional at runtime rather than by role alone: competitions appear for a teacher according to their panel membership, and the notification badge reflects unread count.

12.3 Role dashboards

Table 12.3 — What each dashboard leads with

Dashboard	Content
Super administrator	Platform-wide totals, centre listing with activation controls, and the centre activity index.
Administrator	Pending centre applications, system status, audit highlights, per-centre activity.
Centre manager	Student, teacher and circle counts, attendance rate, outstanding fees, honour roll, circle performance overview.
Teacher	His circles, today's roll, today's recitation list, students behind on review, and his circles' standing in the aggregate ranking.
Student	Own metrics, journey map, badges, challenge progress, position on the four boards within the circle, certificates.
Guardian	Each child's progress and attendance, absence notifications, fee status, certificates.

12.4 Component inventory

Table 12.4 — Notable components

Component	Purpose
ErrorTagger	The four-class error tagger used during a circle. Its labels, descriptions and weights come from the API, never hard-coded — so a recalibration reaches the interface without a release.
StudentReportSheet	A printable single-student history for the guardian.

Component	Purpose
CertificateSheet	+ <code>Ornaments</code> : the printable certificate, with fixed colour values rather than theme tokens (§12.6).
HalaqatRanking	+ <code>HalaqahDetailModal</code> : the aggregate circle ranking and the manager's per-circle detail view.
StudentAccountsModal	Account issuance, showing a generated password exactly once.
TeacherEvaluationModal	The five-criterion evaluation with written strengths and weaknesses.
CenterActivityPanel	The supervisory activity index with its per-signal breakdown.
PermissionBadge	Shows the viewer what their role permits on the current screen, so that a missing button is explained rather than mysterious.
NumberInput	Numeric entry with correct Arabic digit grouping — a recurring source of confusion before it existed.

12.5 Progressive Web App delivery

Table 12.5 — Installation by platform

Platform	Method	Result
Android (Chrome)	Menu ⋮ → “Install app”	“المشكاة” under the icon, standalone window.
iOS / iPadOS (Safari)	Share → “Add to Home Screen”	As above.
Desktop (Chrome / Edge)	Install icon in the address bar	A separate application window.

HTTPS is mandatory: browsers offer installation only over a secure connection (or `localhost`). The manifest declares standalone display, an RTL Arabic locale, a dark theme colour and shortcuts to the most frequent tasks. All icons are generated from a single vector source by a build script.

TWO OPERATIONAL RULES FOR THE PWA

(1) The displayed name comes from `short_name` in the manifest on Android and desktop, and from `apple-mobile-web-app-title` in the HTML on iOS. Changing one without the other gives a different name on different devices. iOS shows roughly twelve characters and then truncates, which is why the iOS title is the short form alone.

(2) The service worker caches the application shell under a versioned cache name. **Changing any icon requires bumping that version**, or installed devices keep showing the old icon indefinitely. The shell is cached asset by asset rather than atomically, so one missing file cannot block the entire update.

12.6 Printing and document output

Certificates, student report sheets and centre reports are produced through the browser's print pipeline from print-styled components, with a landscape helper for wide tables. One rule governs all of them: **printed output uses fixed colour values, never design-system tokens**. Tokens invert under the dark theme, so a user printing in the evening would receive pale text on white paper. CSV exports are written as UTF-8 with a byte-order mark, because Excel otherwise opens them in the wrong encoding and renders Arabic names as symbols.

Verification and Quality Assurance

13.1 Testing philosophy

The test strategy is behavioural rather than unit-oriented, and it rests on three commitments that differ from conventional practice.

COMMITMENT 1 — TEST FOR LEAKAGE, NOT FOR SUCCESS

In an isolation suite, **a 200 response is not a failure; a 200 response carrying another party's data is**. Each assertion asks whether the payload contains something that does not belong to the caller. A suite that only checks status codes would have passed against every one of the seventeen leaks that motivated the scope module.

COMMITMENT 2 — EVERY ASSERTION IS VERIFIED TO FAIL

Each test was run against the defect reverted, to confirm that it actually fails. An assertion that passes both with and without the fix protects nothing. This is stated explicitly in the headers of the regression and performance suites.

COMMITMENT 3 — A NEW ENDPOINT TOUCHING STUDENT OR CIRCLE DATA IS ADDED TO THE ISOLATION SUITE IN THE SAME COMMIT

This is the rule that keeps the boundary from eroding endpoint by endpoint, which is how it eroded before.

Suites run against a live MongoDB. Each creates a temporary database, exercises the application layer directly (the startup routine, routers and scope helpers are imported and called), and drops the database at the end, so production data is never touched.

13.2 Test suite inventory

Table 13.1 — Test suites

Suite	Lines	Assertion sites	What it guards
data_isolation_test.py	346	39	The four-level scope model on a fixture centre with two circles, each with its own teacher and two students. Documented at 63 assertions including those executed inside loops. Verifies that a teacher cannot read a student of another circle, cannot record for one, and that a manager reads everything but records nothing.
competitions_accounts_test.py	441	80	Documented at 79 assertions . Branch separation; per-judge marking and the panel mean; that a second judge's marking does not erase the first; immutability after approval; invisibility of a competition to a non-judge and its disappearance after approval; that administration approves but does not mark; that an evaluation is readable by its subject and administration only; and that student and guardian accounts see only what belongs to them.

Suite	Lines	Assertion sites	What it guards
audit_2026_09_03_test.py	402	69	A regression suite pinning each fix of the 2026-09-03 review: dual-key rate limiting, hashed reset codes, atomic audit sequencing, the removal of name-based ownership matching, salary redaction, CSV encoding, ID validation, CORS origin parsing and the security headers.
owner_batch_2026_09_07_test.py	415	66	The closed messaging chain; certificate issuance by the manager only, after approval, after the milestone, and never twice; the once-a-day and once-a-month guards; correctability of every administrative act; a student seeing circle peers on the leaderboard and full competition results <i>without</i> being able to open anyone's file; a teacher seeing the aggregate circle ranking but not another circle's detail; and the activity index measuring movement rather than size.
performance_layer_test.py	297	29	Documented at 31 assertions . The metrics, the taxonomy, badge idempotence and the forecast. Its decisive case is <code>forecast_reacts_to_quality</code> : two students identical in quantity and attendance and differing only in errors — <i>if their forecasts are equal, quality is not entering the computation</i> , and the central contribution of the layer is absent.
center_approval_test.py	140	17	The full approval cycle. Motivated by a complete dead end: the registrant could not log in and saw “account disabled” — a rejection message, not a waiting one — while the administrator's centre list was empty (it scoped a global role to a centre it did not have) and no approval control existed in the interface at all, though both endpoints existed on the server.
register_center_test.py	128	14	Public registration. Motivated by the registration form lacking an email field the server required, so every attempt returned 400 — and then 429, because failures consumed the hourly quota. The decisive assertion is that <i>errors do not consume the quota</i> : five failures followed by one valid attempt must succeed.
Total	2,169	314	

“Assertion sites” counts calls to the suites' assertion helper in the source. Where a suite's own header or the project documentation states a different figure, that figure counts assertions *executed*, including those inside loops; both are given above.

A separate historical artefact, `backend_test_results.json`, records an end-to-end API run of 2 April 2026 with 34 of 34 checks passing — health, logins for five roles, and the principal CRUD paths. It predates the scope model and the analytical layer and is retained as a record, not as current coverage.

13.3 Representative invariants

Table 13.2 — A selection of invariants under test

Invariant	Why it is the right thing to assert
A teacher requesting a student from another circle receives 403 — <i>not</i> a 200 with an empty body	An empty 200 would indicate the scope filter is being applied where an authorisation decision was required, which fails differently on the next endpoint.

Invariant	Why it is the right thing to assert
A student who sees a peer's name on a leaderboard still receives 403 when opening that peer's record	This is the exact boundary of sanctioned exception E1; without the assertion, the exception would quietly widen.
A second judge's marking does not erase the first's	The previous single-score model lost marks silently, with no way to tell who had given what.
Two identical students differing only in errors receive different forecasts	Equality here would mean the entire quality-sensitivity contribution is inoperative while appearing to work.
Five failed registrations followed by one valid one succeed	A limiter that punishes typographical errors makes the feature unusable while looking like abuse protection.
Reprocessing a session or re-submitting a roll grants no additional XP	Idempotence is a property of the unique index; asserting it prevents a future refactor from removing the index as “redundant”.
A voided fee disappears from every total, not merely from the list	A void that only hides a row is a cancellation on screen and money still present in the figures.
An approved competition rejects marking, registration, editing and deletion	Four separate paths; freezing three of them is not freezing.

13.4 Static quality gates

Table 13.3 — Pre-deployment checks

Gate	Purpose
<code>python -m compileall</code>	Syntax integrity of the entire back end.
<code>python -m pyflakes</code>	Unused and undefined symbols; must print nothing.
<code>tsc -b</code>	Type checking as a build gate. <code>noUnusedParameters</code> is enforced — and once broke the build entirely, because an unused middleware parameter meant <code>npm run build</code> had been failing unnoticed.
<code>eslint</code>	Front-end linting including React hook rules.
<code>pip audit</code> in CI	Dependency vulnerabilities; specified to fail the build on critical findings.
Audit-tag count check	A grep-based confirmation that the security review's fixes are present in the deployed tree before a centre is onboarded.

13.5 Coverage gaps

Stated plainly, because a report that claims complete coverage is not credible:

- **No automated front-end tests.** No component, integration or end-to-end browser suite exists. Interface behaviour is verified manually.
- **No continuous integration pipeline is committed.** The gates above are documented as procedure; they are not enforced automatically on every push.
- **No load or performance testing.** The index strategy and the bulk-query patterns are reasoned rather than measured; no figure for concurrent users or p95 latency is claimed anywhere in this report.

- **No formal coverage measurement.** The suites target authorisation boundaries and business rules by design; ordinary CRUD paths are covered only where they intersect those boundaries.
- **The forecast has not been validated against outcomes.** What is tested is that quality affects the prediction — not that the prediction is accurate. Accuracy requires a season of real completions (§16.2).

Deployment and Operations

14.1 Deployment topologies

14.1.1 Docker Compose (recommended)

1. Copy `backend/.env.example` to `backend/.env` and complete every field; create a top-level `.env` with the MongoDB root and application credentials and the database name.
2. Place the TLS certificate and key in `ops/tls/` and set the real hostname in `ops/nginx.conf`.
3. Build the front end (`npm run build`) so that Nginx has static assets to serve.
4. `docker compose up -d --build`.
5. Watch the first boot: if `INITIAL_ADMIN_PASSWORD` was not set in a development environment, the generated password is printed **once**.

The stack gives, by construction, a topology in which Mongo and the application are unreachable from the internet: neither publishes a port, and both sit on a network declared `internal: true`. The application container additionally runs with a read-only root filesystem, a `tmpfs` at `/tmp`, all capabilities dropped and `no-new-privileges`.

14.1.2 Single VPS with systemd

The application is installed under `/opt/mishkaat` with its environment file at `/etc/mishkaat/mishkaat.env` (mode 600, owned by root). Three units are supplied: the API service, a one-shot backup service and a timer that runs it daily. The API unit runs under `DynamicUser` with `NoNewPrivileges`, no write access outside `/tmp`, no access to kernel tunables, and an outbound network allow-list restricted to localhost — so the process can reach MongoDB and nothing else.

14.2 Configuration reference

The complete environment variable reference is Appendix A. Three settings deserve emphasis here:

- `TRUST_PROXY` — must be true when, and only when, a reverse proxy under the operator's control sets `X-Forwarded-For`. False behind a proxy means five wrong passwords from one person lock out everybody; true without a proxy means a client can forge the header and evade the limiter entirely.
- `PII_ENCRYPTION_KEY` — must be in a secret manager and backed up. Losing it destroys the encrypted fields permanently.
- `SEED_DEMO_DATA` — must remain false in production, and its absence must be verified during onboarding rather than assumed.

14.3 Backup and disaster recovery



Figure 14.1 — Backup pipeline.

The design point is the key split: a backup-only GPG key pair is generated, and **only the public key is installed on the production host**. The private key stays with the operator, off-host, so a compromise of the server yields encrypted archives that the attacker cannot read. The timer fires daily at 03:17 UTC. A restore drill is documented as a requirement to be performed *before* it is needed, not after.

14.4 Health and observability

Table 14.1 — Monitoring signals

Signal	What to watch and why
GET /api/health	Liveness, unauthenticated. Monitor this <i>and</i> the site root separately, so that an Nginx or TLS failure is distinguishable from an application failure.
GET /api/health/ready	Readiness, including database reachability.
<code>LOGIN_FAILED</code> rate	A spike from one address within the lockout window indicates brute forcing.
<code>PUBLIC_REGISTER_CENTER</code> rate	A spike indicates registration spam.
Backup journal	Alert if the backup unit has not run in more than 26 hours.
TTL collection sizes	<code>revoked_tokens</code> , <code>login_attempts</code> and <code>register_attempts</code> should stay small (< 10k documents). Growth means TTL indexes are missing.
Application logs	Shipped to a central store with ≥ 90 days retention. Every 500 carries a request identifier that correlates the client's response with the server-side traceback.
Audit review	A scheduled weekly review of <code>GET /api/audit-logs</code> , plus periodic chain verification.

14.5 Centre onboarding checklist

A per-centre security checklist is part of the repository and is to be completed **once for every centre** before credentials are handed over. Its sections, in order:

Table 14.2 — Onboarding checklist sections

§	Section	Representative checks
0	Pre-flight	The audit commit is deployed; the back end compiles; static analysis is clean; the cryptography dependency is importable.
1	Environment	Every secret is per-instance and real; <code>.env</code> is git-ignored.
2	Demo accounts	<code>SEED_DEMO_DATA</code> false; <i>verify</i> that no demo user exists in the database.

§	Section	Representative checks
3	Centre creation	Credentials delivered out of band; the manager changes the password on first login (which invalidates other sessions); the centre link is verified on the manager account.
4	Isolation probes	A scripted list of cross-tenant requests from the new manager's token. Every probe must return 403 or an empty list; if any returns 200 with data, stop and redeploy. Probes are repeated as a guardian and as a student.
5	Session lifecycle	Refresh rotation works; logout genuinely revokes; a password change invalidates other tokens; a disabled user is rejected on both paths.
6	Encryption at rest	Inspect the database directly: the encrypted fields must appear as Fernet tokens, not plaintext, and must return decrypted through the API. Confirm the key is in the secret store.
7	Database hardening	Bound to localhost or a private network; a scoped database user; daily encrypted backups; a rehearsed restore; TTL indexes verified present.
8	Proxy and TLS	HTTPS enforced; security headers set at the edge; body size capped; an edge rate-limit zone on the login path.
9	Monitoring	Alert rules in place; weekly audit review scheduled; privacy regime acknowledged — minor data is not exported to third-party analytics.
10	Patch hygiene	Dependency audit in CI; pinned requirements; quarterly platform review.
11	Sign-off	Centre name, onboarding administrator, date; every box checked or explicitly waived with the justification logged in the audit record.

14.6 Scaling considerations

Table 14.3 — Scaling posture

Dimension	Current position and next step
Application processes	Four Uvicorn workers behind Nginx. Scaling horizontally is safe because no correctness-bearing state lives in process memory: rate limiting, notification fan-out and counters are all in MongoDB. The one in-memory item is the forecast model cache, which is per-worker and explicitly advisory.
Database	A single MongoDB instance. The indexes cover every hot path. A replica set is the next step, both for availability and to move heavy analytical reads to a secondary.
Analytical reads	Metrics are computed on read. For a large centre the circle overview and leaderboards load history in bulk, but the cost still grows with volume. If it becomes a constraint, the correct response is a cached <i>snapshot</i> with an explicit computation timestamp — not a stored derived value that silently diverges from its sources.
File storage	GridFS. Files are small and few; if attachment volume grows, an object store with signed URLs is the migration, at the cost of leaving the single-backup property behind.
Live notifications	The stream polls the database, which is correct across workers and requires no broker. At high connection counts a publish/subscribe layer would reduce read load — at the cost of the property that makes the current design robust.

Project Metrics and Development History

15.1 Size metrics

Table 15.1 — Code base size

Measure	Value	Notes
Back-end source (Python)	10,390	Including the entry wrapper and the index migration script.
— domain routers	6,866	22 modules.
— cross-cutting modules	~3,400	17 modules, including 638 lines of Pydantic models and 562 of start-up.
Front-end source (TS/TSX)	15,843	27 pages, 6 role dashboards, 23 components, contexts, services, libraries and types.
Test source (Python)	2,169	7 suites.
Tracked text files	32,346	Excluding lock files and binary assets.
Tracked files	198	Including operational templates and generated icons.

Table 15.2 — Interface and data surface

Measure	Value	Notes
REST endpoints	136	62 GET, 43 POST, 14 PUT, 17 DELETE.
Routers	22	Included in a fixed order so that literal paths precede parameterised ones.
Application collections	28	Plus the two GridFS collections of the <code>files</code> bucket.
Index definitions	71	Created idempotently at start-up, including 9 unique or partial-unique constraints and 8 TTL indexes.
User roles	6	Resolved through a four-level scope model.
Audited action types	35	12 classified as permanent and never purged.
Client routes	26	Each with a role guard mirroring the server.
Performance metrics	5	Plus 2 exposed forecasting features.
Badges / challenges / boards	10 / 4 / 4	Motivation layer.
Isolation assertion sites	314	Across 7 suites.

15.2 Development history

Table 15.3 — Commit distribution

Period	Commits	Character of the work
--------	---------	-----------------------

Period	Commits	Character of the work
May 2026	15	Foundation and the first full security hardening pass: token lifecycle, bcrypt cost, password complexity, twenty-two cross-tenant authorisation fixes, database-backed rate limiting, field-level encryption of personal data, the centre approval workflow, and the first 31 indexes.
September 2026	48	A second review pass (dual-key limiting, hashed reset codes, atomic audit sequencing, the removal of name-based ownership matching, security headers); then the circle-level scope model; then the analytical layer; then a sequence of owner-directed feature batches — competitions with judging panels, teacher evaluation, student and guardian accounts, certificates and centre identity, the closed messaging chain, correctness of administrative acts, and the derived review plan.
Total	63	7 May – 10 September 2026

15.2.1 Development method

Two characteristics of how the work proceeded are visible in the repository and are worth recording, because they explain the shape of the system.

The project pivoted once, early and completely. An initial mobile application built with Expo and React Native reached a working MVP — authentication, dashboards, and CRUD for students, teachers and finance — and was then abandoned in favour of a web application. The current PWA delivery (§12.5) is the resolution of that decision: installable on a phone without a store, from one code base.

Defects were fixed at their root and recorded at the site. Non-obvious decisions carry an inline record stating what the previous behaviour was and why it was wrong. This report draws on those records directly, and they are the reason it can state, for each mechanism, the specific failure it answers rather than a general rationale. The same discipline produced the rule that a leak is closed in the module that decides scope rather than in the endpoint where it was noticed — after patching endpoint by endpoint demonstrably failed.

15.3 Documentation artefacts

Table 15.4 — Documents in the repository

Document	Content
README.md	Requirements, how to run, project layout, features, PWA installation.
docs/PERMISSIONS.md	The authorisation reference. Changing a permission requires changing this document and <code>scope.py</code> together — a rule written because the two had already diverged once.
docs/PERFORMANCE_LAYER.md	Requirement-to-code mapping for the analytical layer, the formulas, the invariant decisions, the calibration table, and what was deliberately not carried over.
SECURITY_CHECKLIST.md	The per-centre onboarding checklist (§14.5).
PRODUCTION_HEALTH_REPORT.md	The security review of 2026-05-22: every vulnerability with its previous behaviour and its fix.
DEPLOYMENT_GUIDE.md	Deployment by Docker Compose and by hand, with the production security checklist.
ops/README.md	The operations runbook: topology, systemd sandboxing, backups, secret hygiene, health monitoring.

Evaluation, Limitations and Future Work

16.1 Evaluation against the objectives

Table 16.1 — Objective attainment

Obj.	Status	Evidence and qualification
01	MET	Fourteen functional domains, 136 endpoints, 28 collections. Complete record management for every entity named in the scope.
02	MET	A four-level scope model in one module, one ownership gate, and 314 assertions testing for leakage rather than success. <i>Qualification:</i> testing proves the boundary for the paths it covers; it is not a proof of absence of leaks, which is why the rule requiring new endpoints to be added to the suite exists.
03	MET	A weighted taxonomy and five metrics that separate quality from quantity, computed at read time from session records.
04	PARTLY	The model is implemented, trained per centre, sign-constrained and fully disclosed. Its accuracy is unmeasured and the system claims none — see §16.2.
05	MET	Four boards, ten badges, four challenges, an append-only idempotent ledger.
06	MET	Field-level encryption, short-lived tokens with revocation, dual-key limiting, a hash-chained audit log, content-sniffed uploads, fail-fast production guards. <i>Qualification:</i> phone numbers remain plaintext by necessity (§10.11).
07	MET	Edit and cancel paths on every administrative record; money voided with a reason and excluded from every total.
08	MET	Arabic RTL interface, six role dashboards, installable PWA, explained empty states. <i>Qualification:</i> no formal usability study has been conducted with the intended teachers.
09	MET	A single-command deployment with isolated networks, sandboxed systemd units, encrypted off-host backups, health endpoints and a per-centre onboarding checklist. <i>Qualification:</i> the restore drill is documented but no evidence of execution is in the repository.

16.2 Limitations

Table 16.2 — Limitations, stated without mitigation

#	Limitation	Detail
L1	The forecast is unvalidated	What has been tested is that recitation quality changes the prediction, which is the contribution claimed. Whether the predicted dates are <i>correct</i> requires a cohort of real completions to compare against. No accuracy figure is presented anywhere, and the source's own “80% within ± 2 weeks” is a design target, not a result.
L2	Every analytical constant is provisional	The severity weights, the mastery ceiling, the smoothing factor, the severity threshold and the window lengths are design choices awaiting calibration against real data. They are in configuration for exactly this reason.
L3	Page counts are an approximation when not entered	Derivation from an āyah range assumes uniform verse density across the mushaf, which is false — early sūrahs have far longer verses than late ones. The teacher can enter the true page count, and should for accuracy.

#	Limitation	Detail
L4	No front-end test suite	Interface regressions are caught by hand.
L5	No load testing	Performance is reasoned from the index and query design; no measured throughput or latency figure is claimed.
L6	Single-language interface	Arabic only. Deliberate (§2.3.2), but it limits adoption outside Arabic-reading administrations.
L7	No offline data entry	The shell is cached but writes require connectivity — a real constraint where circles are held in places with poor coverage.
L8	Notifications are in-application only	A guardian who does not open the application does not learn of an absence. SMS is the channel that would actually reach them in this context.
L9	Naive UTC timestamps	Consistent, but requiring timezone awareness for a multi-region deployment.
L10	The sponsorship page is a prototype	Backed by hard-coded sample data with no endpoint behind it. It is presentation, not function, and is documented as such rather than counted as a feature.
L11	No cross-centre analytics	The activity index measures engagement, not academic outcomes. A supervisory body cannot yet compare centres on memorisation results — deliberately, since that would require reading across tenant boundaries.
L12	Bulk read is limited by scope, not prevented	An authorised insider can extract everything within their own scope. Export is audited; it is not rate-limited.

16.3 Future work

Table 16.3 — Prioritised forward work

Pri.	Item	Scope of the work
1	Forecast validation and calibration	Collect a season of real completions; measure the error distribution; recalibrate the severity weights and <code>d_max</code> against them. Only then may any accuracy claim be made — and it must be published with the model version it refers to.
1	SMS notification channel	Absence alerts and reset codes delivered to a phone. The notification records already exist and are persisted; only the transport is missing.
1	Continuous integration	Run the seven suites, the static gates and the dependency audit on every push. The procedure exists; the automation does not.
2	Offline write queue	Queue recitation and attendance entries in the service worker and reconcile on reconnection. Idempotency keys already exist for both — the once-per-day keys make replay safe, which is the hard part.
2	Front-end test suite	Component tests for the error tagger, the account modal and the certificate sheet; end-to-end coverage of the role guards.
2	PII key rotation tooling	A re-encryption pass, written before a rotation is ever attempted rather than during one.
2	Timezone-aware timestamps	A data migration: every stored row and every comparison. Not a function substitution.
3	Full English interface	A broad mechanical translation pass, deserving its own review.
3	Qur'anic corpus integration	A verse-level database would make page counts exact, validate āyah ranges, and enable sūrah-accurate review portions.

Pri.	Item	Scope of the work
3	Replica set and read splitting	Availability, plus moving analytical reads to a secondary.
3	Server-side document generation	Certificates as PDFs generated server-side, so that a certificate is a stored artefact rather than a browser print.
3	Payment integration	Mobile money for fee collection. It changes the regulatory posture of the system and should be scoped as a separate project (§2.2).

Conclusion

Mishkaat set out to do two things: to give a Qur'an memorisation centre a single, auditable system for its academic, administrative and financial work, and to make the *quality* of memorisation — not merely its quantity — measurable, comparable and explainable. Both are delivered in the system documented here, and the second is what distinguishes it from a school-administration product with Arabic labels.

The engineering position that the system takes, and which this report has traced through every chapter, can be stated in five sentences.

1. **Authorisation is a design problem, not a set of checks.** Seventeen data leaks came from one root cause — a system that knew about centres but not about circles — and patching them individually returned them with the next endpoint. Defining the four scope levels in one module, gating every student identifier through one function, and asserting the boundary with tests that look for leakage rather than for success is what actually closed them.
2. **Derived values must not be stored.** Every metric, rank and forecast is a function of the session records at the moment of the query, which is what makes a student's position explainable by pointing at their recitations. A stored metric would diverge from its sources at the first correction and take the explicability of the whole motivation layer with it.
3. **Uniqueness that matters belongs in the database.** A check followed by a write is not atomic, and the duplicate it fails to prevent is not a cosmetic duplicate in a list: it is a salary paid twice, a certificate issued twice, a metric inflated against the truth.
4. **Every administrative act is fallible, and money is voided rather than erased.** A system with no correction path forces its users to work around it, and the records then stop describing reality — which is the failure the software was adopted to fix.
5. **Claims must be proportionate to evidence.** The forecast returns its method, its training size and its fit, and no confidence figure, because none has been measured. A coefficient that contradicts the domain is zeroed rather than inverted. A metric with no data returns “no data”, not zero. A system that tells a parent something it cannot support loses the standing it needs to say anything at all.

What remains is honest and specific. The forecasting model is implemented and quality-sensitive but its accuracy is unvalidated; the analytical constants are provisional by declaration and await calibration against real memorisation data; there is no automated front-end testing and no measured performance envelope; and notifications do not yet reach a guardian who is not looking at the application. These are stated in Chapter 16 with the work each would require, because a report that presents a system as complete is less useful than one that says precisely where the edges are.

The system is, on the evidence assembled in this report, ready for supervised production use in a centre whose operator completes the onboarding checklist of §14.5 — and it is ready to be measured, which is the precondition for the calibration work that comes next.

References and Sources

Primary sources — the system itself

1. Mishkaat source repository, `asse-byte/mishkaat`, application version 2.1.0. Every factual statement in Chapters 5–14 is drawn from this code base: the router and module inventories, the endpoint catalogue, the data dictionary, the index definitions, the formulas and constants of the analytical engine, the security mechanisms, and the deployment manifests.
2. `docs/PERMISSIONS.md` — the authorisation reference: the scope model, the capability matrix, the messaging chain, the sanctioned exceptions and the rules that are not to be broken.
3. `docs/PERFORMANCE_LAYER.md` — the requirement-to-code mapping of the analytical layer, its formulas, its invariant decisions and its calibration table.
4. `PRODUCTION_HEALTH_REPORT.md` — security review of 22 May 2026, documenting each vulnerability's previous behaviour and its fix.
5. `SECURITY_CHECKLIST.md` — the per-centre pre-onboarding security procedure.
6. `DEPLOYMENT_GUIDE.md` and `ops/README.md` — deployment topologies, sandboxing, backup pipeline and operational monitoring.
7. Inline decision records throughout `backend/app/` and `frontend/src/`, each stating the behaviour that preceded a change and why it was wrong.

External sources

8. *Halaqtna* — graduation project report, CPIT-498, Faculty of Computing and Information Technology, King Abdulaziz University. Source of the analytical layer's requirements FR6–FR18, of the five metrics, of the three-feature forecasting design, of the multi-criteria motivation model, and of the provisional constants of its Table 3.4. Its own statement that these constants are provisional and that its accuracy target is a design goal is carried into this system and this report.
9. Hassan, S. and Zailaini, M. A. — classification of recitation and tajwīd errors in Qur'an memorisation assessment; the basis for a closed, comparable error taxonomy in place of free-text error notes.
10. OWASP Application Security Verification Standard and the OWASP API Security Top Ten — the basis for the treatment of broken object level authorisation (BOLA/IDOR), the password-storage guidance (bcrypt cost 12) and the session-management controls documented in Chapter 10.
11. FastAPI, Pydantic, Motor, MongoDB, React, Vite and Tailwind CSS project documentation, at the versions pinned in `backend/requirements.txt` and `frontend/package.json`.
12. Web application manifest and service worker specifications (W3C) — the basis for the Progressive Web App delivery described in §12.5.

ON THE PROVENANCE OF FACTUAL CLAIMS

Counts and measurements in this report (lines of code, endpoint totals, collection and index counts, assertion counts, commit distribution) were taken directly from the repository at the state of 11 September 2026. Where a count derived from the source differs from a figure stated in the project's own documentation, both are given and the difference explained — see §13.2.

Environment Variable Reference

Table A.1 — Runtime and security configuration

Variable	Default	Required	Effect
APP_ENV	development	always	<code>production</code> enables every fail-fast guard, disables the interactive API documentation and its schema, marks the refresh cookie <code>Secure</code> , and adds HSTS.
SECRET_KEY	built-in	prod	JWT signing and reset-code HMAC key. The built-in default aborts the boot in production. Generate with <code>secrets.token_hex(64)</code> .
PII_ENCRYPTION_KEY	—	prod	Fernet key for student personal data. Absent in production aborts the boot; in development an ephemeral key is generated with a warning that the data will not survive a restart. Losing it destroys the encrypted fields permanently.
MONGO_URL	mongodb://localhost:27017	always	Connection URI. Use a scoped application user, never the root account.
DB_NAME	quran_center	always	Database name.
ALLOWED_ORIGINS	localhost:3000, :5173, 127.0.0.1:3000	always	Comma-separated origins; surrounding whitespace is stripped. <code>*</code> aborts the boot in production because credentials are enabled.
TRUST_PROXY	false	—	Honour <code>X-Forwarded-For</code> . True only behind a proxy you control; false behind one locks out every user after five failures from one person, true without one lets a client forge the header.
INITIAL_ADMIN_PASSWORD	—	first prod boot	Bootstraps the administrator. In development a random password is generated and printed once.
INITIAL_SUPER_ADMIN_PASSWORD	—	first prod boot	Bootstraps the super-administrator. Must differ from the administrator's — a leak of one must not be a leak of both.
ADMIN_EMAIL	admin@quran-center.com	—	Administrator contact address.
SEED_DEMO_DATA	false	—	Seeds demo users, centres, circles and students. Never true in production; absence of demo accounts is a checklist item.

Variable	Default	Required	Effect
ACCESS_TOKEN_EXPIRE_MINUTES	60	—	Access token lifetime; lower for higher-sensitivity deployments.
REFRESH_TOKEN_EXPIRE_DAYS	7	—	Refresh token lifetime, and the cookie's max-age.
BCRYPT_ROUNDS	12	—	Password hashing cost. Raise to 13–14 where CPU allows.
MAX_ACCOUNT_ATTEMPTS	10	—	Per-account failure limit — what stops an attacker who rotates IP addresses. The per-IP limit of 5 per 15 minutes is fixed.
ACCOUNT_LOCKOUT_MINUTES	15	—	Per-account lockout duration.
RESET_CODE_TTL_MINUTES	10	—	Reset-code lifetime; five wrong guesses invalidate it regardless.
UVICORN_WORKERS	4	—	Worker processes (container environment).

Table A.2 — Analytical calibration (all optional)

Variable	Default	Effect
MASTERY_DMAX	3.0	Weighted error density at which mastery reaches zero.
MOMENTUM_ALPHA	0.4	Exponential smoothing factor for momentum.
HIGH_SEVERITY_THRESHOLD	0.5	Weight at or above which an error counts as severe.
CONSISTENCY_WINDOW_SESSIONS	8	Sessions in the consistency window.
REVIEW_DEPTH_WINDOW_DAYS	28	Review-depth window.
XP_PER_PAGE	10	Points per page recited.
XP_PER_SESSION	5	Points per attended session.
VERSES_PER_PAGE	6236/604	Āyah-to-page derivation.
TOTAL_QURAN_PAGES	604	Mushaf page count.
TOTAL_QURAN_VERSES	6236	Mushaf āyah count.

Table A.3 — Container stack variables (`docker-compose.yml`)

Variable	Effect
MONGO_ROOT_USER / _PASSWORD	MongoDB root credentials; required, with no default, so the stack cannot start unconfigured.
MONGO_APP_USER / _PASSWORD	The scoped application user created on first boot by <code>ops/mongo-init.js</code> . These must reach the Mongo container: when they did not, the init script skipped user creation and the application then failed authentication, so the whole stack never came up.
RETENTION_DAYS	Backup retention (default 30).
GPG_RECIPIENT	Backup encryption recipient. Only the <i>public</i> key is installed on the production host.
RCLONE_REMOTE	Optional off-host backup destination.

Audit Action Catalogue

Thirty-five action types are written to the hash-chained audit log. Those marked **PERM** are classified permanent: they carry no `purge_at` and are never removed. All others expire 730 days after they are written.

Table B.1 — Audited actions

Action	Class	Recorded when
Authentication and credentials		
LOGIN_SUCCESS	2 years	A successful authentication.
LOGIN_FAILED	2 years	A failed attempt. A spike from one address within the lockout window is the brute-force signal.
LOGIN_DENIED_PENDING	2 years	A manager of an unapproved centre attempted to log in.
LOGOUT	2 years	Tokens revoked and the cookie cleared.
PASSWORD_CHANGED	PERM	Self-service change; invalidates every other session.
PASSWORD_RESET_VIA_CODE	PERM	Reset completed with a valid code.
ADMIN_RESET_PASSWORD	PERM	An administrator set another user's password.
PROFILE_UPDATED	2 years	Name, email or phone changed.
ACCOUNT_ISSUED	2 years	A student or guardian account was created by the manager.
ACCOUNT_PASSWORD_RESET	2 years	A student or guardian password was replaced.
Tenancy and governance		
PUBLIC_REGISTER_CENTER	2 years	A public self-registration; a spike is the spam signal.
ADMIN_REGISTER_CENTER	2 years	A centre created by an administrator.
SUPER_REGISTER_CENTER	PERM	A centre created by the super-administrator.
CENTER_APPROVED	PERM	A pending centre was activated.
CENTER_REJECTED	PERM	An application was refused.
SUPER_CENTER_STATUS_CHANGE	PERM	A centre suspended or reactivated.
CENTER_LOGO_UPDATED	2 years	Centre identity changed.
EXPORT_STUDENTS	2 years	A CSV export of student data was taken.
Finance — all permanent		
collect_fee	PERM	A fee was collected.
update_fee	2 years	A fee record was corrected.
void_fee	2 years	A fee was voided with a written reason.
pay_salary	PERM	A salary was paid.
update_salary	2 years	A salary record was corrected.

Action	Class	Recorded when
void_salary	2 years	A salary was voided.
create_expense	PERM	An expense was recorded.
update_expense	2 years	An expense was corrected.
delete_expense	PERM	An expense was voided.
Academic and honours		
UPDATE_RECITATION	2 years	A session was corrected.
DELETE_RECITATION	2 years	A session was soft-deleted.
UPDATE_ATTENDANCE	2 years	An attendance status was corrected.
DELETE_ATTENDANCE	2 years	An attendance row was removed.
TEACHER_TRANSFERRED	2 years	A teacher moved between circles; the entry names both.
COMPETITION_CREATED	2 years	A competition was created; the entry records the panel size.
COMPETITION_APPROVED	2 years	Results approved; the competition froze.
CERTIFICATE_ISSUED	2 years	A certificate was issued; the entry records its serial.
CERTIFICATE_REVOKED	2 years	A certificate issued in error was revoked.

Complete Endpoint Index

All 136 endpoints, in the order the routers are mounted. Router order is deliberate: literal paths are registered before parameterised ones, so that `/api/centers/{id}` cannot swallow `/api/centers/pending`.

Table C.1 — Endpoint index (1 of 2)

Method	Path	Method	Path
auth (10) · audit_logs (2) · notifications (5) · dashboard (4)			
POST	/api/auth/login	POST	/api/auth/refresh
GET	/api/auth/me	POST	/api/auth/logout
POST	/api/auth/change-password	POST	/api/auth/forgot-password
GET	/api/auth/reset-codes	POST	/api/auth/reset-password
POST	/api/auth/admin-reset-password	PUT	/api/auth/profile
GET	/api/audit-logs	GET	/api/audit-logs/verify
GET	/api/notifications	POST	/api/notifications/{id}/read
POST	/api/notifications/read-all	POST	/api/notifications/stream-ticket
GET	/api/notifications/stream	GET	/api/dashboard/stats
GET	/api/dashboard/honor-roll	GET	/api/analytics/rankings
GET	/api/analytics/student/{id}		
centers (17) · center_activity (1)			
GET	/api/centers	GET	/api/public/best-centers
GET	/api/admin/system/status	POST	/api/public/register-center
POST	/api/centers/{id}/approve	POST	/api/centers/{id}/reject
GET	/api/centers/pending	POST	/api/centers
PUT	/api/centers/{id}	POST	/api/centers/{id}/logo
GET	/api/centers/{id}/logo	DELETE	/api/centers/{id}/logo
DELETE	/api/centers/{id}	GET	/api/centers/{id}/details
POST	/api/super/centers	POST	/api/super/centers/{id}/status
GET	/api/super/dashboard/stats	GET	/api/admin/centers-activity
students (5) · accounts (3) · teachers (8)			
GET	/api/students	GET	/api/students/{id}
POST	/api/students	PUT	/api/students/{id}
DELETE	/api/students/{id}	GET	/api/students/{id}/accounts
POST	/api/students/{id}/accounts	POST	/api/students/{id}/accounts/reset
GET	/api/teachers	POST	/api/teachers
PUT	/api/teachers/{id}	POST	/api/teachers/{id}/transfer
DELETE	/api/teachers/{id}	POST	/api/teachers/{id}/evaluations
GET	/api/teachers/{id}/evaluations	DELETE	/api/teachers/{id}/evaluations/{eid}
halaqat (4) · halaqat_overview (3) · schedules (4)			
GET	/api/halaqat	POST	/api/halaqat
PUT	/api/halaqat/{id}	DELETE	/api/halaqat/{id}
GET	/api/halaqat-overview	GET	/api/halaqat-overview/ranking
GET	/api/halaqat-overview/{id}	POST	/api/academic-schedules
GET	/api/academic-schedules	PUT	/api/academic-schedules/{id}
DELETE	/api/academic-schedules/{id}		

Table C.2 — Endpoint index (2 of 2)

Method	Path	Method	Path
competitions (13) · certificates (5)			
GET	/api/competitions/branches	GET	/api/competitions/my-role
POST	/api/competitions	GET	/api/competitions
GET	/api/competitions/years	PUT	/api/competitions/{id}
POST	/api/competitions/{id}/approve	POST	/api/competitions/{id}/archive
POST	/api/competitions/{id}/register	GET	/api/competitions/{id}/contestants
POST	/api/competitions/contestants/{id}/grade	DELETE	/api/competitions/contestants/{id}
DELETE	/api/competitions/{id}	GET	/api/certificates
GET	/api/certificates/eligible	POST	/api/certificates/milestone
POST	/api/certificates/competition/{cid}	DELETE	/api/certificates/{id}
messages (9) · recitations (6) · performance (6)			
GET	/api/messages/audiences	POST	/api/messages/attachments
POST	/api/messages/broadcast	GET	/api/messages/inbox
GET	/api/messages/broadcasts	POST	/api/messages/{id}/reply
PUT	/api/messages/{id}	DELETE	/api/messages/{id}
GET	/api/messages/{id}/attachments/{fid}	GET	/api/recitations
POST	/api/recitations	GET	/api/recitations/student/{id}
PUT	/api/recitations/{id}	DELETE	/api/recitations/{id}
GET	/api/analytics/predict/{id}	GET	/api/performance/error-types
GET	/api/performance/students/{id}	GET	/api/performance/students/{id}/journey
GET	/api/performance/leaderboards	GET	/api/performance/challenges/{id}
POST	/api/performance/recompute		
attendance (7) · finance (15)			
GET	/api/attendance	GET	/api/attendance/halaqah/{id}
POST	/api/attendance	GET	/api/attendance/student/{id}
GET	/api/attendance/center	PUT	/api/attendance/{id}
DELETE	/api/attendance/{id}	GET	/api/fees
POST	/api/fees	POST	/api/fees/{id}/pay
PUT	/api/fees/{id}	DELETE	/api/fees/{id}
GET	/api/fees/student/{id}	GET	/api/salaries
POST	/api/salaries	PUT	/api/salaries/{id}
DELETE	/api/salaries/{id}	GET	/api/expenses
POST	/api/expenses	PUT	/api/expenses/{id}
DELETE	/api/expenses/{id}	GET	/api/finance/summary
review_plans (5) · exports (2) · health (2)			
GET	/api/review-plans	POST	/api/review-plans
PUT	/api/review-plans/{id}	DELETE	/api/review-plans/{id}
GET	/api/review-plans/today	GET	/api/export/students.csv
GET	/api/export/attendance.csv	GET	/api/health
GET	/api/health/ready		

Glossary of Domain Terms

Table D.1 — Arabic and domain terminology used in this report

Term	Arabic	Meaning in this system
Mishkaat	المشكاة	The system's name; the niche in a wall that holds a lamp.
Halaqah (pl. ḥalaqāt)	حلقة / حلقات	A study circle: a group of students with one teacher. In Mishkaat it is a first-class authorisation scope, not a subdivision of a centre.
Muḥaffiẓ	محفّظ	The circle teacher — literally “the one who causes memorisation”. Modelled as the <code>teacher</code> role with <code>teacher_type: halaqah</code> .
Tasmī‘	تسميع	Recitation from memory to the teacher; the central academic event and the source of every performance metric.
Murāja‘ah	مراجعة	Review of previously memorised material, as opposed to new memorisation. Recorded as <code>recitation_type: review</code> .
Juz’ (pl. ajzā’)	جزء / أجزاء	One of the thirty parts of the Qur’an; ≈ 20.13 pages of the 604-page mushaf.
Juz’ ‘Amma	جزء عمّ	The thirtieth part; the entry-level competition branch.
Khatm / Khatmah	ختم / ختمة	Completion of the entire Qur’an; the highest milestone and the highest badge.
Hāfiẓ	حافظ	One who has memorised the Qur’an.
Tajwīd	تجويد	The rules of correct recitation — articulation, elongation, stopping. One error class and one competition criterion.
Mushaf	مصحف	The printed copy of the Qur’an. The reference edition here has 604 pages and 6,236 āyāt.
Āyah (pl. āyāt)	آية / آيات	A verse. Recitation ranges are recorded in āyāt and converted to pages.
Sūrah	سورة	A chapter; 114 in total.
Hafṣ ‘an ‘Āṣim	حفص عن عاصم	A transmission of the Qur’anic reading; recorded as a teacher's specialisation.
Walī al-amr	وليّ الأمر	The guardian; the <code>parent</code> role.
Dār / markaz taḥfīẓ	دار / مركز تحفيظ	A Qur’an memorisation centre; the tenant of this multi-tenant system.
FCFA	—	West African CFA franc, the default currency.
Technical terms		
BOLA / IDOR	—	Broken object level authorisation: reading another party's record by supplying its identifier. The threat that <code>check_student_access</code> exists to prevent.
Scope	النطاق	The set of records a user may see, resolved at four levels: system, centre, circle, self.
Fail closed	—	Absent assignment means denial. An empty scope yields nothing, never everything.
Void	إلغاء	Cancelling a financial record by marking it with a reason and excluding it from every total — never by deletion.

Term	Arabic	Meaning in this system
Soft delete	حذف ناعم	Marking an academic record deleted so that it leaves every computation while remaining available for audit.
TPI	—	The teacher evaluation total: five criteria scored out of ten each, summing to one hundred.
Momentum	الزخم	Exponentially smoothed new-memorisation pages per week.
Mastery	الإتقان	An index derived from weighted error density per page.
Precision	الدقة	The inverse share of severe errors among tagged errors.
Consistency	الانتظام	Attendance across the last eight sessions.
Review depth	عمق المراجعة	Review pages relative to new pages over 28 days, capped at 200%.
XP ledger	—	The append-only experience-point record, made idempotent by a unique key on (student, reason, source).
PWA	—	Progressive Web App: installable from the browser without an application store.
GridFS	—	MongoDB's binary storage; holds centre logos and message attachments.
TTL index	—	An index that deletes documents automatically after a period; used for every transient security collection and for notification retention.
JTI	—	The unique identifier of a token, used for server-side revocation.
Fernet	—	The authenticated symmetric encryption scheme (AES-128-CBC with HMAC-SHA256) used for student personal fields.

End of report.

Mishkaat — Scientific & Engineering Report, version 1.0, 11 September 2026.

Copyright © 2026 Abdoul Malick Cisse. Proprietary and confidential. All rights reserved.